

Generative AI Through CAS Lens: An Integrated Overview of Algorithmic Optimizations, Architectural Advances, and Automated Designs

Chuan Zhang¹, Senior Member, IEEE, You You², Naigang Wang³, Jongsun Park⁴, Senior Member, IEEE, and Li Zhang⁵, Senior Member, IEEE

Abstract—Generative artificial intelligence (GenAI) has emerged as a pivotal focus in global innovation agendas, revealing transformative potential that extends beyond technological applications to reshape diverse societal domains. Given the fundamental dependency of GenAI deployment on circuits and systems (CAS), a co-evolutionary approach integrating both technological paradigms becomes imperative. This synergistic framework confronts three interrelated challenges: 1) developing deployment-ready GenAI algorithms, 2) engineering implementation-efficient CAS architectures, and 3) leveraging GenAI for autonomous CAS designs - each representing critical innovations vectors. Given the rapid advancement of GenAI-CAS technologies, a comprehensive synthesis has become an urgent priority across academia and industry. Consequently, this timely review systematically analyzes current advancements, provides integrative perspectives, and identifies emerging research trajectories. This review endeavors to serve both AI and CAS communities, thereby catalyzing an innovation feedback loop: GenAI-optimized CAS architectures in turn accelerate GenAI evolution through algorithm-hardware co-empowerment.

Index Terms—Generative artificial intelligence (GenAI), circuits and systems (CAS), algorithms, architectures, autonomous design.

I. INTRODUCTION

GENERATIVE artificial intelligence (GenAI) has become one of the most prominent and widely discussed topics in recent years. Its most renowned application, ChatGPT [1], becomes the first non-human to be included in the Nature's 10 [2], acknowledging the role of GenAI in the development and progress of science. ChatGPT and the upcoming DeepSeek's [3] remarkable success epitomizes the immense potential of

GenAI, demonstrating its transformative power not just in technology, but across multiple sectors of society. GenAI is gaining traction in both industry and academia due to its impressive capabilities.

- For industry, GenAI is now used in many fields, like drug discovery [4], financial services [5], telecommunications [6], energy [7], education [8], public sector [9], media [10] and entertainment [11], and so on. Numerous large language models (LLMs) have surfaced besides the aforementioned ChatGPT and DeepSeek, including: Meta's LLaMA [12], Google's Gemini [13], NVIDIA's Nemotron [14], Alibaba's Qwen [15], and so on.
- In academia, GenAI has shown great potential in supporting the new AI for Science (AI4S) paradigm [16]. Current applications include research topic selection [17], literature investigation [18], data analysis [19], manuscript review [20], and so on. Academic societies have issued multiple guidelines [21], [22], [23] governing the use of GenAI, aiming to uphold academic ethics.

As GenAI deployment remains inherently interdependent with yet gapped from circuits and systems (CAS), the exponential scaling of GenAI architectures introduces escalating implementation barriers to their operational deployment in real-world settings. Among the prevalent methods of AI implementation, options such as graphics processing units (GPUs) [24], [25], [26], field programmable gate arrays (FPGAs) [27], [28], [29], and application-specific integrated circuits (ASICs) [30], [31], [32] are commonly utilized, each with its distinct limitations. Implementations harnessing GPUs are often hampered by their substantial power consumption and comparatively lower efficiency. On the other hand, FPGAs, despite their versatility, struggle to accommodate the sheer size of cutting-edge generative models. ASIC-based solutions, including Google's pioneering tensor processing units (TPUs) [33], have chalked up noteworthy successes but are hamstrung by inflexibility, making them less adaptable to fast-evolving generative models. Contemporary GenAI implementations face several critical challenges:

- **Resource limitations:** The rapid escalation in the complexity of generative models increasingly exceeds the computational capabilities of contemporary hardware platforms.

Received 4 March 2025; revised 27 April 2025; accepted 12 May 2025. Date of publication 6 June 2025; date of current version 16 June 2025. This work was supported in part by NSFC under Grant 62331009 and Grant 62401642, in part by Jiangsu Provincial NSF under Grant BG2024004 and Grant BM2023016, and in part by the Fundamental Research Funds for the Central Universities. This article was recommended by Guest Editor W.-H. Peng. (Corresponding author: Chuan Zhang.)

Chuan Zhang and You You are with LEADS, the National Mobile Communications Research Laboratory, and the Frontiers Science Center for Mobile Information Communication and Security, Southeast University, Nanjing 211189, China, and also with Purple Mountain Laboratories, Nanjing 211100, China (e-mail: chzhang@seu.edu.cn).

Naigang Wang is with the IBM T. J. Watson Research Center, Yorktown Heights, NY 10598 USA (e-mail: nwang@us.ibm.com).

Jongsun Park is with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: jongsun@korea.ac.kr).

Li Zhang is with the School of Electronic and Electrical Engineering, University of Leeds, LS2 9JT Leeds, U.K. (e-mail: l.x.zhang@leeds.ac.uk).

Digital Object Identifier 10.1109/JETCAS.2025.3575272

- **Architectural rigidity:** Prevailing hardware architectures, often designed with a general-purpose paradigm, lack the adaptability required to accommodate the heterogeneous demands of diverse AI workloads.
- **Energy constraints:** The exponential growth in computational requirements presents significant obstacles to energy efficiency, rendering many existing systems unsustainable under stringent power budgets.

Addressing these challenges necessitates coordinated advancements in both algorithmic design and hardware architecture. Concisely, implementation-oriented optimization of algorithms, and efficient hardware architectures for GenAI. Moreover, accelerating hardware design with GenAI is highly regarded as well [34].

Algorithm optimization for efficient GenAI deployment is essential. Besides the architectural breakthroughs of GenAI [3], [35], [36], [37], researchers are actively pursuing strategies to simplify generative models, aiming to improve efficiency without compromising performance. For instance, quantization [38] reduces the precision of parameters, pruning [39] removes unnecessary parameters, and knowledge distillation [40] lowers computational overhead. Essentially, by leveraging data patterns like sparsity, model compressions are performed. These techniques have successfully downsized large generative models. However, downsizing has its limitations. With the rapid development of GenAI, downsized models remain resource-intensive, imposing considerable demands for hardware. Another approach is to conduct hardware-friendly optimization for GenAI. The optimization takes algorithm implementations into consideration. Various optimization methods have been developed for general-purpose processors [41], [42], while others are tailored for hardware accelerators [43], [44]. Optimizing algorithms for hardware compatibility is crucial for the efficient deployment of GenAI models, it is an intriguing subject for future research.

Circuits and systems for GenAI implementation play a crucial role in meeting the growing computational demands of generative models. As generative models grow in complexity, the demand for computational power continues to rise. Companies like NVIDIA [45] and Qualcomm [46] are developing scalable, flexible, and efficient hardware solutions to meet these needs. A key advancement is the rise of embedded AI [47], which reduces reliance on cloud computing by processing data locally. This not only reduces latency but also improves efficiency in real-time applications such as edge computing [48] and autonomous systems [49], [50]. Despite these strides, specialized neural network accelerators like TPUs and ASICs are still needed to handle the massive computational demands of next-generation AI models. While GPUs have been optimized for AI tasks, they are not always sufficient for the complex, parallel processing required by large-scale models such as GenAI. Another critical challenge is power consumption. Training and deploying large AI models can be energy-intensive, raising concerns about operational costs and environmental impact. To address this, energy-efficient computing paradigms, including low-power circuits and adaptive computing, are becoming essential. Research in these areas aims to balance high performance with

sustainability. Although progress has been made, the evolving needs of GenAI will require continued innovation in hardware design, particularly in developing efficient circuits and systems to support future AI applications.

GenAI for efficient circuits and systems design is promising. Recent advances in GenAI, including LLMs, show tremendous promise to transform the CAS field [51]. For example, LLMs can generate code for hardware architectures [52], automate drafting of technical documentation [53], and many more. Additionally, GenAI holds promise for enhancing electronic design automation (EDA) tools [54], optimizing designs [55], and expediting verification processes [56]. GenAI further promises to lower the technical barriers for CAS design with its tremendous knowledge pool [57]. Despite all these promising future applications, these technologies need to be more reliable, efficient, and flexible, which remains challenging for researchers [52]. On the one hand, GenAI models require substantial training data and computational resources, which are not always available [58]. On the other hand, GenAI models are not always accurate. For instance, LLMs can generate plausible but incorrect text or code, requiring human validation [59]. Making LLMs domain-specific and building user trust remains an open area.

The remainder of the paper is organized as follows. Section II introduces the background of GenAI. Section III provides an overview of algorithm optimization for GenAI deployment. Section IV reviews CAS for GenAI implementation, and Section V explores GenAI for efficient CAS design. Section VI discusses the future directions of GenAI-CAS technologies. Finally, Section VII concludes the paper.

II. BACKGROUND

GenAI represents a class of AI systems specifically designed to create and synthesize new data across various modalities, including images, text, and videos. In recent years, significant progress in GenAI has been driven by rapid advancements in deep learning technologies. Leveraging these foundational techniques, numerous generative models have been developed, tailored to diverse applications and challenges.

A. Generative Adversarial Networks (GANs)

GANs [35] are a kind of deep learning frameworks celebrated for their capability to generate realistic synthetic data. A GAN consists of two parts: a generator network and a discriminator network. GANs are trained in an iterative process, where the discriminator network judges if the given data is genuine or synthesized, while the generator network strives to generate samples realistic enough to fool the discriminator network. Despite their extensive potentials in fields such as computer vision and healthcare, GANs often face challenges such as unstable training dynamics and mode collapse [60].

B. Variational Autoencoders (VAEs)

VAEs [36] enhance classical autoencoder architectures by incorporating probabilistic modeling into the learning process. Rather than mapping input data to fixed points in latent space,

VAEs learn a latent probabilistic distribution. The encoder transforms input data into parameters of this latent distribution, while the decoder samples from the predicted distribution to reconstruct data samples. The encoder and decoder are trained so that the reconstruction loss is minimized while the output of the encoder is forced to resemble a simple prior distribution.

C. Diffusion Models

Diffusion models, initially known as the denoising diffusion probabilistic models (DDPMs) [37], frame the process of data generation as a reverse-time Markov chain that performs iterative denoising to produce the generated data. In diffusion models, a forward network adds noise, typically Gaussian, to the input data, while a denoising network is trained to recover increasingly less corrupted versions of the original data. DDPMs exhibit high capability in generating realistic videos and images.

D. Large Language Models

LLMs refer to advanced machine learning models trained on extensive text datasets spanning multiple domains. Typically implemented with transformer-based architectures, these models contain parameters numbering in the billions, allowing them to learn complex language representations. Unlike smaller-scale models such as BERT [61], LLMs exhibit advanced emergent capabilities, such as in-context learning. GPT-3 [62] was one of the first LLMs to demonstrate such abilities. In recent years, advancements in LLMs have gained momentum through breakthroughs and novel approaches in model architectures and training methodologies, producing representative models such as GPT-4 [63], LLaMA [12], OPT [64], Deepseek-R1 [3], and CodeGen [65], among others. Numerous LLMs can be fine-tuned for specific domains, such as programming or solving mathematical problems. Additionally, their performance can be enhanced by incorporating user feedback, enabling continuous improvements in functionality and application-specific expertise.

III. ALGORITHM OPTIMIZATION FOR GENAI DEPLOYMENT

The rapid advancement of GenAI has outpaced current hardware capabilities, particularly in terms of computility, power consumption, and memory capacity. Consequently, deploying GenAI models on existing platforms becomes increasingly challenging. Algorithm optimization for GenAI offers a cost-effective solution with superior adaptability. Its potential has attracted extensive research interest. This section highlights recent advances in GenAI from three key perspectives: the evolution of GenAI models, model compression techniques, and hardware-friendly optimization.

A. Generative Models

The developments of GenAI have been driven by a series of architectural breakthroughs, progressing from GANs and VAEs to diffusion models and LLMs. Concurrently, numerous optimization techniques have emerged alongside these

architectural advancements, serving as a critical enabler for advancing GenAI's processing power, computational economy, and architectural flexibility, as summarized in Table I.

1) *Generative Adversarial Networks*: Researchers have made substantial progress in optimizing GANs, focusing on enhancing training efficiency, stability, and performance. Consistency-GAN [66] accelerated GAN training by introducing instance noise and utilizing a consistency model. This method significantly decreased required sampling iterations, resulting in markedly accelerated generation speeds. Building on this, adversarial error correction GAN (AEC-GAN) [67] addressed key challenges in generating long temporal sequences. By incorporating an error correction module and adversarial sample augmentation, this model effectively mitigated issues such as distribution shift and bias amplification. Furthermore, Pandey [68] has successfully integrated evolutionary algorithms (EA) with GANs. This hybrid approach leveraged a neural evolution technique that optimized both EA and backpropagation, thereby improving the performance of multi-generator GAN architectures. By incorporating evolutionary strategies, this method introduced a novel layer of optimization to GANs. To address training instability, AdaptiveMix method [69] enhanced the robustness of training samples, providing a solution to one of the most persistent challenges in GAN training.

Building on the foundational work in GAN algorithm optimization, recent research has also explored adapting and enhancing these models for specialized applications. Uncertainty-aware GAN (UGAN) [70] tackled the challenge of single-image super-resolution (SISR) by integrating uncertainty estimation into the GAN framework. This approach enhances the quality of reconstructed images, providing a more reliable solution for high-resolution image restoration. Zeng et al. [71] proposed an enhanced GAN-based model for image inpainting, specifically designed to handle high-resolution images. This approach leverages GANs to restore missing or damaged regions, resulting in higher fidelity and more detailed completed images. Similarly, SmartGD [72], a novel framework for image generation, focused on optimizing diverse quantitative aesthetic objectives, regardless of their differentiability. By offering this flexibility, SmartGD effectively tackled a wide range of image generation tasks, ultimately improving the perceptual quality of synthesized outputs. Additionally, the structure-aware image composition GAN (SAC-GAN) [73] introduced a novel end-to-end architecture that advanced the state of image-to-image translation. This framework enabled the seamless integration of object patches into background images, resulting in synthesized images that were both contextually accurate and visually coherent, thereby improving the overall realism and quality of the generated images.

While significant research has concentrated on fine-tuning GANs for image-centric applications, such as restoration, super-resolution, and generation, recent studies have successfully extended these models to tackle more intricate generation tasks across diverse domains. By combining self-attention mechanisms with GCNs, the SG-GAN model [74] is able to generate high-quality three-dimensional (3D) point clouds

TABLE I
TAXONOMY OF GENAI ALGORITHM OPTIMIZATION

GANs	Models	Model algorithm optimization and training improvements	[66] [67] [68] [69]
	Applications	Image-related tasks optimization	[70] [71] [72] [73]
		Extension to complex generation tasks	[74] [75] [76] [77]
		Integration of generative models with other techniques	[78]
VAEs	Models	Model and architecture optimization	[79] [80] [81]
	Extensions	Practical applications of VAEs	[82] [83] [84]
		Latent variable learning and representation efficiency	[85] [86]
LLMs	Efficiency	Training acceleration and efficiency	[87] [88] [89]
		Fine-tuning and inference optimization	[90] [91] [92]
		System and framework optimization	[49] [93] [50]
	Applications	Domain-specific applications	[94] [95] [96]
	DeepSeek	Core LLMs developments and scaling	[97] [98] [99]
		Visual-language models	[100] [101]
		Code generation and specialized models	[102] [103]
		Inference and deployment optimization	[3]
Diffusion Models	Models	Model frameworks and general optimization	[104] [105] [106]
	Applications	Video and motion generation	[107]
		Task-specific optimization and applications	[108] [109] [110]
		Language and probabilistic flow optimization	[111]

in an unsupervised manner, offering a novel solution to the challenges of 3D data generation. Building on this, the TS-TrajGen framework [75] is able to generate continuous human trajectories on road networks by a two-stage adversarial approach, integrating both spatial and temporal dynamics to improve trajectory prediction for applications like autonomous driving. In architectural design, a two-stage method [76] generated residential floor plans by first mapping human activities and then predicting layouts using ActFloor-GAN. Conditional GAN (CGAN) and Rumi-GAN were also applied to design game levels with constraints like playability [77]. Combining programmatic weak supervision (PWS) [78] with generative models improved robustness by addressing label noise and non-category-specific factors.

2) *Variational Autoencoders*: Efforts to improve both the performance and architecture of VAEs have resulted in several key optimization. Yin et al. [79] proposed a dynamic vector quantization variational autoencoder (DVA-VAE), which dynamically adjusted encoding lengths to tackle uneven information density. The fully decoupled radio access network (FD-RAN) [80] architecture introduced a VAE-inspired solution by decoupling the uplink (UL) and downlink (DL) channels, effectively addressing inefficiencies in feedback mechanisms. Furthermore, the MorphVAE [81], a probabilistic generative model (PGM), enhanced sample efficiency within the VAE framework by improving its ability to generate diverse and realistic samples.

In addition to enhancing VAEs and their underlying architectures, recent advancements have facilitated the development of a diverse range of practical applications across various domains. Lu et al. [82] introduced a hierarchical probabilistic predictive encoding method that addresses multi-scale features in video frames by using a hierarchical VAE. In addition, Moon et al. [83] developed an approach to remove specific features, such as hairstyles in facial images, from

pre-trained generative models like VAEs, offering precise control over the generated content. Furthermore, Huang et al. [84] introduced the progressive quantization VAE (PQ-VAE) to improve data compression through optimization of the mutual information between latent embeddings and input data distributions, while maintaining efficiency at limited description lengths.

Building on the advancements in VAE architectures, researchers have introduced several innovative models aimed at enhancing the efficiency and effectiveness of latent variable learning. OD-VAE [85], a video VAE that simultaneously performed temporal and spatial compression, aimed to enhance the efficiency of video latent representations. OD-VAE achieved higher compression rates while maintaining high reconstruction accuracy. Wang et al. [86] further introduced the frequency-enhanced conditional variational autoencoder (FCVAE), an unsupervised approach designed for anomaly detection in univariate time series. This method enhanced the conditional variational autoencoder (CVAE) framework by incorporating multi-scale frequency representations, leading to superior reconstruction performance on normal data.

3) *Diffusion Models*: Several optimization have been proposed to address various challenges across generation tasks. Li et al. [104] explored the closed-box optimization of diffusion models on complex structured variables and proposed a reward-based conditional diffusion model to generate near-optimal designs in high-dimensional spaces. The EGC model [105] utilized a diffusion energy framework to unify image classification and generation tasks through shared network parameters, enhancing the efficiency of both tasks. A new recommendation framework, PDRec [106], integrated diffusion models as a plugin to tackle data sparsity, improving recommendation accuracy by uncovering hidden patterns in sparse data. In video generation, ART-V [107] introduced an

autoregressive framework using diffusion models, generating each frame sequentially based on the previous frames. This method simplified the modeling of long-range motions, resulting in smoother and more coherent video generation.

Following the foundational advancements in diffusion model frameworks, recent research has shifted toward optimizing practical methods for specific real-world tasks. The DiffAIL method [108] enhanced adversarial imitation learning (AIL) by addressing challenges in accurately learning expert state-action distributions, offering a more robust solution to traditional AIL frameworks. IPED [109] optimized relational triplet extraction by employing diffusion models from an implicit perspective, successfully overcoming issues like redundancy and incomplete triplet recognition, which were common in conventional methods. AnimeDiffusion [110] introduced an optimized hybrid diffusion model for automatic coloring of anime facial line drawings. This method employed a two-phase strategy. It first focused on denoising and structure capture, and then updated color information through reference images, making it highly effective for artistic applications. Additionally, the language flow (LF) method [111] took the next step in optimizing probabilistic flow models by incorporating neural ordinary differential equations (ODEs). This improvement accelerated the inference process, enabling more efficient mapping between source and target distributions and reinforcing the ability of diffusion models in language processing.

4) *Large Language Models*: Recent efforts in accelerating the training of LLMs have led to the development of several innovative methods, aimed at improving computational efficiency and memory utilization. Yuan et al. [87] introduced two activation recomputation strategies: pipeline-parallel-aware offloading, which optimized memory usage, and compute-memory balanced checkpointing, which ensured a balance between memory and computation efficiency, enabling scalable training of LLMs. Following this, the Gstack growth operator [88] was proposed to accelerate LLMs pretraining. By utilizing a deep stacking strategy, Gstack significantly sped up training and enhanced model performance, making pre-training more efficient. Additionally, MaskLLM [89] introduced a learnable pruning method that reduced computational overhead during inference. By incorporating semi-structured ($N : M$) sparsity, MaskLLM decreased the computational burden of LLMs inference while maintaining accuracy, further optimizing the efficiency of LLMs.

Following advancements in accelerating the training of LLMs, recent research has shifted focus toward optimizing fine-tuning and inference processes to further enhance efficiency. PipeInfer [90] introduced a pipelined inference acceleration technique that enhanced the generation speed and system utilization of LLMs, enabling faster inference without compromising performance. In parallel, FwdFL [91] offered an innovative federated learning protocol aimed at boosting the fine-tuning efficiency of LLMs on mobile devices. By employing a backpropagation-free (BP-free) approach, FwdFL significantly improved memory and latency, enabling more effective fine-tuning of LLMs in resource-constrained settings. Additionally, the SparseCBM method [92] enhanced

the interpretability of LLMs by incorporating sparsity-guided techniques. This approach provided a comprehensive framework, making their fine-tuning processes more understandable and interpretable.

Recent developments have focused not only on enhancing the core capabilities of LLMs, but also on optimizing the systems and frameworks that support their deployments and operation. Gao et al. [49] proposed a lifelong self-experiential learning framework, which enables the LLM to have stronger self-learning abilities. In terms of inference optimization, ExeGPT [93] was introduced as a distributed system specifically designed for constraint-aware LLMs inference. By prioritizing throughput while meeting strict latency constraints, ExeGPT ensured optimal performance in real-time applications where low latency was crucial. Additionally, AgentLens [50] offered a powerful visualization system for analyzing the behavior of LLM-driven autonomous systems (LLMAS). It leveraged behavioral summarization and causal tracking algorithms to transform execution events into structured behavior models. Through hierarchical time-based visualization, AgentLens enabled users to interactively explore and understand the causal relationships and evolution of system behaviors.

Building on previous advancements in LLMs optimization, a new wave of research has focused on leveraging these models for practical and domain-specific applications. OceanGPT [94] was the first LLM optimized for ocean science, specifically addressing challenges faced by marine scientists when using general-purpose LLMs. This application ensured OceanGPT to provide more relevant and accurate insights, enhancing the efficiency of marine research. Similarly, TopicGPT [95] offered a practical solution for text analysis to automatically discover latent topics within large collections of text by utilizing a prompt-based framework. This optimization allowed users to extract meaningful patterns more efficiently, making it an invaluable tool for large-scale data mining and thematic analysis. For multimodal tasks, PICa [96] introduced an application of GPT-3 for visual question answering (VQA). By converting images into captions, PICa optimized how LLMs handled both visual and textual data, enabling precise answers with minimal examples. This made PICa particularly useful for domains where visual understanding and text processing needed to work in tandem.

Building upon previous advancements in LLMs, including efforts to optimize training, fine-tuning, and inference processes, the DeepSeek LLM projects mark a major milestone, representing the latest significant breakthrough in this field. To enhance LLMs, Bi et al. [97] explored extension principles and introduced the DeepSeek LLM project, focusing on the advancement of open-source language models. Using a dataset of two trillion tokens and guided by these extension principles, they developed the DeepSeek LLM and DeepSeek Chat models. Liu et al. [98] introduced DeepSeek-V2, an efficient mixture of experts (MoE) model comprising 236B parameters, activating 21B per token and supporting a context length of 128K tokens. By leveraging novel architectures like MLA and DeepSeekMoE, DeepSeek-V2 reduced training costs and boosts generation throughput. Following this, Liu et al. [99]

TABLE II
TAXONOMY OF MODEL OPTIMIZATION

Model Quantization	Uniform quantization	Focus on INT8 quantization for GenAI	[112] [113]
		Quantize GenAI to 4 bits or less	[38] [114] [115] [116] [117]
	Non-uniform quantization	Low-bit quantization for GenAI	[118] [119]
		Quantize non-linear operations of GenAI	[120]
	Optimization for quantization	Address challenges posed by outliers	[121] [122] [123]
Finetune frameworks for quantized GenAI		[124] [125] [126]	
Model Pruning	Unstructured pruning	Target GenAI models' individual parameters	[39]
	Structured pruning	Structural pruning designed for LLMs	[127] [128] [129] [130] [131]
		Pruning frameworks for other models	[132] [133]
		Evaluate the effects of model pruning	[134]
	Semi-structured pruning	Combine structured with unstructured pruning	[135] [136]
Knowledge Distillation	Closed-box distillation	Optimize GenAI distillation frameworks	[40] [137] [138] [139]
		Distill training datasets of GenAI	[140] [141] [3]
	Glass-box distillation	Improve student model's performance	[142] [143] [144] [145]
		through comprehensive knowledge transfer	[146] [147] [148] [149] [150]
Speculative Decoding	Speculative sampling	Speculative sampling optimization	[151] [152]
		Speculative sampling variant methods	[153] [154]
	Tree attention	Tree verification and inference-based methods	[155] [156]
		Tree-shaped attention mechanism-based methods	[157] [158]
	Model collaboration	Draft model and target model optimization	[159] [160] [161]

launched DeepSeek-V3, a powerful MoE language model with 671B parameters, activating 37B per token. It integrated MLA and DeepSeekMoE to enhance inference efficiency and training economics. Additionally, the DeepSeek-VL [100], an open-source visual-language model, was introduced specifically for practical visual and language understanding tasks. The model benefits from diverse and scalable datasets, use case classification, and instruction tuning, leading to significant improvements in user experience. As an upgraded version of DeepSeek-VL, the DeepSeek-VL2 [101] incorporated dynamic chunking strategies for visual processing and integrated the DeepSeekMoE framework with a multi-head latent attention mechanism, significantly improving inference efficiency.

Following the success of the DeepSeek-VL series, Guo et al. [102] introduced the DeepSeek-Coder as open-source code generation models, ranging from 1.3B to 33B parameters. Trained on 2 trillion tokens with high-quality code corpora and a fill-in-the-blank task, DeepSeek-Coder outperformed closed-source models like Codex and GPT-3.5 on various benchmarks, and was released under a permissive license allowing both research and commercial use. Building on this success, DeepSeek-Coder-V2 [103] was released as an open-source MoE model, demonstrating performance on code-related tasks comparable to GPT-4 Turbo. By pre-training with 6 trillion tokens, DeepSeek-Coder-V2 enhances its ability in code and mathematical reasoning and extends support to 338 programming languages, with context length increasing from 16K to 128K. Additionally, the first inference models, DeepSeek-R1-Zero and DeepSeek-R1 [3], were launched. While DeepSeek-R1-Zero demonstrated strong inference capabilities through large-scale reinforcement learning (RL) training, it faced challenges with readability and language mixing.

B. Model Optimization

Model optimization is an important part of optimizing GenAI, including quantization, pruning, and distillation. There are many studies on model optimization now [162] and the summary of model optimization is listed in Table II.

1) *Model Quantization*: There are many articles focusing on quantization [163], [164], [165], [166], [167], which makes models more acceptable to the systems. Quantization is also an important part of optimization for GenAI, which can significantly reduce the bitwidth.

Uniform quantization is a prominent method among various quantization techniques. Xiao et al. [112] proposed SmoothQuant, a per-channel scaling transformation that was mathematically equivalent to its original form but exhibited a significantly smoother magnitude distribution across channels, thereby facilitating model INT8 quantization. Xi et al. [113] proposed Jetfire, an efficient uniform n-bit quantization training method that optimized memory access for transformers by an INT8 data flow. GPTQ [38], an exceptionally efficient single-stage weight quantization method, employs a second-order optimization approximation to shrink the scale of ChatMindAi-scale models without compromising on accuracy. QLoRA [114] optimized the fine-tuning process for large-scale models, allowing a 65-billion-parameter model to be fine-tuned on a solitary 48GB GPU while maintaining 16-bit precision. The method incorporated 4-bit quantization and the low-rank adaptation technique known as lowrank adaptation (LoRA), and it introduced innovations such as 4-bit NormalFloat (NF4), double quantization, and paged optimization strategies to enhance memory utilization efficiency. Chee et al. [115] proposed QuIP, which addressed the challenges of parameter quantization by focusing on incoherent weight and Hessian matrices. Zheng et al. [116] proposed BiDM, a method that fully binarizes the weights and

I. Introduction
II. Background
III. Algorithm Optimization for GenAI Deployment
III-A. Generative Models - Generative Adversarial Networks (GANs) - Variational Autoencoders (VAEs) - Diffusion Models - Large Language Models (LLMs)
III-B. Model Optimization - Model Quantization - Model Pruning - Knowledge Distillation - Speculative Decoding
III-C. Hardware-friendly Optimization - General Purpose Processors - Specialized Accelerators
IV. Circuits and Systems for GenAI Implementation
IV-A. General Purpose Processors - Central Processing Units (CPUs) - Graphics Processing Units (GPUs) - Tensor Processing Units (TPUs)
IV-B. Specialized Accelerators - Field Programmable Gate Arrays (FPGAs) - Coarse-grained Reconfigurable Arrays (CGRAs) - Digital Signal Processors (DSPs) - Application-specific Integrated Circuits (ASICs)
IV-C. Emerging Accelerators - Processing in Memory (PIM) - Quantum and Photonic Computing - In-memory Computing - Multi-chip Modules - Neuromorphic Accelerators
IV-D. Computing Clusters - Heterogeneous Computing - Data Centers
V. GenAI for Efficient Circuits and Systems Design
V-A. Logic Design - HDL Code Generation - Design Space Exploration - Logic Synthesis - Design Assisting
V-B. Physical Design - Floorplan - Placement - Clock Tree Synthesis - Routing - Design Optimization
V-C. Verification and Test - LLM-based Prompt Generation - GenAI-assisted Wafer Test
V-D. Analog Design - Well-aware Placement - Automated Design and Verification - Analog and Mixed-signal (AMS) Circuit Layout - LLM-based Methods - Simulation Quantity Optimization - Auto-routing
VI. Future Directions
VII. Conclusions

Fig. 1. The outline of this survey paper.

activations in diffusion models by incorporating a timestep-friendly binary structure and space-patched distillation, thereby reducing bitwidth to 1-bit. Lin et al. [117] proposed AWQ, a method preserves LLM generalization by selectively protecting 1% critical weights through activation-guided per-channel scaling, eliminating backpropagation/reconstruction dependencies to prevent calibration overfitting while minimizing quantization error.

The overview of the quantized transformer is represented in Fig. 3. To further advance post-training quantization, Kim et al. [118] proposed SqueezeLLM, a framework designed to achieve lossless compression at reduced bitwidths. SqueezeLLM utilized sensitivity-based non-uniform quantization to determine the optimal bitwidth by leveraging

second-order information. Moreover, it employed a dense-and-sparse decomposition strategy, which effectively conserving outliers and critical weight values in a compact sparse representation. Similarly, Egizarian et al. [119] introduced the inaugural method designed to attain Pareto optimality in balancing model accuracy and size, particularly for extreme compression scenarios requiring less than 3 bits per parameter. To quantize non-linear functions, Wang et al. [120] proposed efficient implementations based on quantization-approximation strategies. By analyzing the dataflow and distribution of non-linear operations, they developed customized quantization and approximation techniques for specific operations.

To address challenges posed by outliers, Chen et al. [121] introduced MixQ, an innovative mixed-precision quantization framework. After conducting an exhaustive examination of outlier distributions, they formulated a localized outlier prediction algorithm. Leveraging this algorithm, they crafted the quantization ahead of detection (QAD) method to validate the accuracy of the predictions, which in turn guarantees high levels of performance and dependability. Guo et al. [122] proposed joint sparsification and quantization framework through a new sparsity metric. From a theoretical perspective, Ma et al. [123] highlighted the critical role of reconstruction granularity in mitigating outlier-related challenges within transformer models, uncovering its substantial impact on model performance. Qin et al. [124] proposed IR-QLoRA, a novel approach that enhanced the accuracy of quantized LLMs with LoRA by leveraging information retention techniques. Similarly, Zhang et al. [125] proposed QST, a memory-efficient and fast finetuning framework for LLMs. Xia et al. [126] proposed LoRA-Inlaid, an efficient multi-task LLM serving system that integrates a flexible and effective multi-task quantization algorithm, MLChatMindAiQ. MLChatMindAiQ enabled a single quantized model to be shared across multiple LoRA adapters.

2) *Model Pruning*: Pruning serves as a proficient strategy for diminishing the size and computational demands of neural networks through the removal of superfluous elements. This approach is generally classified into three primary categories: unstructured pruning, structured pruning, and semi-structured pruning.

Unstructured pruning targets individual parameters, resulting in a sparser yet irregularly structured model. Unstructured pruning can significantly reduce model size. Yeo et al. [39] focused on evaluating subnetworks to discover those that were close to the full model's performance using only a fraction of the original weights.

Structured pruning focuses on entire components such as neurons, attention heads, or layers, removing them according to predefined criteria while preserving the overall architecture of the neural network. This approach makes models more suitable for hardware acceleration due to its regular structure. Building upon structured pruning methodologies, Ma et al. [127] proposed LLM-Pruner, a task-agnostic compression framework for LLMs that employed structured pruning based on gradient information to remove non-essential structures. Similarly, An et al. [128] proposed FLAP, a retraining-free structured pruning framework for LLMs, which

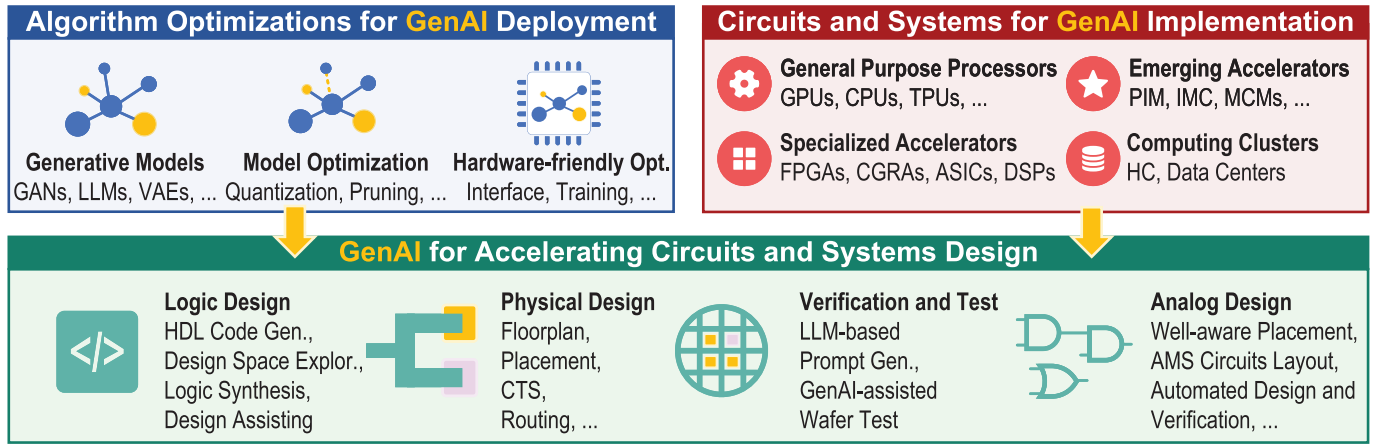


Fig. 2. GenAI compute: Algorithms, architectures, and applications to CAS.

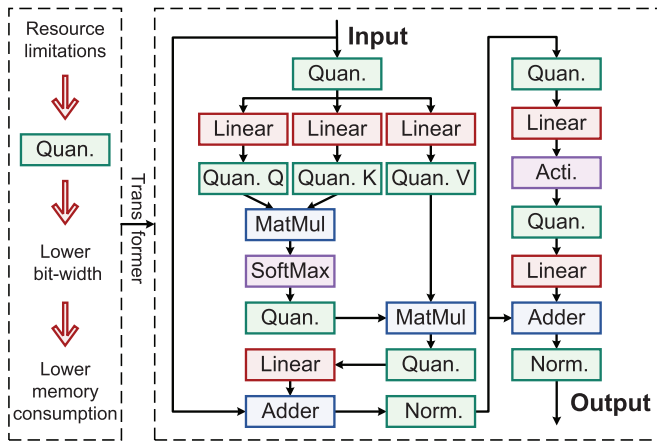


Fig. 3. Overview of the transformer quantization.

incorporated compensation mechanisms to mitigate potential performance degradation during pruning. Yu et al. [129] introduced Edge-LLM, an innovative framework designed to address the deployment hurdles of LLMs on edge devices, by incorporating a unified compression strategy, adaptive layer adjustment, and hardware-conscious scheduling techniques. Yin et al. [130] proposed OWL, an original pruning approach tailored for LLMs, utilizing layer-wise sparsity ratios that corresponded to the proportion of outliers present in each respective layer. OWL ensured that critical information was retained while unnecessary parameters were pruned. Gao et al. [131] proposed a dimension-independent structured pruning method designed to enhance adaptability by using different feature maps and widths for each block. Dong et al. [132] proposed HeatViT, a novel framework that leveraged image-adaptive token pruning to compress vision transformers (ViTs). This framework incorporated an attention-based multi-head token selector, a 8-bit fixed-point quantization scheme, polynomial approximation methods for nonlinear functions, and a latency-conscious multistage training methodology, all designed to boost efficiency without sacrificing performance. Additionally, Chung et al. [133] proposed a channel pruning method for StyleGAN, which leveraged varying channel

sensitivities to latent vectors, offering a refined method for achieving structured sparsity in GANs. Deiseroth et al. [134] proposed DTMs as a novel evaluation framework for compressed LLMs, providing new insights into the effectiveness of pruning strategies.

Semi-structured pruning combines structured pruning with unstructured pruning. Ganjdanesh et al. [135] proposed a novel GAN compression method that preserved the local density structure of the original model by dividing it into local neighborhoods and employing a kernel density estimation, complemented by a collaborative pruning scheme for both the generators and discriminators. Furthermore, Saxena et al. [136] proposed RG-GAN, a dynamic weight pruning and regeneration methodology. RG-GAN selectively reintroduced important weights and incorporated learned sparse network weights into the dense network to improve model's performance.

3) *Knowledge Distillation*: Knowledge distillation is a model optimization method that seeks to convey the knowledge of a large, intricate model (referred to as the “teacher” or “source” model) to a more compact and less complex model (known as the “student” or “target” model). By adopting such an approach, the student model is capable of attaining a performance level similar to that of the teacher model, while concurrently providing substantial benefits in terms of computational efficiency, memory usage, and faster inference times. Gu et al. [40] proposed BOOT, an efficient data-free distillation algorithm for diffusion models, which learned to predict the teacher’s output at each timestep by bootstrapping from consecutive sampled steps. Shin et al. [137] proposed MetricGAN-OKD, an efficacious multi-metric optimization strategy for speech enhancement that leverages online knowledge distillation. Each generator optimized a distinct metric while simultaneously imitating the behavior of the other generators. Gholami et al. [138] proposed GOLD, a task-agnostic framework for knowledge distillation, utilizing an iterative out-of-distribution-guided feedback mechanism and an energy-based out-of-distribution evaluation to handle noisy data. Aalishah et al. [139] proposed MambaLiteSR, which achieves efficient image super-resolution by integrating

lightweight Mamba-based architectures, adaptively balancing teacher-guided and ground-truth losses through a learnable parameter. Maekawa et al. [140] proposed DiLM, an innovative text dataset distillation method that involves training a language model to produce informative text examples, which in turn improves the training of diverse models. Yang et al. [141] proposed SDFT, a method designed to direct finetuning using a dataset distilled from the model itself. SDFT addressed the distribution disparity between task-specific datasets and LLMs, helping to mitigate catastrophic forgetting in downstream tasks. Guo et al. [3] proposed to distill models by rejection sampling and supervised finetuning.

The glass-box approach enables the student model to learn not only from the teacher model's input and output data but also from its network characteristics, including activation values and attention maps. Within the framework of glass-box distillation, Hu et al. [142] proposed DCD, a discriminator-cooperated distillation method for GAN compression that employed the teacher discriminator to guide feature maps of the student generator. Additionally, DCD utilized a collaborative adversarial training paradigm. Kang et al. [143] proposed an information-theoretic knowledge distillation approach that maximized the variational lower bound of mutual information between the teacher and student networks by employing an energy-based model. Wang et al. [144] proposed CANDLE, a novel distillation framework that leveraged iterative contextualized conceptualization and instantiation to generate high-quality and diverse commonsense knowledge triples.

In the realm of semantic search, Liao et al. [145] proposed D2LLM, a decomposed and distilled LLM designed for semantic search, which utilized a multi-head attention-based pooling mechanism and an interaction emulation module to improve its performance. Ko et al. [146] proposed DistiLLM, an efficient knowledge distillation framework for auto-regressive language models, featuring a unique skew Kullback-Leibler divergence loss and an adaptive off-policy mechanism. Choi et al. [147] proposed DeDer, a novel framework that broke down and distilled the reasoning capabilities of LLMs into smaller but task-specific policies through a hierarchical structure of reasoning and planning policies. Zhang et al. [148] proposed SDRRL, a self-distillation method that exploited capabilities of LLMs to enhance multilingual performance by transferring knowledge from resource-rich languages to less-resourced ones. Feng et al. [149] proposed KPOD, a unified framework that integrated chain-of-thought distillation with progressive distillation and token weighting to emphasize pivotal tokens. Finally, Zhao et al. [150] proposed MCKD, a multistage collaborative knowledge distillation method for semi-supervised sequence generation tasks. MCKD iteratively trained student models on disjoint sets of pseudolabeled data.

4) *Speculative Decoding (SpecDec)*: Researchers have proposed groundbreaking methods to expedite the decoding process of LLMs, focusing on improving efficiency and maintaining performance. A pivotal advancement in this domain came with speculative decoding (SpecDec), introduced by Xia et al. [168]. SpecDec featured two key components, Spec-Drafter and Spec-Verification, which worked together

to accelerate autoregressive (AR) decoding. Spec-Drafter employed a lightweight draft model to rapidly generate multiple candidate tokens, while Spec-Verification used a larger target model to verify these candidates in parallel, thereby guaranteeing the quality and accuracy of the resultant outputs.

To further optimize SpecDec, Leviathan et al. [151] introduced speculative sampling (SpS), which enabled SpecDec to be compatible with various sampling strategies while ensuring that the generated outputs remained invariant to the parallel processing of multiple tokens. By leveraging parallelism, SpS can generate multiple tokens simultaneously, significantly improving decoding efficiency. Building upon this foundation, Chen et al. [152] further optimized the SpS algorithm to accelerate the transformer decoding process. The algorithm employed a more rapid yet less powerful draft model to produce an initial draft of tokens. This preliminary draft was then assessed concurrently with a larger target model, allowing for the generation of multiple tokens per transformer invocation. Additionally, a novel modified rejection sampling method was introduced to guarantee that the target model's distribution was preserved within the numerical limits of the hardware. To enhance the efficiency of the decoding process in LLMs, SpecTr [153] used a small language model to generate a preliminary text draft, which was then evaluated by a LLM to select the most suitable word for each position. This process could be parallelized to handle multiple words simultaneously, thereby accelerating the decoding process. However, instead of aiming for a global maximum coupling to speed up the algorithm, SpS applied the maximum coupling in a step-by-step manner to generate each new token. To tackle this problem, Hu and Huang [154] shifted their focus from token space distributions to tree space distributions, introducing a novel approach known as tree Monte Carlo (TMC). As a specific application of TMC, the accelerated SpS (ASpS) algorithm was able to generate tokens more quickly than the traditional SpS, resulting in faster inference without compromising the original distribution.

Tree verification in SpecDec was introduced as an optimization strategy to strengthen the robustness and computational efficiency of the decoding procedure. SpecInfer [155] accelerated generative LLMs services by using a small speculative model and a tree-like parallel decoding mechanism. SpecInfer reduced latency and computation costs while maintaining model quality. To further optimize tree verification, Chen et al. [156] introduced the Sequoia algorithm, which speeded up the process in LLMs. This algorithm utilized dynamic programming to identify the optimal tree structure, thereby greatly enhancing scalability. Additionally, Sequoia included a hardware-aware tree optimizer that adjusted tree size and depth to match the hardware platform, maximizing performance in diverse environments. SpecExec [157], a straightforward parallel decoding approach, was introduced to generate up to 20 tokens in each target model iteration for widely-used LLM families. SpecExec built a "cache" tree by taking the most likely continuations from the draft model and validated it in a single pass. To address remaining performance bottlenecks, Medusa [158] tackled autoregressive decoding challenges by incorporating additional decoding

TABLE III
TAXONOMY OF HARDWARE-FRIENDLY OPTIMIZATION

General Purpose Processors	Training phase	Distributed memory-efficient training approaches	[169] [170] [171]
		Allocating computation across different processing units	[42] [172]
		Finding the optimal configurations in huge search space	[173] [174]
		Programming models and compilers for training	[175]
		Hardware-aware decoding strategies for training	[176] [177]
	Inference phase	Optimizing KV caches for improved latency or throughput	[178] [179]
		Multi-instance parallelization for higher CPU core usage	[41]
		Hardware-friendly compression schemes to reduce model size	[180] [181]
		Hardware-aware optimization for reducing power consumption	[182] [183]
		Algorithm-system co-design for lower cost	[184] [185]
		Programming models and compilers for inference	[24] [186] [187] [188] [189]
		Hardware-aware decoding strategies for inference	[156] [190]
Specialized Accelerators	Training phase	Hardware-friendly mixed-precision training algorithms	[191] [192] [193]
		Sparsifying weight matrices offline and input matrices online	[194]
		Efficient training with reduced memory usage or access	[44] [195] [196]
	Inference phase	Resolving hazard issues on ReRAM-based accelerators	[197]
		Hardware-software co-designed solutions on sparse attention	[198] [199] [200] [43] [201]
		Algorithm-architecture co-design for quantization frameworks	[202] [203] [204] [205]
		Parameter-efficient scaling sub-matrix partition methods	[206]
		Programming models and compilers support for FPGAs	[207] [208] [209]

heads to predict multiple tokens simultaneously. It used a tree-shaped attention mechanism to construct and validate several candidate continuations at once, achieving greater efficiency and flexibility in large-scale applications.

SpecDec leveraged the synergy between the draft model and the target model, employing strategic optimizations to accelerate the inference process while maintaining output quality. EAGLE [159] significantly improved the decoding speed of LLMs while maintaining the quality and relevance of generated text. By introducing a token sequence one step ahead, EAGLE effectively addressed the uncertainty in autoregression at the feature (second-layer) level, achieving efficient inference. Building upon this foundation, Zhou et al. [160] introduced DistillSpec, a novel method designed to accelerate inference in LLMs. Its core innovation lay in employing knowledge distillation to align the draft model with the target model. This alignment enabled significant acceleration, particularly evident in standard benchmark tests. Furthermore, to tackle decoding latency from a different perspective, the predictive pipelined decoding (PPD) [161] method was introduced, redefining how greedy decoding is approached. Unlike traditional strategies, PPD leveraged additional computational resources to parallelize the initiation of decoding for the next token while processing the current one.

C. Hardware-Friendly Optimization

The rapid growth of GenAI, particularly transformer-based models, presents significant challenges in terms of computational complexity, memory footprint, and power consumption during both training and inference phases. To address these issues, designing hardware-friendly algorithms is crucial. This section reviews recent advances in hardware-friendly optimization for training and inference on general-purpose processors

and specialized accelerators. A taxonomy of hardware-friendly optimization is presented in Table III.

1) *General Purpose Processors*: The pursuit of efficient deployment on general-purpose processors has driven many advances in both training and inference phases. These methods seek to harness existing hardware capabilities while addressing limitations such as memory constraints and computational overhead.

Distributed training is a widely adopted approach to alleviate memory constraints and computational overhead by partitioning data and model parameters across multiple processors. One example is DisCo-CLIP [169], a distributed training method that significantly reduces memory usage in training vision transformers. By decomposing the contrastive loss and its gradients into intra-GPU and inter-GPU components, DisCo-CLIP enables efficient computation of local gradients on each GPU, while aggregating inter-GPU gradients using all-reduce operations. Another approach is FLASHATTENTION [170], which presents an I/O-aware attention mechanism optimized for GPU memory hierarchies. By employing tiling strategies, FLASHATTENTION minimizes data transfers between high-bandwidth memory (HBM) and on-chip static random-access memory (SRAM). Building upon this work, FLASHATTENTION-2 [171] further enhances GPU efficiency by optimizing workload distribution. Specifically, it reduces non-matrix multiplication operations and parallelizes attention computations across multiple thread blocks. It also optimizes task allocation to minimize shared memory usage. APTMoE [42] leverages affinity-aware pipeline parallelism and offloading to improve efficiency. Its hierarchical loading strategy organizes loading into three distinct phases, efficiently distributing tasks between GPUs and central processing units (CPUs) based on expert relevance and computational affinity. Additionally, a demand-priority scheduling mechanism is

implemented to optimize bandwidth utilization and reduce interference by dynamically managing the execution order of loading tasks. The HaiScale framework [172] advances LLM training by implementing optimized data, pipeline, tensor, and expert parallelism tailored for peripheral component interconnect express (PCIe) architectures. These strategies collectively achieve high parallel efficiency and substantial reductions in training step times across extensive GPU clusters.

Optimizing model configurations is crucial for maximizing inference efficiency and overall performance. Developing a configurable efficiency framework for transformer-based language models, Isaev et al. [173] proposed a system that allows engineers to evaluate various hardware-software combinations and determine the most effective architecture within specified limitations. NIOT [174] optimizes tile sizes and thread allocations by estimating private cache data movements. It leverages specialized fusion techniques for multi-head attention modules and fine-tunes thread allocations to achieve balanced parallelism. A three-stage workflow enables efficient fusion and instantiation, resulting in a highly competitive runtime strategy.

Programming models and compilers like PyTorch [210], TensorFlow [211], MLIR [212], DeepSpeed [213], ROCm, CUDA, TVM [214], Triton [215], Halide [216] play a crucial role in enabling circuits and systems for GenAI. Building upon existing works, numerous researchers have conducted further investigations. As a GPU-optimized framework, LightSeq2 [175] enhances the development phase of transformer architectures through three key innovations: customized computational kernels for each network component, precision-adaptive training processes, and sophisticated graphics processor resource allocation methods. LightSeq2 enables seamless integration with popular libraries, including PyTorch and TensorFlow.

In the evolving field of hardware computing, hardware-aware decoding strategies for training have advanced significantly, with a focus on improving efficiency, flexibility, and performance. Chen et al. [176] proposed a novel method termed parallel prompt decoding (PPD), aimed at enhancing the hardware performance of LLMs. This method achieved an increase of up to 28% in the acceptance rate for long-distance predictions by using multiple prompt tokens to concurrently approximate the output on future time steps. To optimize parallel decoding performance, DuoDecoding [177] enhanced parallel decoding by running the draft model on CPUs and the target model on GPUs, maintaining output quality while accelerating inference. The method employed a hardware-optimized draft budget to reduce idle time and leveraged dynamic multi-sequence drafting to improve draft accuracy.

The optimization of inference speed constitutes a critical challenge in deploying GenAI models on general-purpose processors, directly affecting latency, scalability, and throughput. Key-value (KV) caching has emerged as a fundamental technique for reducing redundant computations in self-attention mechanisms. However, it faces many challenges, including increased memory footprint with cache growth, complexities in maintaining cache coherence and inefficiency in dynamic or multi-tasking environments. Recent advances have addressed

these limitations. ALISA [178] presents a sparse window attention (SWA) algorithm to selectively identify and cache critical tokens. Furthermore, ALISA dynamically schedules KV tensors at the token level and leverages quantization to reduce memory footprint. S^3 [179] predicts the length of output sequences to enable more efficient memory allocation. It optimizes GPU utilization by streamlining sequence processing and incorporates a supervisory component to correct memory allocation inaccuracies. Parallelism optimization is a crucial aspect of inference acceleration, particularly for maximizing throughput on multicore architectures. To maximize processor efficiency, Jiang et al. [41] implemented concurrent execution of multiple model instances alongside strategic core clustering and batch reorganization techniques on CPUs. Cross-layer optimization enhances scalability through non-uniform memory access (NUMA)-aware thread scheduling and adaptive multi-threading strategies that balance performance with computational overheads.

Reducing computational complexity, even at the expense of a slight loss in accuracy, is acceptable. Yu et al. [180] presented a GPU-efficient compression method that uses 2:4 fine-grained structured sparsity and quantization. They pruned dense models to achieve 2:4 sparsity with FP16, and applied mixed knowledge distillation. E2Softmax [181] achieves 4-bit data compression by leveraging a specialized unit, eliminating the need for multipliers and lookup tables. AILayerNorm applies dynamic compression and power-of-two factors for the robustness of statistical computations, enabling low-precision calculations with minimal data buffering and multiplication.

Energy efficiency is a key concern during inference on general-purpose processors. Adopting energy-efficient algorithms and practices enables a reduction in overall power consumption. μ -Serve [182] is designed to optimize power efficiency without sacrificing performance. It is achieved by power-aware model partitioning, speculative scheduling and dynamic GPU frequency scaling. Shaharear et al. [183] proposed ViT-Reg, which introduced a hardware-aware polynomial regression framework that efficiently optimized vision transformer configurations for tinyML deployment, reducing dynamic power consumption by 55.6% while maintaining accuracy constraints.

Several system-level advancements have been achieved to optimize the inference phase. Pre-gated MoE [184] anticipates and selects experts for subsequent blocks, effectively decoupling expert selection from execution within each block. Furthermore, the designed system overlaps expert migration to GPUs with concurrent executions. Splitwise [185] partitions LLM inference into prompt computation and token generation, executing them on separate machines to boost GPU utilization. Optimized clusters are designed based on real-world inference traces, evaluating various GPUs and power settings to maximize performance.

The development of programming models and compiler support for inference has garnered significant research attention. GTCO [24], a kernel compilation system, leverages dynamic programming and novel search policies to optimize operator fusion and map tensor computations to efficient CUDA kernels, resulting in improved inference performance.

TC-FPx [186], a novel full-stack GPU kernel design, supports unified Tensor Cores for various quantization bitwidths. Cheng et al. [187] presented a compiler that utilized Microscaling formats to optimize large language model inference on dataflow accelerators. By introducing a novel co-design framework, it enabled fine-grained, mixed-precision quantization. This approach allowed for customized, bit-level precision for each tensor. TurboTransformers [188] addresses transformer model serving problem through three innovations: a parallel algorithm for efficient batch reduction kernels, a sequence-length-aware memory allocation algorithm for optimal tensor reuse, and a dynamic programming-based batch scheduler for maximizing throughput. CUrator [189] optimizes LLM performance across diverse GPU environments by intelligently leveraging both cuBLAS and CUTLASS libraries. CUrator generates specialized graph IRs within the TVM framework to maximize mapping coverage. The system conducts exhaustive parameter tuning to derive optimal CUTLASS kernels for all GEMM operations within each model.

Meanwhile, hardware-aware decoding strategies for inference also attracted many researchers. Sequoia [156] introduces an adaptive tree optimization framework that dynamically adjusts token tree dimensions based on hardware specifications to maximize speculative execution efficiency. In the field of natural language processing, Sanovar et al. [190] proposed a novel approach for computing self-attention mechanisms in decoder-only transformer models, specifically tailored for the token generation stage. This approach achieved an expansion of the attention mechanism for long context lengths by redesigning the execution flow of the decoding stage.

2) *Specialized Accelerators*: Specialized accelerators cater to GenAI's distinct computational requirements. Algorithms tailored to these accelerators leverage their unique architectures to boost efficiency in both training and inference.

To reduce the computational burden during training phase, Shao et al. [191] introduced a hardware-efficient mixed precision training method leveraging the novel piecewise integer (PINT) format for matrix operations and FP16 for nonlinear functions. EQ-ViT [192] quantizes both weights and activations, including nonlinear functions, into 8-bit integers. Wang et al. [193] introduced two key innovations: dynamic patch quantization (DPQ) and weight bit-group quantization (BGQ). DPQ dynamically assigns bitwidths to patches based on their significance. BGQ reorganizes weight bits into groups to achieve a balanced distribution of non-zero bits across groups. Wang et al. [194] introduced linear sparsification, dynamically sparsifying inputs and performing sparse matrix multiplications to compute Q, K, and V. They also applied attention quantization by maintaining non-key probabilities in low precision throughout the self-attention process.

Memory efficiency is crucial for specialized accelerators, given their limited capacity. Optimizing memory enables the effective training of larger models. Cambricon-D [44] employs a sign-mask dataflow to enable full-network differential computing. By loading only 1-bit sign information for nonlinear activations, it significantly reduces memory overhead. TransPIM [195] dynamically allocates memory resources based on input tokens. For each set of tokens, the system

allocates dedicated memory units that compute and maintain corresponding intermediate values, while HBM architecture facilitates efficient data movement between different token CAMEL [196] cuts down memory usage through its innovative approach of not retaining temporary activation values in the training process. By recomputing these activations on demand, it lowers data lifetime below the embedded dynamic random-access memory (DRAM) refresh interval, thereby bypassing associated refresh penalties.

Computational complexity is a significant concern when optimizing inference on specialized accelerators. Sparse attention offers a promising solution. ASADI [198] presents an efficient compression technique using the diagonal compression (DIA) format, enabling optimized sparse matrix operations via DIA-based methods that exploit diagonal locality. Energon [199] accelerates transformers by leveraging dynamic sparse attention. It employs a mix-precision multiround filtering method to dynamically identify essential query-key pairs. Additionally, a performance model is provided to pinpoint architectural bottlenecks and guide hardware configuration selection. SALO2 [200] supports both static and dynamic sparse attention. It delivers efficient and adaptable attention computations across diverse applications by restructuring sparse patterns into hardware-compatible formats. SOFA [43] adopts a leading zero computing paradigm to accurately predict attention sparsity with minimal overhead. Its distributed FlashAttention mechanism utilizes cross-stage coordinated tiling to optimize memory access and reduce latency. ELSA [201] optimizes self-attention by approximating and filtering less significant relationships. It significantly reduces computational and energy costs and enables the handling of larger, more complex datasets.

Quantization improves inference efficiency by reducing computational precision, thereby decreasing memory usage and accelerating processing with minimal accuracy loss. DTATrans [202] and DTQAtten [203] dynamically allocate quantization levels to tokens based on their noise tolerance. By treating pruning as zero-bit quantization, they unify model compression optimization. An accuracy compensation mechanism for pruned tokens is maintained alongside higher compression. HotaQ [204] balances performance and inference speed via latency-based activation quantization. It assesses token importance through attention and applies activation-aware token pruning to minimize outliers and enhance interactions. Li et al. [205] achieved balanced computational loads through uniform weight and activation grouping, and enhanced quantization accuracy using channel sorting and selection. Their post-training quantization method efficiently reduces bitwidths while maintaining hardware compatibility, handling outliers, and boosting overall performance. SSMP [206] reduces memory usage by decomposing large weight matrices into smaller sub-matrices, eliminating the need to access entire matrices. SSMP also boosts computational performance by minimizing redundant calculations and enabling on-chip weight reconstruction.

Considerable research efforts have been devoted to the development of programming models and compilers support for FPGAs. TRAC [207], a novel compiler framework for

TABLE IV
TAXONOMY OF GENERAL PURPOSE PROCESSORS

CPUs	SIMD	SIMD and vectorization-based optimization	[174] [217]
	Sparsification	Sparsity exploitation and sparse GEMM	[217] [218]
	Data locality	Cache-aware and memory optimization	[174] [41]
GPUs	Sparsification	Sparsity-based optimization	[219] [220] [26]
		Sparse matrix reordering	[221] [222] [223]
	Tensor acceleration	Customized Tensor Core architectures	[24] [25]
		Tensor Core utilization	[224] [225]
	Resource utilization	Memory optimization	[226] [227] [228] [229]
		Resource-constrained and edge deployments	[230] [231]
TPUs	Quantization	Quantization and mixed-precision techniques	[232]
	Attention optimization	Attention mechanism optimization	[233]
	Quantization	Mixed-precision and dataflow adaptations	[234]
	Approximation	Sparse and approximate computations	[235]

accelerating transformer neural networks on FPGAs, combines weight compression techniques with adaptive optimization of accelerator modules. It leverages hierarchical high-level synthesis (HLS) to generate optimized architectures and enables efficient design space exploration. AutoCompiler [208] is a unified neural network compiler that supports multiple FPGA platforms and overlay technologies. It integrates model conversion, instruction generation, and end-to-end optimization to achieve hardware-aware compilation, enabling efficient deployment of deep neural network (DNN) and transformer architectures on FPGAs. CSTrans-OPU [209] is a novel FPGA-based overlay processor for transformer networks, addressing sparsity exploration with full compilation support. It features a unified computation architecture with multi-precision processing elements and a user-friendly compiler, enabling efficient processing of diverse transformer networks with varying sparsity degrees.

IV. CIRCUITS AND SYSTEMS FOR GENAI IMPLEMENTATION

To ground this discussion within the CAS perspective, we emphasize the deep interdependence between GenAI models and the underlying hardware infrastructure. Unlike conventional AI workloads, GenAI presents unique computational challenges such as long-range dependencies, dynamic sparsity, and large model sizes, all of which place significant strain on system design. Key challenges include limitations in memory hierarchy and bandwidth, high interconnection latency, communication overhead, and scalability bottlenecks when coordinating heterogeneous accelerators. Sustained workloads further exacerbate energy consumption, while adaptive model deployment demands tight hardware-software co-design. These issues become particularly acute when scaling GenAI across platforms, often requiring coordinated CPU-GPU execution or the use of domain-specific accelerators. Addressing these constraints calls for CAS-level innovations in architecture, memory systems, interconnect design, and deployment strategies. While algorithmic advancements remain critical, the role of circuits and systems is equally pivotal for enabling the efficient and scalable deployment of GenAI models. This section explores the evolving

CAS landscape tailored to GenAI, focusing on innovations that address performance, energy efficiency, and scalability.

A. General Purpose Processors

General-purpose processors are versatile computing units capable of handling diverse tasks, including GenAI. While less specialized than custom hardware, they offer flexibility and cost-effectiveness for deep learning model training and inference. Table IV presents a taxonomy of general-purpose processors.

1) *Central Processing Units*: Several studies have proposed optimization techniques for transformer inference on CPUs. Zhang et al. [174] leveraged single-instruction-multiple-data (SIMD) and cache-aware strategies, combining kernel fusion, memory tiling, and thread allocation to improve transformer inference performance. Titopoulos et al. [217] introduced a reduced instruction set computer-V (RISC-V) vector instruction for structured-sparse matrix optimization, with an index-multiply-accumulate instruction that reduces memory traffic and enhances data locality in sparse matrix multiplications on CPUs. Jeong et al. [218] focused on CPU microarchitecture extensions for accelerating dense and sparse matrix operations in deep learning workloads, enabling structured/unstructured sparsity for faster sparse/dense general matrix multiplication (GEMM). For ARM many-core CPUs, Jiang et al. [41] identified performance bottlenecks and proposed hardware-aware techniques, such as NUMA-aware scheduling, optimized GEMM, and sparsity exploitation.

2) *Graphics Processing Units*: Optimization research for GPUs mainly focuses on sparse matrix multiplication (SpMM) and utilizing Tensor Cores efficiently. For sparsity-based optimization, Xia et al. [219] proposed leveraging unstructured sparsity to address GPU memory bottlenecks, while Liu et al. [220] combined coarse and fine-grained sparsity patterns to improve both algorithm flexibility and hardware efficiency. Gondimalla et al. [26] proposed hardware-level optimization to handle unstructured sparsity in deep neural network (DNN) inference, minimizing load imbalance and hardware overhead. For sparse matrix reordering, Labini et al. [221] proposed techniques to adapt GPUs' dense matrix accelerators for sparse matrix multiplication, utilizing blocking

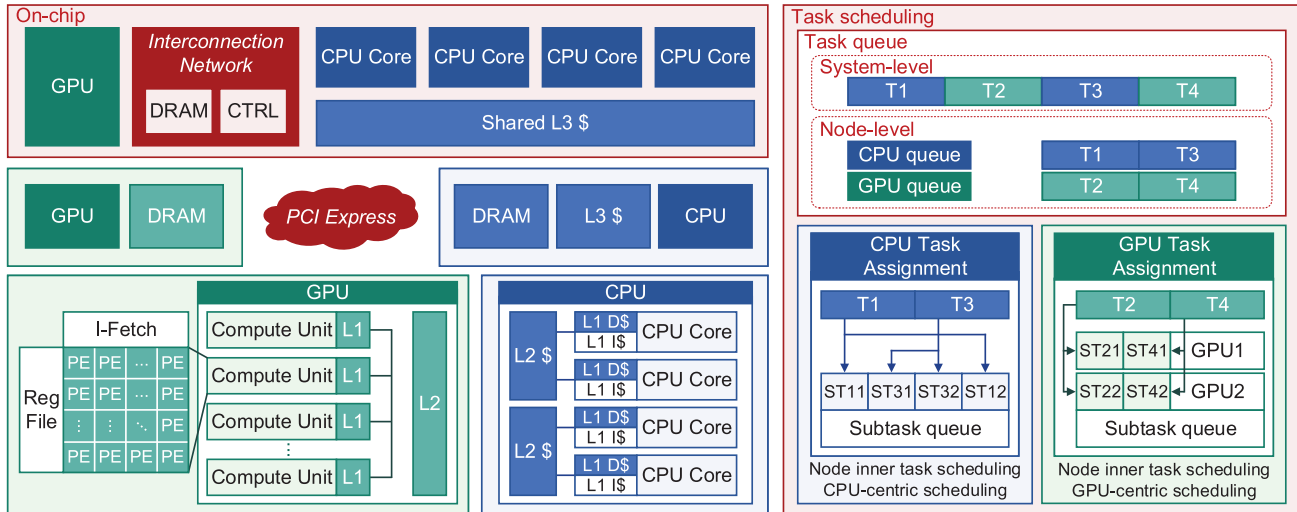


Fig. 4. Heterogeneous CPU and GPU architectures with discrete or on-chip integrated structure, with task scheduling diagrams illustrating GenAI workload coordination.

and reordering to leverage NVIDIA Tensor Cores for acceleration. Wang et al. [222] proposed a GPU-based optimization for sparse Tensor operations using Tensor Core units (TCUs), developing a hybrid Tensor format and parallel algorithm to address sparse Tensor irregularities, improving memory bandwidth and computational efficiency. Guo et al. [223] proposed a new tile-vector-wise (TVW) sparsity pattern for accelerating sparse DNNs on GPUs, combining tile-level and 2:4 register-level sparsity for efficient sparse Tensor Core execution.

For customized Tensor Core architectures, Bai et al. [24] proposed hardware-level Tensor compilation optimization for transformer tasks, using dynamic programming and CUDA kernel generation to optimize fused operators. Huang et al. [25] proposed a GPU Tensor Core architecture optimized for sparse DNNs, utilizing native sparse encoding and row-merge dataflow to enhance energy efficiency during training and inference. For Tensor Core utilization and innovations, Ho et al. [224] proposed a hardware-based optimization to improve GPU resource utilization, focusing on offloading GEMM operations between Tensor and CUDA cores, achieving a 19% performance gain. Sun et al. [225] proposed GPU Tensor Core optimization for sparse deep neural network inference, introducing matrix transformation and emulation algorithms to efficiently utilize Tensor Cores for sparse and single-precision workloads.

For memory optimization, Du et al. [226] proposed fine-grained GPU-level computation and memory optimization for transformer inference to reduce redundancy in multi-layer perceptron (MLP) computations and optimize memory allocation using GPU-specific data layouts. Song et al. [227] proposed a GPU-CPU hybrid inference engine that leverages the power-law distribution of neuron activation in LLMs, preloading frequently activated “hot neurons” on the GPU while offloading “cold neurons” to the CPU, achieving high-speed inference on consumer-grade GPUs with minimal accuracy loss. To illustrate the architectural layout and workload management in

GenAI deployments, Fig. 4 presents heterogeneous CPU and GPU architectures, highlighting both discrete and on-chip integration, along with representative task scheduling strategies. Kwon et al. [228] proposed PagedAttention, a virtual memory-inspired algorithm to reduce KV cache fragmentation and redundancy. And Zhang et al. [229] extended PagedAttention with JENGA, using a two-level GPU memory allocator to optimize caching, eviction, and minimize fragmentation for LLM embeddings’ diverse sizes and patterns. In resource-constrained and edge deployments, Dun et al. [230] proposed adaptive memory and compute optimization for sparse DNN inference on cost-efficient GPUs, improving performance by reducing memory usage via memory-aware data partitioning and fusing Tensor Core and CUDA Core operations. Sheng et al. [231] proposed FlexGen, a high-throughput LLM inference engine that enables efficient large-model execution on constrained GPUs via optimized tensor offloading, 4-bit compression, and linear programming-based scheduling.

In the area of quantization and mixed-precision techniques, Kluska et al. [232] proposed efficient GPU kernels for mixed-precision execution in vision transformers, achieving up to 7× speedup.

3) *Tensor Processing Units:* Research on TPU optimization focuses on enhancing transformer model inference efficiency and sparse matrix handling. Du et al. [233] proposed a TPU-based hardware architecture optimized for attention mechanisms in transformers, introducing a fine-grained padding method to address sequence length variability in scaled dot-product attention on TPUs. Elbity et al. [234] proposed an approximate TPU design for improved performance and efficiency, focusing on approximate computing techniques applied to TPU systolic arrays. This includes low-precision multipliers and operand preprocessing, with approximate processing elements (APEs) and pre-approximate units (PAUs) to optimize systolic array operations. FlexTPU [235] was proposed for sparse matrix optimization in TPU-like architectures, introducing Z-shape mapping and sparse data

TABLE V
COMPARISON OF GPU/TPU ACCELERATORS FOR GENAI

	Year	Platform	Networks	Speedup	Energy efficiency
Huang <i>et al.</i> [25]	2023	GPU	DNN	1.39×DS-STC [236]	1.50×DS-STC [236]
Xia <i>et al.</i> [219]	2023	GPU	LLM	3.8×deepspeed [237]	significantly higher than deepspeed [237]
Gondimalla <i>et al.</i> [26]	2023	GPU	CNN/Transformer	4.8×dense implementation	3.1×dense implementation
FlexTPU [235]	2022	TPU	Sparse Matrix Compute	3.55× Sparse-TPU [238]	3.27× Sparse-TPU [238]
Elbittiy <i>et al.</i> [234]	2022	TPU	DNN/CNN	5.2×TPU [239]	4.4× TPU [239]

TABLE VI
TAXONOMY OF SPECIALIZED ACCELERATORS

FPGAs	Approaches and tools	FPGA acceleration for transformers and GenAI	[240] [27] [29]
	FPGA optimization	Memory bottleneck handling, low-latency, and sparse attention	[241] [28] [242]
	FPGA-based models	Optimization for multi-head attention and sparse transformers	[243] [244]
	FPGA accelerators	NLP/CV tasks and flexible overlay processors	[245] [246] [247] [248] [249] [250] [251] [252]
CRGAs	Approaches and tools	ECGRA frameworks for CNNs and transformers	[253]
	CGRA optimization	Hardware optimization for SoC design and mapping	[254]
	Genetic algorithm-based methods	Kernel partitioning and mapping	[255]
DSPs	Convolution Acceleration	CNN acceleration via Winograd on DSPs	[256]
	Compiler/System Optimization	DNN optimization on mobile DSPs with SIMD/VLIW	[257]
	Matrix Factorization Optimization	SVD acceleration with vectorization and DMA on FT-M7002	[258]
ASICs	Approaches and tools	ASIC accelerators for transformer models	[259] [260] [32]
	ASIC optimization	Hybrid architectures for GPT inference	[31] [30]
	Energy-efficient ASICs	Sparse self-attention and quantized transformer optimization	[261] [262]
	In-memory accelerators	Token-pruning scheduling and nonvolatile memory	[32] [263]
	Logic-centric ASIC accelerators	Utilize Sparse Transformers	[264] [199] [198]

loaders to efficiently handle sparse matrix computations on modified TPU systolic arrays.

B. Specialized Accelerators

Specialized accelerators are optimized for GenAI tasks, offering high-performance parallel processing to accelerate model training and inference, improving efficiency over general-purpose processors. Table VI presents a taxonomy of specialized accelerators.

1) *Field Programmable Gate Arrays*: Recent research has introduced various FPGA-based accelerations to optimize the performance of GenAI models and transformer architectures.

Hong et al. [240] proposed low-latency hardware implementations for GPT-2 inference, optimizing dataflow and resource utilization on multi-FPGA platforms and edge devices. Wang et al. [27] and Peng et al. [29] proposed FPGA acceleration for GenAI tasks, addressing memory bottlenecks through techniques like nonlinear-operator fusion, dynamic pipelining, and sparse attention mechanisms. Ye et al. [241], Du et al. [28] and Calabash [242] proposed FPGA accelerators for multi-head attention and self-attention mechanisms in transformer models, introducing systolic arrays, energy-efficient optimization, and efficient memory usage. Li et al. [243] and Li et al. [244] proposed optimization for dynamic sparse transformers and mixed-precision accelerators, reducing computation and memory demands via techniques like token pruning and bfloat16/integer operations. Qi et al. [245] and Zhao et al. [246] proposed reconfigurable systolic arrays and

model compression techniques for FPGA, focusing on efficient transformer deployment with memory optimization like sparse matrices and token packing. Zeng et al. [247] and Qin et al. [248] proposed large-scale transformer-based LLMs, leveraging adaptive compilation, sparse digital signal processor (DSP) chains, and tiling softmax to optimize performance for long input sequences. HPTA [249], Wang et al. [250], and Bai et al. [251] proposed high-performance FPGA accelerators, combining optimization for both neuro-linguistic programming (NLP) and computer vision (CV) tasks, as well as flexible overlay processors for improved speed and lower latency compared to other platforms. Chen et al. [252] proposed a spatial acceleration architecture that assigns dedicated hardware units to specific operators or layers, connecting them directly via a dataflow design to minimize off-chip memory accesses.

2) *Coarse-Grained Reconfigurable Arrays*: Several recent studies have proposed innovative coarse-grained reconfigurable arrays (CGRA)-based frameworks and optimization techniques to enhance the performance, accuracy, and scalability of computational models such as CNNs and transformers.

Mao et al. [253] proposed a CGRA-based framework for accelerating CNNs and transformers, with hardware optimization for system on chip (SoC) design, nonlinear functions, and mapping schemes to improve computational efficiency for NLP models. VecPAC [254] introduced a precision-aware, vectorizable CGRA design that combines scalar and vector functional units to optimize computational accuracy, enhancing performance and scalability. Kojima et al. [255] proposed a

genetic algorithm-based kernel partitioning method for CGRA application mapping, optimizing resource usage and reducing mapping failures through a deep graph CNN (DGCNN) model that predicts mappability and resource requirements.

3) *Digital Signal Processors*: Recent studies have proposed various methods to optimize DNN computations on DSPs, focusing on different aspects such as convolution acceleration, compiler systems, hardware-algorithm co-design, and low-power accelerators for edge devices.

Li et al. [256] proposed a method for accelerating CNN computations on DSPs, optimizing convolution layers using the Winograd algorithm on general-purpose DSP (GPDSP) to improve parallelism and reduce computation time. Niu et al. [257] proposed a compiler system for optimizing DNN workloads on mobile DSPs, focusing on SIMD and vector instructions, very long instruction word (VLIW) pipelines, and global operator mapping for efficient inference. Xie et al. [258] proposed an singular value decomposition (SVD) optimization method for DSPs using vectorization and SIMD. The efficient SVD on the FT-M7002 DSP was implemented by leveraging SIMD parallelism, vector processing, and optimized DMA for data movement.

4) *Application-Specific Integrated Circuits*: Several researchers have proposed ASIC-based accelerators targeting various bottlenecks in transformer models, focusing on optimizing energy efficiency and performance for GPT inference and self-attention tasks.

Wu et al. [259] proposed a processing in memory (PIM)-based accelerator for GPT inference using a hybrid PIM-ASIC design, directly performing matrix operations in DRAM while using ASIC for non-linear functions, optimizing memory-bounded tasks. Guo et al. [260] and Sridharan et al. [32] proposed in-memory accelerators for transformer models. CIMFormer targets attention challenges with token-pruning-aware scheduling for energy-efficient compute-in-memory (CIM) operations, while X-Former reduces latency and energy in matrix-vector multiplications for attention using non-volatile memory (NVM). Wang et al. [31] proposed an energy-efficient ASIC targeting GenAI self-attention bottlenecks, introducing dynamic processing elements to reduce energy consumption from weakly related tokens. Wang et al. [30] also proposed an ASIC with techniques like principal-component-prior speculation unit (PCSU), similar-vector tracked computing engine (STCE), and bit-wise stationary processing element (BSPE) to optimize energy usage, achieving 77.35 TOPS/W. Mao et al. [261] proposed a hybrid analog resistive random access memory (ReRAM) and digital SRAM in-memory accelerator for sparse self-attention in transformers, focusing on energy efficiency through the use of ReRAM and SRAM. Marchisio et al. [262] proposed a specialized ASIC for quantized transformer models, optimizing fixed-point quantization to achieve high speed and low power consumption. Qin et al. [263] proposed a transformer-specific hardware accelerator with sparse optimization for attention and linear layers, addressing energy and throughput challenges while boosting throughput for varying input lengths.

Lin et al. [264] proposed a sparse transformer accelerator design with architectural and hardware-level optimizations.

Energion [199] proposed a multi-round filtering algorithm and co-processor architecture to improve efficiency with dynamic sparsity. ASADI [198] proposed a sparse attention accelerator with hardware-software co-design for Transformers.

C. Emerging Accelerators

Emerging accelerators offer novel architectures that integrate memory and computation, enhancing efficiency and scalability for GenAI tasks over traditional accelerators. Table VIII presents a taxonomy of emerging accelerators.

1) *Processing in Memory*: PIM has integrated computational capabilities within memory to reduce data movement between memory and processors, thereby enhancing performance and energy efficiency. It preserves the traditional separation between the processing and memory units, while reducing the physical distance between them compared to conventional architectures. Considering that GenAI involves substantial data reading and writing, PIM significantly enhances computational and energy efficiency to accelerate the training and inference of deep learning models. SIMDRAM framework [265] established an adaptable and versatile architectural framework for bit-level parallel computation through innovative manipulation of DRAM cell arrays. This design achieved optimized execution of computationally intensive tasks while maintaining architectural extensibility for custom operator integration. PIM-DRAM architecture [266] implemented distributed arithmetic units for multiplicative operations within DRAM subarrays, with subsequent accumulation and nonlinear transformation processes being orchestrated at the bank level.

Several accelerators that adopt the bank-level PIM approach have been designed. StreamPIM [267] presented a novel PIM architecture using racetrack memory (RM), where domain-wall nanowires directly constructed matrix processors, demonstrating substantial enhancements in computational throughput and energy proportionality for matrix operations. Newton [268] introduced a fixed data flow accelerator optimized for matrix-vector multiplications, implemented at the HBM bank level to enhance data flow efficiency for machine learning tasks. HBM-PIM [269] implemented FIMDRAM architecture with bank-embedded 16-wide SIMD engines, employing bank-level concurrency to obtain magnitude-higher bandwidth than off-chip solutions.

Commercial PIM has been deployed in practice, featuring both application-specific optimizations and versatile multi-domain implementations. uPIMulator [270] introduced a novel simulation framework targeting UPMEM's commercial general-purpose PIM architecture, demonstrating its applicability for computer architecture research. It established the inaugural industry-standard general-purpose PIM architecture via systematic characterization analysis, defining critical research frontiers for computer architects.

2) *Quantum and Photonic Computing*: Quantum and photonic computing offer new approaches to addressing complex computational challenges in GenAI. These technologies have been used to accelerate specific computational tasks through non-traditional computing methods, particularly those that are

TABLE VII
COMPARISON OF FPGA/ASIC ACCELERATORS FOR GENAI

	Year	Platform	Networks	Throughput (GOP/s)	Energy efficiency (GOP/J)
Du <i>et al.</i> [28]	2024	ZCU102	Transformer	944.87	681.72
Peng <i>et al.</i> [29]	2022	FPGA	Transformer	3600	102
Li <i>et al.</i> [243]	2024	ZCU102	Dynamic Sparse Transformer	634.27	170.50
Qin <i>et al.</i> [248]	2024	Alveo U50	BERT-Base	1420	29.58
Wang <i>et al.</i> [250]	2024	FPGA	BERT-Base	1223.90	26.9

TABLE VIII
TAXONOMY OF EMERGING ACCELERATORS

PIM	In-subarray	Computation within DRAM cells	[265] [266]
	Bank	Computation with memory banks as basic units	[267] [268] [269]
	Commercial	Standardized PIM toolchains	[270]
Quantum and Photonic Comput.	Quantum	Optimization via superposition and entanglement with HPC	[271] [272]
	Photonic	Optimization via lightspeed transmission and parallel processing	[273]
IMC	SRAM	Digital compute-in-memory and high compatibility with CMOS	[274] [275]
	RRAM	Physical laws as computation instructions	[276]
	PCM	Non-volatile analog compute-in-memory	[277]
MCMs	Domain-specific	Distributed integrated computing	[278] [279] [280] [281] [282]
	Commercial	Economics-driven compute scaling	[283] [284]
Neuromorphic Accelerators	Hardware	Asynchronous architectures mimicking biological neurons	[285] [286]
	Hybrid	Integration of SNNs with ANNs	[287] [288] [289]
	Applications	Hybrid neuromorphic platforms with SNN integration	[290]

challenging for traditional computing systems. Quantum computing, with properties like superposition and entanglement, provides higher parallelism and efficiency for specific computational tasks, making it suitable for optimization algorithms and big data analysis.

A survey on quantum computing (QC) technologies [271] has revealed that quantum machine learning presents a highly promising research avenue. This discipline concentrated on designing quantum computational methods that surpass classical algorithms in ML applications, with specific emphasis on recommendation systems. Aligned with this direction, Google released TensorFlow Quantum (TFQ) [272], a publicly accessible quantum ML library for experimental implementation of hybrid quantum-classical models.

Photonic computing shows significant potential in accelerating matrix computations in GenAI via light-speed transmission and parallel processing, especially for large-scale transformer networks used in natural language processing. Zhu et al. [273] developed Lightning-transformer, a domain-specific photonic acceleration architecture, along with an innovative photonic tensor processing unit structured as reconfigurable optical vector dot-product crossbars. This approach resolved intrinsic constraints in prior designs, achieving massively parallel computation for dual dynamic full-range matrices.

3) *In-Memory Computing*: In-memory computing (IMC) has been instrumental in surmounting the obstacles related to memory and processing in AI tasks involving extensive data. It integrates computation within memory to diminish data transfer, enhance energy efficiency, and boost performance

while capitalizing on the parallelism of memory to augment computational throughput. Compared with PIM, IMC eliminates the traditional boundary between processing and memory units, with both data storage and computation occurring within the same unit.

Full-digital IMC has been focusing on digital designs using digital logic and control mechanisms for efficient computation. Desoli et al. [274] demonstrated 77 TOPS/W full-stack system energy optimization and 13.6 TOPS/mm² area efficiency, specifically optimized for inference acceleration. NeuroCIM [275], an alternative digital in-memory computing architecture, achieved record 310.4 TOPS/W via hardware-implemented dynamic memory reduction and conditional computation termination.

RRAM-based IMC has combined analog and digital designs to achieve efficient matrix-vector multiplications. Cao et al. [276] identified persistent ADC-induced energy bottlenecks in conventional PIM ReRAM accelerators. Their co-optimized architecture employed analog charge accumulation combined with neural-approximation peripheral circuitry, effectively reducing DC-stage operations to enhance deep learning acceleration efficiency.

Phase change memory (PCM)-based IMC uses analog circuits and optimized ADCs for efficient computation. Hermes [277] represented the inaugural PCM-based silicon prototype specifically engineered for DNN inference acceleration. It incorporated a 256×256 PCM cross-bar array with precision-engineered ADC components. This co-optimized design minimizes readout latency and energy overhead, providing efficient solutions for DNN inference.

4) *Multi-Chip Modules*: Multi-chip modules (MCMs) have been proposed to integrate multiple small chips into a single system to form a high-performance platform suitable for large-scale DNN models in inference and training. They leverage chip-to-chip parallelism and efficient interconnect technologies to enhance performance. Verhelst et al. [291] investigated the recent trend of multi-core in the evolution of deep-learning accelerators. This trend was recognized as a critical approach for boosting computational throughput to address escalating processing demands.

Chimera [278] developed a non-volatile integrated chip architecture targeting DNN training/inference, achieving complete elimination of off-chip memory reliance. Its multi-chip scaling methodology enabled large-model inference beyond single-chip memory capacity with merely 5% energy overhead. The Arvon accelerator [279] implemented an advanced system-in-package design incorporating heterogeneous FPGA-DSP chiplets. It was designed to support diverse workloads through a specialized compilation procedure optimizing workload distribution across the chiplets. SPRINT [280] introduced a photonic-based interconnect. It has been adopted as a next-generation solution replacing metallic inter-chiplet connections in DNN inference accelerator designs. Simba [281], a seminal chiplet-based DNN accelerator, implemented scalable 36-chiplet configurations demonstrating optimized area/energy efficiency and peak performance across single-chiplet to full-system scales. Tan et al. [282] proposed a 2.5D-packaged accelerator prototype with heterogeneous integration of 1 HUB chiplet and 6 SIDE chiplets on RDL layers, delivering 10/20/40 TOPS peak performance at INT16/8/4 precisions.

MCM technology plays a core role in commercial implementations. AMD Instinct [283] employs chiplet-based acceleration for exascale high-performance computing (HPC) and AI workloads. Tesla DOJO [284] utilizes homogeneous modular design with hierarchical chiplet organization, specifically optimized for ML training.

5) *Neuromorphic Accelerators*: Neuromorphic accelerators mimic biological neural networks to deliver efficient computation and energy savings, excelling in real-time and adaptive learning applications. Utilizing asynchronous designs for efficient spike data processing, they are ideal for real-time and adaptive tasks. While sharing common goals, each accelerator offers unique features and applications.

Loihi [285] established neuromorphic hardware foundations. It was a fully asynchronous design that embedded between 114K (9-bit) and 1M (binary) synapses, along with up to 180K neurons per chip. SpiNNaker2 [286] emphasized hybrid designs and scaling mechanisms necessary for large AI workloads.

Lu et al. [287] dominated hybrid spiking neural network (SNN)-artificial neural network (ANN) integration research, focusing on energy-delay trade-offs, signal conversion, and addressable use cases like sparse generative sub-tasks. They provided a framework for implementing hybrid ANN-SNN models on neuromorphic hardware. It explored the Tianjic chip's architecture enabling efficient ANN-SNN integration with low latency, low power, and minimal accuracy loss.

In addition, they proposed hybrid accelerator for SNNs and ANNs on shared hardware [288]. It focused on optimizing signal transformations, reducing communication delays, and energy-efficient mappings critical for adapting GenAI workloads to neuromorphic systems.

While full integration of GenAI (e.g., GANs, transformers) on neuromorphic chips remains unachieved, researchers have explored task partitioning, sequence processing, and hybrid ANN-SNN preprocessing. Seekings et al. [289] analyzed the deployment of hybrid SNN-ANN architectures in neuromorphic computing and edge AI platforms. They utilized Intel's Loihi for SNNs and Jetson Nano for ANNs in a hybrid system, performing comprehensive benchmarking across accuracy, latency, power consumption, and energy efficiency metrics. Lu et al. [290] provided an overview of hybrid neuromorphic platforms with SNN integration, exploring the co-design of SNN algorithms, hardware, and applications to harness power efficiency and bio-inspired computing capabilities.

D. Computing Clusters

The rapid growth of GenAI, including GANs and LLMs, has intensified demands on computing clusters for performance, scalability, and energy efficiency. Heterogeneous computing leverages specialized hardware like FPGAs and ASICs alongside automated design frameworks to optimize GAN operations. Simultaneously, data centers increasingly employ GenAI and machine learning for thermal management, fault detection, and energy optimization. This bidirectional relationship-where GenAI both benefits from and enhances computing clusters-is driving hardware and infrastructure evolution, enabling efficient deployment of next-generation AI systems.

1) *Heterogeneous Computing*: Heterogeneous computing plays a crucial role in optimizing hardware accelerators for GANs, leveraging platforms like FPGAs, ASICs, and automated design tools to achieve higher energy efficiency, computational performance, and scalability. Recent works explore various methods for enhancing hardware designs, focusing on FPGA-based accelerators serving as reconfigurable components in heterogeneous computing systems, energy-efficient architectures, and GAN-specific hardware optimization.

FPGA-Based accelerators for GANs have been extensively studied, with several works focusing on improving performance for GAN-specific operations. Di et al. [292] introduced an FPGA-based accelerator tailored for transposed convolution layers in GANs, utilizing the Winograd algorithm and parallelism-aware techniques to optimize memory partitioning and computational efficiency. Similarly, Kim et al. [293] demonstrated a FPGA accelerator which improved energy efficiency for GAN training by implementing selective-layer retraining and layer reordering, which reduced computation and memory access overhead. Yin et al. [294] further enhanced FPGA-based GAN acceleration by introducing a reconfigurable architecture optimized for the distinctive computational patterns seen in GANs, improving both throughput and energy efficiency during training. In terms of energy-efficient and scalable architectures, several works have focused on designs

tailored to low-power GAN applications. Alhussain and Lin [48] presented an integrated hardware-software design targeting FPGA platforms for GAN implementation, specifically targeting quantized deconvolution GANs (QDCGAN) for low-power inference on heterogeneous edge computing platforms. This scalable architecture was designed to efficiently support domain-specific GAN operations. Yang et al. [295] advanced this field by introducing a fast algorithm paired with a reconfigurable architecture to accelerate deconvolution layers in GANs. Their approach, validated through CycleGAN implementations on diverse heterogeneous hardware platforms, significantly enhances computational efficiency.

2) *Data Centers*: Data centers face significant challenges in thermal management, energy consumption, and fault diagnosis, particularly as AI workloads scale. Emerging studies demonstrated the application of GenAI and ML techniques to support engineering design and operational optimization in data centers. These methods aim to improve infrastructure performance through data-driven modeling, predictive analytics, and generative design. In particular, applications such as heat optimization, fault detection, and network architecture have benefited from the integration of GenAI and traditional ML models. The following works demonstrate how these techniques contribute to enhancing data center efficiency through innovative, GenAI-driven engineering solutions. Thermal optimization through ML techniques has been recognized as a key focus in enhancing data center efficiency. Sarkar et al. [296] proposed a machine learning-based computational fluid dynamics (CFD) surrogate using a 3D CNN (U-Net) to generate accurate heat maps. This approach enabled thermal optimization and enhanced cooling strategies, leading to better energy efficiency in data centers. Similarly, Flynn and Qian [297] combined GANs with CNNs to optimize heat sink designs, generating innovative solutions that enhance thermal efficiency. Their approach integrated GAN-generated designs with CNN-based temperature predictions, effectively addressing thermal challenges at the hardware level. Du et al. [298] tackled the challenge of finite fault data, which was then integrated with support vector machine (SVM). This approach demonstrated significant potential in improving the reliability of data center cooling systems. Energy efficiency and AI workloads have been a growing concern as the energy demands of GenAI increased. Wood [299] discussed the energy-intensive nature of these workloads in data centers, highlighting the environmental challenges but without focusing on specific AI-driven optimization techniques. In contrast, Ilager et al. [300] applied data-driven machine learning, specifically gradient boosting models, to predict host temperatures in hyperscale cloud data centers. Ilager further proposed a dynamic scheduling algorithm that reduced peak temperatures and overall energy consumption, showcasing how machine learning can be leveraged to optimize thermal management in large-scale data centers.

Data center network design has played a critical role in optimizing AI workload performance, particularly for LLMs. Alibaba HPN [301] introduced a data center network specifically designed for LLM training, utilizing a dual-plane architecture with a dual top-of-rack design. This architecture

addressed issues such as hash polarization and single points of failure, and has been successfully deployed in production environments for over eight months, demonstrating its effectiveness in managing the demands of large-scale AI training workloads.

V. GENAI FOR EFFICIENT CIRCUITS AND SYSTEMS DESIGN

The rapid advancement of GenAI has created an unprecedented demand for computational power, posing significant challenges to the design of circuits and systems. Conventional design flows of circuits and systems are cumbersome and time-consuming, requiring months of iterative refinement across abstraction layers. Although numerous prior works have attempted to automate circuit and system design [302], [303], [304], [305], [306], [307], [308], they have primarily served as auxiliary tools, yet to replace human designers. The exceptional capabilities of GenAI, however, have revealed immense potential for transforming the design process. By leveraging GenAI, researchers have made notable attempts to optimize the design pipeline, covering logic design, physical design, verification and test, as well as analog design.

A. Logic Design

Using GenAI, researchers have made significant progress in automating hardware logic design. Including but not limited to hardware description language (HDL) code generation, design space exploration, logic synthesis and design assisting. The taxonomy of GenAI for logic design is summarized in Table IX.

1) *HDL Code Generation*: In effort to automate digital hardware design, recent studies leveraged GenAI to generate HDL code. Most significantly, using LLMs to convert natural language specifications into HDL code has been a popular research direction.

DAVE [52] was the first to utilize the GPT-2 model fine-tuned on a synthetic dataset derived from hand-written templates to convert natural language into HDL code. Although DAVE demonstrated the potential of LLMs for HDL code generation, it was limited by the suboptimal quality of the generated code and the lack of a comprehensive dataset.

Following DAVE's footsteps, many works proposed more advanced LLM-based solutions for HDL code generation. RTLCoder [309] introduced an efficient LLM-based solution for Verilog HDL code generation by leveraging a synthetic RTL code dataset and a tailored LLM algorithm. Nadimi and Zheng [310] developed a multi-expert LLM architecture that integrates multiple LLMs, each fine-tuned for different levels of design complexity, thereby enabling more targeted and accurate code generation. VeriGen [312] focused on partial Verilog HDL code completion. Delorenzo et al. [311] addressed syntax errors, functionality errors, and power, performance, and area (PPA) unawareness by proposing an automated transformer decoding algorithm that incorporates Monte Carlo tree-search for predictive lookahead.

To evaluate the performance of LLMs in HDL code generation, researchers have developed various benchmarking frameworks. Thakur et al. [58] constructed a benchmarking

TABLE IX
TAXONOMY OF GENAI FOR LOGIC DESIGN

HDL Code Generation	LLM fine-tuning	Generating HDL code based on natural language specifications	[52] [309] [310] [311]
		Multi-expert LLMs for different levels of complexity	[310]
		Partial Verilog HDL code completion	[312]
	Benchmarking LLM-HDL code generators	Syntax errors of generated HDL code	[58] [313] [314] [315] [316] [317] [318]
		Functional errors of generated HDL code	[58] [313] [315] [316] [317] [318]
		PPA metrics of generated hardware designs	[313]
		Creativity metrics of generated designs	[319]
		HDL code completion benchmarks	[314]
		Verilog HDL-targeted benchmarks	[58] [319] [314] [315] [317]
		VHDL-targeted benchmarks	[316]
		Python-embedded DSL-targeted benchmarks	[317]
		General HDL-targeted benchmarks	[313] [318]
	Dataset construction	HDL code datasets	[310] [312] [311] [58] [313] [315] [316] [320]
		Natural language description-HDL code datasets for training	[309] [310] [321] [322]
		Synthetic HDL code datasets	[309] [321] [320]
		GenAI for HDL code labeling	[310] [315] [321] [322]
		GenAI for HDL code data augmentation	[323]
		Domain-specific HDL datasets	[324] [325]
	Prompting techniques	Prompt engineering techniques for LLM-HDL code generators	[326] [327] [320] [328]
		Post-LLM search processes	[329]
	LLM-based design flows	Using syntax-check as feedback	[330] [331] [332] [59] [333]
		Using functional verification as feedback	[331] [332] [59] [333]
		Using PPA evaluation as feedback	[59] [333]
		Using generative discriminators to produce feedback	[334]
		Domain-specific applications	[335] [334] [336]
		Natural language formalization	[337]
	Multimodal inputs	Utilizing visual representations	[338] [339]
		Translating C to HLS-C code	[340] [341] [342]
		Domain-specific representations	[57] [343] [344]
Design Space Exploration	PPA prediction	Pre-synthesis PPA prediction using GenAI	[345] [346]
		PPA prediction and DSE in latent space using GenAI	[347] [55]
	Domain-specific DSE	Generating and optimizing NoCs with GANs	[348] [349]
Logic Synthesis	Operator synthesis	Approximate operator synthesis using GANs	[350]
	DAG generation	Generating DAGs using diffusion models	[351]
	Sequence generation	Performing logic synthesis as a sequence generation task	[352] [353]
Design Assisting	Documentations	Generating documentations from formal specifications using LLMs	[53] [354]
	Helper agents	Chip design-focused LLMs with domain-specific adaptations	[355] [356] [51]
		Evaluating TCO of LLMs for EDA script generation	[357]
	Applications	Designing specific circuits with LLMs	[358] [359] [360]

framework with functional and syntax metrics for LLMs generating Verilog HDL code. Under this framework, the authors benchmarked fine-tuned LLMs (with datasets collected from open-source repositories and textbooks) and commercial pre-trained LLMs. RTLLM [313] introduced a comprehensive open-source benchmark incorporating syntax, functional, and PPA metrics. CreativEval [319] introduced a creativity evaluation framework that assesses fluency, flexibility, originality, and elaboration in the generated code. RTL-Repo [314] introduced a benchmark specifically for generating large-scale RTL projects, including a dataset of 4000+ Verilog HDL

code samples from public repositories. Verilog-Eval [315] and VHDL-Eval [316] focused on benchmarking LLMs for Verilog HDL and VHDL code generation, respectively. Focusing on Python-embedded domain-specific languages (DSLs), PyHDL-Eval [317] benchmarked LLMs for HDL generation and observed that existing language models perform better on RTL code generation than on Python-embedded DSLs. HDLEval [318] benchmarked LLMs for generating multiple HDLs.

To facilitate the training and assessment of LLMs in the context of HDL code generation, researchers have

undertaken extensive efforts to collect HDL code snippets and develop data augmentation frameworks to create larger synthetic datasets. Thakur et al. [58] utilized a comprehensive dataset sourced from open-source repositories and textbooks, providing a solid foundation for benchmarking LLM performance. RTLCoder [309] introduced an RTL dataset generation pipeline that incorporates design instructions alongside RTL codes, enhancing the diversity and applicability of the training data. Chang et al. [321] developed a data augmentation framework that generates natural language descriptions aligned with Verilog HDL code and EDA scripts, additionally supporting the creation of erroneous Verilog HDL code to facilitate error correction training. MG-Verilog [322] showcased a multi-grained Verilog HDL dataset that includes descriptions provided at different levels of abstraction paired with corresponding code samples, thereby capturing a wide range of design complexities. SA-DS [324] introduced a specialized dataset for systolic array-based accelerators in Chisel using the Gemmini generator template [361]. Fang et al. [323] leveraged LLMs for RTL data augmentation, enhancing the robustness and variability of the training data. Liao et al. [325] proposed a synthetic high-level synthesis (HLS) dataset generation framework that adapts VAEs and GANs, further expanding the scope and quality of available datasets.

To enhance the accuracy and quality of LLM generated HDL code, various prompting techniques have been employed. Nekkab et al. [326] introduced a hierarchical prompting approach for generating complex hardware modules, its effectiveness was demonstrated on benchmark circuits with explicit hierarchical structures. Du et al. [327] utilized in-context learning (ICL) and chain-of-thought (CoT) prompting techniques to generate a 64-point Verilog HDL fast Fourier transform (FFT) module, highlighting the potential of these methods in designing signal processing modules. Vijayaraghavan et al. [320] proposed a chain-of-descriptions strategy for VHDL generation, which employs a series of intermediate descriptive steps to guide LLMs in generating VHDL code. This method was evaluated using the VHDL-Eval and the authors' newly introduced VHDL-Xform dataset, comprising VHDL codes from open-source repositories and their synthetic variations. SynthAI [328] integrated ReAct agents, CoT prompting, the retrieval-augmented generation (RAG) framework, and web searching to generate HLS code with LLMs, which is subsequently translated into HDL code using HLS tools.

In addition to various prompting techniques, recent studies have integrated LLMs into HDL design workflows using feedback mechanisms to guide the generation process. Chang et al. [329] introduced a post-LLM search process that integrates design space exploration into the HDL generation workflow, thereby improving design quality. To address syntax errors in LLM-generated Verilog HDL, RTLFixer [330] developed a debugging framework that leverages RAG and ReAct prompting in combination with a syntax-checking feedback loop for interactive code debugging. Addressing functional errors, Chip-Chat [331] established a conversational framework for HDL code generation that incorporates feedback from verification tools and human interactions, resulting in higher success rate. Building on this, Autochip [332] presented

a fully automated iterative feedback loop utilizing simulators to enhance the functional accuracy of RTL code produced by LLMs. Introducing PPA awareness into HDL generation, ChipGPT [59] leveraged a four-stage closed-loop framework that employs human feedback and PPA evaluations from EDA tools to refine Verilog HDL code generation. Expanding this approach, Thorat et al. [333] presented a dual-stage feedback framework that separately targets functionality and PPA optimization based on functional simulations and synthesis reports. For domain-specific applications, GPT4AIGChip [335] focused on automated AI accelerator design featuring LLM-based prompt generation with a feedback loop, guiding LLMs to produce HLS code of AI accelerators. BetterV [334] integrated generative discriminators trained on domain-specific datasets to guide LLMs in Verilog HDL generation and PPA optimization. Targeting DSLs, ChatChisel [336] utilized multiple expert LLMs within a feedback loop to generate Chisel code, successfully producing a RISC-V CPU and underscoring the potential of LLMs in complex hardware design.

Beyond natural language specifications, researchers have developed various methods that incorporate input formalization and alternative input modalities. nl2spec [337] introduced a framework that leverages LLMs to formalize natural language into temporal logic. Chang et al. [338] proposed a multimodal query language framework that integrates visual representations with natural language descriptions to improve RTL code generation quality, and VGV [339] further advanced this approach by introducing the thinking vision visual prompt method for multimodal LLMs, alongside a benchmark for RTL generation from diagrams.

In addition to formalizing natural language and visual inputs, several approaches focus on translating existing code into HDL. C2HLS [340] developed an automated framework for converting C code to HLS-C using LLMs, while Xu et al. [341] extended this by incorporating automated program repair, bit-width optimization, and PPA optimization through pragma directives. HLSPilot [342] further integrated LLM-based profiling, C/C++ to HLS-C translation, and DSE to fully automate the high-level acceleration flow on hybrid CPU-FPGA platforms.

For domain specific applications, researchers have explored specialized representations. FLAG [57] utilized a formulaic representation of wireless baseband signal processing circuits to formalize inputs, improving the quality of the generated code. Zeng et al. [343] proposed a unified algorithm representation for common operations in various DNNs, enabling the flexible generation of neural network accelerators with LLMs. ChatCPU [344] introduced a processor description language tailored for CPU design automation, successfully generating a RISC-V CPU through fine-tuning LLMs on a specialized CPU dataset.

2) *Design Space Exploration*: DSE is a pivotal aspect of hardware logic design, aiming to identify optimal configurations that balance PPA metrics. Recent advancements have explored the potential of GenAI in DSE. Several studies focus on streamlining DSE through pre-synthesis PPA prediction. VeriDistil [345] leveraged a knowledge distillation technique to convey detailed circuit information through graph-based

representations to a predictor powered by an LLM, enabling accurate PPA evaluations for Verilog HDL code. Chang et al. [346] leveraged VAEs to model the distribution of design features influencing PPA metrics for HLS designs, enabling more accurate and efficient prediction.

In addition to pre-synthesis PPA prediction, other research directly utilized GenAI for DSE. VAESA [347] presented a VAE-based DSE framework that encodes the hardware design space into a dense and continuous latent space, enabling more efficient DSE. GANDSE [55] employed GANs to learn the correlations between design requirements and optimal design solutions.

For domain-specific applications, GANNoC [348] introduced a GAN-based framework for generating network-on-chip (NoC) topologies, facilitating the exploration of diverse NoC configurations. Building on this, Mirka et al. [349] utilized GANs to prune the complex design spaces of heterogeneous NoCs, incorporating GNNs to accurately model NoCs and streamline the design process.

3) *Logic Synthesis*: Several GenAI-based approaches have been developed for logic synthesis. SyFAXO [350] introduced a GAN-based method for approximate operator synthesis, specifically targeting look-up table (LUT)-based structures for FPGA applications. SeaDAG [351] presented a semi-autoregressive diffusion model for generating directed acyclic graphs (DAGs), demonstrating its potential in logic synthesis by producing and-inverter graphs (AIGs). GPT-LS [362] redefined the logic synthesis problem as a sequence generation task and developed a GPT model enhanced with offline reinforcement learning to perform logic synthesis. Li et al. [352] proposed a transformer-based model for logic synthesis by predicting subsequent gates and introduced a logic synthesis rewriting operator based on this model [353].

4) *Design Assisting*: GenAI has also been applied to support and assist designers in various aspects of the logic design process. Fernando et al. [354] proposed a documentation generation framework that utilizes LLMs to generate natural language descriptions from formal specifications of hardware systems. While the generated documentations still require manual review to ensure accuracy, the framework significantly boosts productivity by automating the initial drafting stages [53].

Several studies have employed LLMs as helper agents for logic design tasks. Hardware Phi-1.5B [355] was developed as a specialized LLM tailored for the hardware domain, serving as an AI helper agent that aids in both design and verification tasks. ChatEDA [356] fine-tuned an LLM to function as an autonomous agent for existing EDA tools, effectively managing task analysis, script generation, and execution workflow to streamline the overall design process. Similarly, ChipNeMo [51] explored domain-adapted LLMs for chip design tasks, including assistant chatbots, EDA script generation, and fault analysis. Sharma et al. [357] evaluated the total cost of ownership (TCO) of domain-adapted LLMs for EDA script generation.

Implementing the capabilities of LLMs, Tomlinson et al. [358] designed a spiking neuron array ASIC for fabrication by generating Verilog HDL code with LLM. Vitolo et al. [359]

designed a programmable recurrent spiking neural network, using LLM to generate Verilog HDL code, EDA prompts and testbenches. Salcedo et al. [360] accelerated the design of a vector processor SoC with GPT-4 using LLM for generating the instruction set architecture (ISA), HDL code, system software, and test cases.

B. Physical Design

Focusing on the physical design of ASICs, GenAI has been widely applied to improve the efficiency and quality of the design process. Fig. 5 illustrates key aspects of physical design, demonstrating the potential of GenAI-aided design in floorplan, placement, clock tree synthesis, routing, and design optimization. The taxonomy of GenAI for physical design is shown in Table X.

1) *Floorplan*: Researchers have attempted to leverage GenAI to optimize the floorplan stage. An open-source design flow was introduced by Teo et al. [364], which potentially allowed developers and researchers to leverage GenAI for optimizing chip layouts.

2) *Placement*: Precise placement of components can reduce signal interference and power consumption, thus enhancing the overall performance of the chip. Recently, Lu et al. [365] improved an open-source tool DREAMPlace by using a generative adversarial learning technique to process the location information of components with their connection relationships. In addition, researchers have also developed a tool called GAN-Place [366]. This tool utilizes a GPU-accelerated placer as a generator. By reducing layout errors and integrating with open-source tools such as DREAMPlace and Xplace, it enables better performance in component placement tasks.

Recent works also discussed various strategies applied in placement stage, including supervised learning for prediction [54], [363], unsupervised learning for direct optimization [363], [364], and RL concepts [54], [381]. The scarcity, propriety, and multimodality of data issues, were also mentioned [363], [381].

3) *Clock Tree Synthesis*: Recent research also explored the potential of using GenAI for Clock Tree Synthesis (CTS). GAN-CTS [367], [368] combined GANs and RL to predict and optimize the outcomes of CTS. CircuitOps [369] employed a technique called labeled property graphs (LPGs), which facilitated the handling of data related to CTS. Teo [364] proposed an open-source design flow, combining LLMs with open-source CTS tools.

4) *Routing*: GenAI also showcased potential in the routing stage of IC design. Utyamishev and Partin-Vaisband [370] proposed a multi-terminal pathfinding method. This method utilized CGANs to avoid physical obstacles during routing so as to find more optimal paths.

Park et al. [371] focused on the prediction and optimization of routability. They employed explainable artificial intelligence (XAI) to analyze the influence of input features on the prediction of design rule violation (DRV) hotspots.

For buffer insertion, Liang et al. [372] developed a generative ML framework. The generated solutions were very close to the baseline results obtained from traditional solutions, yet the routing speed was significantly increased.

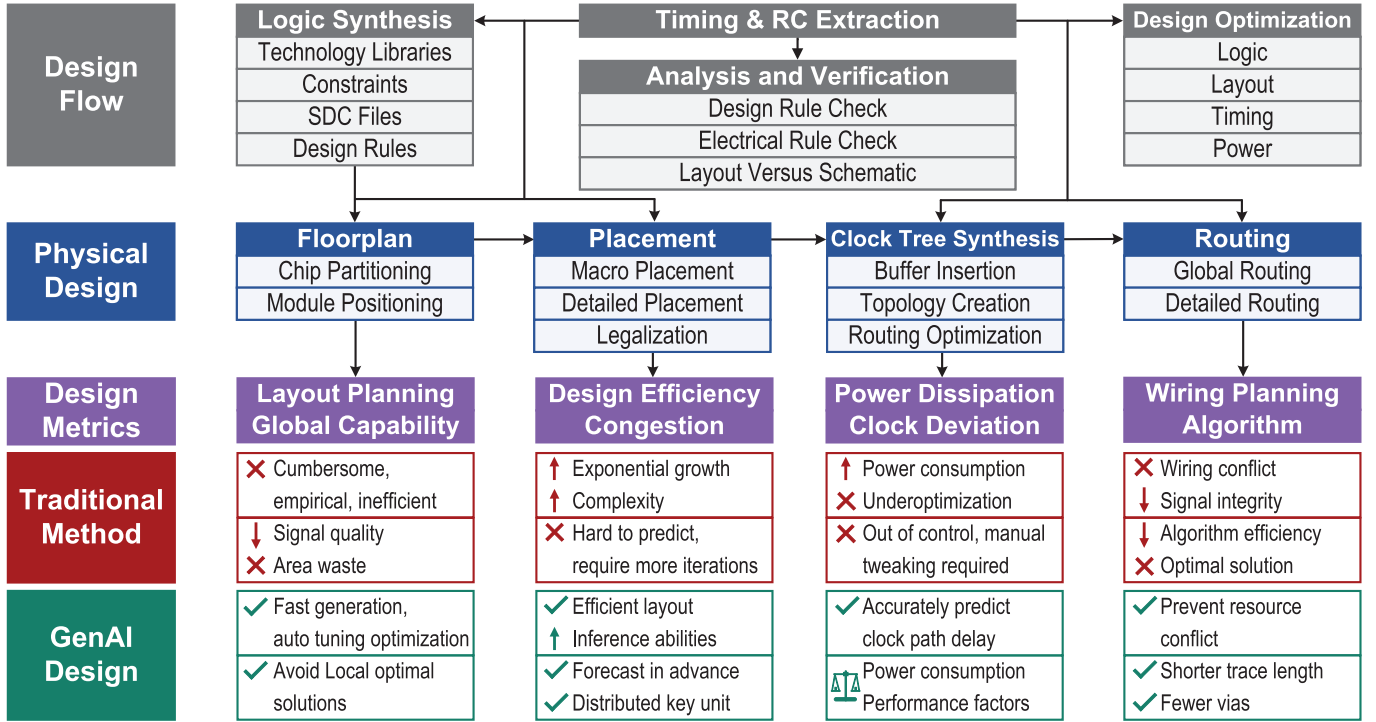


Fig. 5. Key aspects of physical design and the potential of GenAI-aided solutions.

TABLE X
TAXONOMY OF GENAI FOR PHYSICAL DESIGN

Floorplan	Approaches and tools	Open-source and open-access design flows	[363] [364]
	ML	Design module prediction with multiple ML models	[54]
Placement	DREAM-GAN	Data scarcity, proprietorship and multimodality	[365]
	GAN-Place	Enhancement of the quality of DREAMPlace and Xplace	[366]
	ML and RL	Supervised learning for direct optimization	[363] [54]
		Unsupervised learning for direct optimization	[363] [364]
		RL for scarcity, propriety and multimodality of data issues	[54]
CTS	GAN-CTS	Extraction of design attributes and employment of regression	[367] [368]
	GANs	CircuitOps data management	[369]
	LLMs	Open-source design flows	[364]
Routing	cGANs	Multiterminal pathfinding approaches	[370]
	LLMs	Prediction and optimization of routability	[371]
	Transformers	Buffer insertion and construction of buffer-embedded trees	[372]
	CDLS	Constraint-driven frameworks for analog layout synthesis	[373]
Design Optimization	LLMs	Domain-specific tokenization implementation and pretraining	[51]
		PWM design in HDL	[364]
		HDL code generation and integration	[364] [374]
		Data privacy and synthetic data issues	[374]
	Transformers	Parameter encoding processes of FastTuner	[375]
		Circuit component perception research	[376]
	GANs and VAEs	GANs hybrid models in design optimization	[369] [377] [378] [379]
		Applications in design processes with VAEs	[363] [380] [379]

For analog layout synthesis, researchers have introduced a constraint-driven framework [373]. This framework leveraged GenAI to generate crucial design constraints for analog devices, thereby reducing the iteration time of layout design and accelerating the design convergence process.

5) *Design Optimization*: GenAI has been widely applied to aid in physical design optimization. Some works have focused on leveraging LLMs to optimize the design process. ChipNeMo [51], an LLM customized for chip design tasks, has notably enhanced its performance in chip design through domain-specific tokenization techniques. LLMs were

TABLE XI
TAXONOMY OF GENAI FOR VERIFICATION AND TEST

LLM-based Prompt Generation	Prompt generation for HDL applications	Testbench generation	[382] [383] [56]
		LLM evaluation	[317] [316]
		Bug detection and error correction	[384] [385]
		Formal verification	[386] [387]
	Prompt generation for non-HDL applications	System-level test	[388]
		Physical verification	[389] [390]
		Design rule check	[391]
	Open-sourced	Public and academic use	[382] [317] [383] [56] [316]
	Closed-sourced	Buisness and enterprise use	[386] [385] [387] [384]
GenAI-assisted Wafer Test	VAE-based methods	Data augmentation	[392] [393]
		Unsupervised clustering	[394] [395] [396]
		Feature learning	[397] [398] [394]
		Outlier detection	[399] [400]
	GAN-based methods	Synthetic data generation	[401] [402] [403]
		Domain adaptation	[404]
		Multi-scale generation	[405]
	Other methods	Transformer-based models	[406] [407]
		Hybrid architectures	[408]
		Cross-domain integration	[409]

also employed in designing complex circuits [54], as well as generating design code based on prompts and open-source EDA tools [364], [374].

The transformer and self-attention mechanism have also been utilized. FastTuner [375] can quickly explore the design space and optimize physical design parameters. In addition, researchers represented circuit components in the form of point clouds and used the transformer to extract features, enabling highly efficient parallel data processing [376]. Moreover, transformers can also model long-range dependencies and process sequences in parallel [380].

GANs and VAEs are also widely applied in the physical design optimization stage. Lu et al. [377] investigated hybrid models, and Zhao et al. [379] have shown that such hybrid model combining GANs and VAEs can fully leverage the advantages of both. Liang et al. [369] utilized GANs to generate circuit layouts. Kasproicz and Kasproicz [378] illustrated the effective utilization of GANs in enhancing the layout generation process. Wang et al. [380] provided an in-depth exploration of the application for VAEs. Meanwhile, Ghose et al. [363] provided a probabilistic approach that could encode and decode the design space.

C. Verification and Test

The rapid advancement of semiconductor technology has led to increasingly complex hardware designs, posing significant challenges in verification and testing. Traditional verification methods are labor-intensive, time-consuming, and often fail to scale with the complexity of modern hardware systems. Recent explorations into GenAI have shown promise in automating aspects of hardware design verification. However, these current methods often face limitations such as the lack of hardware-specific training data, the need for extensive fine-tuning, and challenges in handling non-traditional architectures. The

taxonomy of GenAI for verification and test is presented in Table XI.

1) *LLM-Based Prompt Generation*: Recent studies have applied LLMs to automate and enhance hardware verification processes. Given that commercial LLMs like GPT are predominantly trained on software code, fine-tuning them with hardware-specific data is essential to adapt their capabilities. Verification and testing based on provided HDL designs is the most popular approach. These works range from testbench generation [56], [382], [383], LLM evaluation [316], [317], bug-detection and self-correction [384], [385], to formal verification [386], [387]. Notably, some of these efforts are open-sourced [56], [316], [317], [382], [383], while others are commercially licensed [384], [385], [386], [387].

On the other hand, for LLM applications based on prompting Non-HDL codes, researchers have delved into subjects including system-level test [388], physical verification [389], design rule check [391], and more. Schwachhofer et al. [388] used LLMs to generate test programs assessing non-functional hardware properties. Xiao et al. [389] leveraged LLMs for verifying neuromorphic processors with non-traditional architectures. Luis et al. [390] automated the generation of physical verification runsets, reducing the reliance on manual coding. Chang et al. [391] combined LLMs and visual language models to streamline design rule checking code generation.

Despite recent progress, LLM-based methods continue to face challenges, including the need for extensive fine-tuning, limitations in handling complex architectures, and reliance on human intervention for code refinement.

2) *GenAI-Assisted Wafer Test*: Conventional wafer test methods are becoming increasingly inadequate for handling high-resolution data. AI offers a transformative solution, enabling automated defect detection, improved accuracy, and reduced inspection times [410], [411]. However, challenges persist due to unlabeled and imbalanced datasets, such as

WM-811K [412]. GenAI methods, such as autoencoders (AEs) and GANs, have emerged as ideal solutions for defect detection.

AE-based techniques have significantly advanced wafer defect detection by enabling unsupervised feature learning and effectively handling unlabeled and imbalanced data. Initially, Wang et al. [392] employed VAEs to generate synthetic data for balancing class distributions in imbalanced wafer defect classification. To reduce reliance on labeled data, Kim et al. [394] utilized self-supervised learning with convolutional autoencoders for dynamic clustering of wafer map patterns, enabling automatic defect pattern identification in unlabeled datasets. To enhance robustness against noise and variations in real-world data, Yu [397] first introduced stacked denoising autoencoders for feature learning, improving recognition performance by strengthening feature representations. Recognizing the importance of spatial relationships in wafer maps, Yu and Liu [398] integrated two-dimensional principal component analysis with convolutional autoencoders, effectively capturing essential spatial features and further enhancing defect detection capabilities. Tulala et al. [395] employed VAEs for unsupervised clustering, grouping similar wafer map defect patterns and streamlining analysis of large unlabeled datasets. To improve model generalization and address previous limitations, Bao et al. [393] combined autoencoder-based data augmentation with convolutional neural networks, enriching training data diversity and showing notable improvements in wafer defect classification. Further contributions include Hwang and Kim [396], who introduced a variational deep clustering approach using VAEs to capture complex defect patterns without labeled data. Hu et al. [399] developed an advanced outlier detection method employing unsupervised autoencoders to enhance defective product screening without extensive labeling. Liao et al. [400] proposed a robust resistive open defect detection technique based on unsupervised deep learning, integrating autoencoders to increase sensitivity to subtle defects in ICs.

On the other hand, GANs have also advanced wafer defect detection by generating synthetic data to address class imbalances in semiconductor datasets. Wang et al. [401] introduced AdaBalGAN, a framework that balances training datasets by synthesizing underrepresented defect patterns, enhancing recognition accuracy. Chen et al. [404] applied GANs to create virtual silicon data for wafer acceptance tests and chip probing, enriching training datasets and improving the modeling of semiconductor manufacturing processes. Mahyar et al. [402] developed DeepWafer, utilizing adversarial networks to produce high-fidelity synthetic wafer defect maps, which augment training data and improve defect classification models. Yu et al. [405] proposed a multi-granularity GAN capable of generating wafer maps at various levels of detail, strengthening the robustness of defect detection models through comprehensive training. Park et al. [403] presented a GAN-based method to create realistic and diverse defect patterns, enhancing the generalization capability and accuracy of detection systems by addressing data insufficiency and imbalance.

These advancements demonstrate the significant impact of advanced GenAI methods for pattern recognition, including

Autoencoder and GAN-based methods in wafer defect detection and classification, offering robust solutions through high-quality synthetic data generation and correction of class imbalances. Some works such as [408] further integrated VAEs and GANs, creating a new model called VAE-GAN to address class imbalance in wafer defect classification. By augmenting data with generated samples, their method improved classification accuracy on imbalanced wafer datasets. However, these approaches still face challenges in terms of computational complexity, stability during training, and the need for careful hyperparameter tuning.

Other GenAI models, including LLMs and transformers, are also applied. Wang et al. [409] leveraged LLMs to interpret complex test data and synthesize cross-domain insights, thereby enhancing defect analysis through the integration of disparate data sources. Lu [406] introduced a transformer-based model utilizing self-attention mechanisms to identify functional semiconductor dies in high-defect-density regions, boosting defect detection and yield prediction. Similarly, Lin et al. [407] presented a transformer-based deep learning model for outlier detection that screens out reliability-weak ICs. Other works also utilize models like graph attention networks [413] and natural language inquiries [414] for wafer detection. Despite their potential, these approaches may struggle with the limited availability of large-scale, high-quality datasets specific to the semiconductor industry, hindering their widespread adoption and generalization.

Collectively, these advancements highlight the substantial impact of GenAI based methods in wafer defect detection. Through unsupervised learning and data augmentation strategies, these approaches offer robust and efficient solutions for handling imbalanced and unlabeled data, improving model generalization, and enhancing the detection capabilities in semiconductor manufacturing processes. However, current methods still face limitations such as the need for extensive fine-tuning, computational complexity, and challenges in adapting to the unique characteristics of semiconductor data. Future research should focus on addressing these limitations and developing more industry-specific, interpretable, and scalable GenAI solutions for wafer defect detection.

D. Analog Design

Generative AI has a wide range of applications in analog circuit design, such as rapidly generating initial circuit design schemes, optimizing circuit parameters, assisting in fault diagnosis, and predicting circuit performance. The taxonomy of GenAI for analog design is shown in Table XII.

1) *Well-Aware Placement*: GANs can be utilized to improve well generation and layout optimization in analog circuit placement, optimizing the circuit layout to enhance performance. Zhu et al. [415] proposed an analytical well-aware analog placer that leverages a GAN to generate wells and guide the placement process. This approach achieved 74% area reduction and over 26% half-perimeter wirelength decrease by facilitating well sharing and explicitly considering wells during placement. Xu et al. [416] proposed a GAN-guided well generation framework for analog/mixed-signal circuit layout. By emulating the expertise of skilled designers, the generated

TABLE XII
TAXONOMY OF GENAI FOR ANALOG DESIGN

Well-aware Placement	GANs	Improve well generation and layout optimization with GANs	[415] [416]
LLM-based Methods	LLMs	Design automation, context learning, and modular reuse	[365] [417] [418]
AMS Circuit Layout	ML	Automated analog routing paradigms	[419] [420] [421] [422]
Automated Design and Verification	ML	Back-end design automation	[423]
	GANs	Generation of well regions	[424]
Simulation Quantity Opt.	FlowSim	Statistical analysis of circuit performance	[425]
Auto-routing	GANs	Routing assistance	[426]

layouts showcased comparable performance with high-quality manual designs.

2) *LLM-Based Methods*: Recent studies have showcased the potential of LLMs in analog circuit design. LLMs have demonstrated significant potential in analog circuit design. Hammoud et al. [427] introduced a layout automation approach using the GLayout framework, which translates analog layouts into an expressive, technology-agnostic, and compact text representation. The method then employed LLMs to understand analog layouts through fine-tuning and ICL using RAG, enabling successful layout of unseen circuits based on new contextual information. Liu et al. [417] proposed an LLM-driven auto-designer, LADAC, that formulates optimized design strategies for analog circuits, and successfully applied it to design amplifiers and ring oscillators. Lai et al. [418] proposed AnalogCoder, a training-free LLM agent that generates Python code for designing analog circuits using a feedback-enhanced flow and customized prompts.

3) *Analog Mixed-Signal Circuit Simulation (AMS) Circuit Layout*: AMS plays a crucial and promising role in analog design. Zhu et al. [420] incorporated key signal paths and utilized prior knowledge from the schematic to optimize AMS circuit layout. In [419], an automated analog routing framework was proposed, which leverages ML to provide guide routing by emulating manual layout techniques. Furthermore, the MAGICAL framework presented in [420] employs customized layout routing and constraint extraction algorithms, enabling a fully automated layout implementation process. This framework has been extended to support additional circuit categories, including SAR-ADC.

In addition, the ALIGN project has made significant contributions to the automation of AMS circuit layout. Kunal et al. [421] presented efforts in analog layout automation under the ALIGN project, which uses a modular approach combining algorithmic and ML-based tools for fast layout generation. The project achieved rapid turnaround by detecting structure and hierarchy in the input netlist and employing a grid-based layout philosophy. Dhar et al. [422] leveraged the ALIGN framework to convert SPICE-level netlists into physical layouts. By utilizing a combination of layout hierarchy, algorithmic techniques, template-driven design, and ML, the system was capable of generating high-quality layouts. This approach was designed to accommodate a diverse range of analog circuits, including low-frequency, wired, radio frequency (RF)/wireless, and power transmission components.

Meanwhile, other optimization studies based on graph structures have also made significant contributions to the design of analog circuits. Gopalakrishnan et al. [428] proposed a modular approach using graph-based optimization to generate well islands and insert well taps during placement. This approach mitigated area and wire length increases and minimized the degradation of circuit performance caused by post-processing steps. Li et al. [429] developed a customized GNN model to predict the impact of placement on analog IC performance and applied this model to analog IC placement. Zhao et al. [430] demonstrated an automated topology synthesis framework for analog ICs. This framework efficiently realized circuit topology synthesis by encoding the circuit topology generation process as tree structure construction.

4) *Automated Design and Verification*: Recent research in AMS circuit design has witnessed increasing integration of AI and ML methodologies to enable automated optimization of design processes. Aliffi et al. [423] presented AMBEATion, an approach to AMS backend design automation that incorporates ML and AI techniques within traditional EDA algorithms. This methodology aimed to optimize the physical design of AMS ICs by improving device placement efficiency, leveraging advanced AI/ML models at various stages of the design flow. By integrating these cutting-edge technologies, AMBEATion significantly enhanced the quality of AMS layouts, reducing design turnaround time and development costs. The approach's modular structure, which incorporates ML models throughout the design process, marked a significant step towards automating complex AMS backend design tasks. Similarly, Rashed et al. [424] proposed a complete automated analog flow that integrates the analog designer's toolbox (ADT) with the machine generated analog IC layout (MAGICAL) tool. This integration combines the gm/ID methodology with precomputed LUTs to fully automate the analog IC design process, encompassing design generation through to post-layout verification. Focusing specifically on voltage regulators, their approach highlights the incorporation of layout-dependent effects during simulation. The integrated workflow significantly reduced time and effort in analog IC development, offering a faster, more efficient solution for designers.

5) *Simulation Quantity Optimization*: To overcome the scalability limitations of traditional Monte Carlo simulations, recent advancements have focused on utilizing alternative methods like normalizing flows for more efficient yield and performance analysis. Park et al. [425] proposed FlowSim, which utilizes density estimation via normalizing flows to

conduct yield and high-sigma analysis, achieving a notable decrease in the number of simulations required. This method offered a more efficient and accurate alternative to conventional Monte Carlo simulations, providing insights into circuit performance under process, voltage, and temperature (PVT) variations with a prediction accuracy of over 95% for all statistical metrics.

6) *Auto-Routing*: Recent advancements in analog IC layout design have introduced interactive tools that enhance efficiency by providing real-time, parasitic-aware routing assistance to engineers.

Chen et al. [426] developed an interactive analog IC layout tool with real-time parasitic-aware automatic routing assistance. This tool employs heuristic and cost functions to offer real-time routing recommendations, effectively minimizing delays and preventing congestion. By integrating Elmore delay estimation and parasitic-aware constraints, the tool facilitates informed decision-making and accelerates the training process, paving the way for more efficient analog layout design.

VI. FUTURE DIRECTIONS

GenAI computing constitutes an interdisciplinary domain that fundamentally integrates AI and CAS. The evolutionary trajectory of this field is structured across four interdependent dimensions: 1) algorithmic foundations, 2) architectural implementations, 3) design automation frameworks, and 4) their co-optimization paradigms.

Algorithmic innovations establish theoretical boundaries that govern error tolerance thresholds and computational complexity limits. Architectural developments enable physical realization through hardware-software co-design methodologies, directly influencing PPA tradeoffs. Automation platforms drive scalable deployment via intelligent design space exploration, critically determining time-to-market efficiency metrics.

Emerging research frontiers encompass, though are not constrained to:

- Complexity reduction in GenAI systems
- Resource-efficient GenAI frameworks
- Hardware-conscious GenAI optimization
- Cross-stack co-design for GenAI implementations
- Ethical and security considerations in GenAI
- Efficient architectural paradigms for GenAI
- Dedicated hardware acceleration for GenAI
- Resource-aware GenAI deployment strategies
- Energy-efficient GenAI computing solutions
- Flexible GenAI implementation methodologies
- Secure GenAI deployment architectures
- Scalable GenAI system design
- GenAI-enhanced VLSI design automation
- Automated co-optimization of GenAI and CAS
- GenAI-CAS 6G integration
- Embodied intelligence through GenAI-CAS synergy
- GenAI-CAS solutions for emerging applications

VII. CONCLUSION

This review investigates the critical challenges and emerging opportunities in the GenAI-CAS domain through a systematic

analysis of recent achievements. Three critical dimensions have been prioritized: algorithm optimization for GenAI deployment, circuits and systems for GenAI implementation, and GenAI for efficient circuits and systems design. Cross-domain technological progresses have been systematically examined in detail. Future research trajectories are also charted to stimulate and orient scholarly efforts toward this promising domain, fostering synergistic innovations across algorithm and implementation. At this juncture of GenAI advancement, this overview is expected to not only offer fundamental GenAI-CAS development but also illuminate convergent pathways for broader technical communities engaged.

ACKNOWLEDGMENT

The authors would like to thank Yu Tian, Yunwei Mao, Changhan Li, and Tao Han for their assistance in composing this article.

REFERENCES

- [1] T. Wu et al., "A brief overview of ChatGPT: The history, status quo and potential future development," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 5, pp. 1122–1136, May 2023.
- [2] K. Kalahasti et al., "Nature's 10," *Nature*, vol. 624, nos. 21–28, p. 495, 2023.
- [3] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
- [4] Y. Bian and X.-Q. Xie, "Generative chemistry: Drug discovery with deep learning generative models," *J. Mol. Model.*, vol. 27, no. 3, pp. 1–18, Mar. 2021.
- [5] A. J. G. Busson et al., "Saturn platform: Foundation model operations and generative AI for financial services," 2023, *arXiv:2312.07721*.
- [6] S. Tarkoma, R. Morabito, and J. Sauvola, "AI-native interconnect framework for integration of large language model technologies in 6G systems," 2023, *arXiv:2311.05842*.
- [7] N. L. Rane, "Contribution of ChatGPT and other generative artificial intelligence (AI) in renewable and sustainable energy," *J. Adv. Artif. Intell.*, vol. 2, no. 1, pp. 1–26, 2024.
- [8] D. Baidoo-Anu and L. O. Ansah, "Education in the era of generative artificial intelligence (AI): Understanding the potential benefits of ChatGPT in promoting teaching and learning," *J. AI*, vol. 7, no. 1, pp. 52–62, Dec. 2023.
- [9] J. Bright, F. E. Enock, S. Esnaashari, J. Francis, Y. Hashem, and D. Morgan, "Generative AI is already widespread in the public sector," 2024, *arXiv:2401.01291*.
- [10] B. Rabowsky, "Applications of generative AI to media," *SMPTE Motion Imag. J.*, vol. 132, no. 8, pp. 53–57, Sep. 2023.
- [11] Z. Zhang, C. Li, W. Sun, X. Liu, X. Min, and G. Zhai, "A perceptual quality assessment exploration for AIGC images," in *Proc. IEEE Int. Conf. Multimedia Expo Workshops (ICMEW)*, Jul. 2023, pp. 440–445.
- [12] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [13] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
- [14] B. Adler et al., "Nemotron-4 340B technical report," 2024, *arXiv:2406.11704*.
- [15] A. Yang et al., "Qwen2.5 technical report," 2024, *arXiv:2412.15115*.
- [16] S. Noy and W. Zhang, "Experimental evidence on the productivity effects of generative artificial intelligence," *Science*, vol. 381, no. 6654, pp. 187–192, Jul. 2023.
- [17] R. Li, L. Jing, C. Han, J. Zhou, and X. Du, "Learning to generate research idea with dynamic control," 2024, *arXiv:2412.14626*.
- [18] B. Owens, "How nature readers are using ChatGPT," *Nature*, vol. 615, no. 7950, p. 20, Mar. 2023.
- [19] J. Priya Inala et al., "Data analysis in the era of generative AI," 2024, *arXiv:2409.18475*.
- [20] R. Jain and A. Jain, "Generative AI in writing research papers: A new type of algorithmic bias and uncertainty in scholarly work," 2023, *arXiv:2312.10057*.

- [21] Univ. Brit. Columbia. (2023). *Principles and Guidelines for Generative Artificial Intelligence (GenAI) in Teaching and Learning*. [Online]. Available: https://it.ubc.ca/sites/it.ubc.ca/files/Guidelines-GenAI-TL_V2.pdf
- [22] L. Drew, "Guidelines for academics aim to lessen ethical pitfalls in generative-AI use," *Nature*, early access, May 22, 2024, doi: [10.1038/d41586-024-01543-1](https://doi.org/10.1038/d41586-024-01543-1).
- [23] Hong Kong Polytech. Univ. (2025). *Guidelines for Students on the Use of Generative Artificial Intelligence*. [Online]. Available: https://www.polyu.edu.hk/ar/docdrive/polyu-students/Student-guide-on-the-use-of-GenAI_revised_250214.pdf
- [24] Y. Bai et al., "GTCO: Graph and tensor co-design for transformer-based image recognition on tensor cores," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 2, pp. 586–599, Feb. 2024.
- [25] G. Huang, Z. Wang, P.-A. Tsai, C. Zhang, Y. Ding, and Y. Xie, "RM-STC: Row-merge dataflow inspired GPU sparse tensor core for energy-efficient sparse acceleration," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2023, pp. 338–352.
- [26] A. Gondimalla, M. Thottethodi, and T. N. Vijaykumar, "Eureka: Efficient tensor cores for one-sided unstructured sparsity in DNN inference," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2023, pp. 324–337.
- [27] X. Wang, S. Miao, P. Qu, and Y. Zhang, "DTrans: A dataflow-transformation FPGA accelerator with nonlinear-operators fusion aiming for the generative model," in *Proc. 34th Int. Conf. Field-Programmable Log. Appl. (FPL)*, Sep. 2024, pp. 339–345.
- [28] C. Du, S.-B. Ko, and H. Zhang, "Energy efficient FPGA-based binary transformer accelerator for edge devices," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.
- [29] H. Peng et al., "A length adaptive algorithm-hardware co-design of transformer on FPGA through sparse attention and dynamic pipelining," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Aug. 2022, pp. 1135–1140.
- [30] Y. Wang et al., "A 28 nm 77.35TOPS/W similar vectors traceable transformer processor with principal-component-prior speculating and dynamic bit-wise stationary computing," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2023, pp. 1–2.
- [31] Y. Wang et al., "An energy-efficient transformer processor exploiting dynamic weak relevances in global attention," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 227–242, Jan. 2023.
- [32] S. Sridharan, J. R. Stevens, K. Roy, and A. Raghunathan, "X-former: In-memory acceleration of transformers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 8, pp. 1223–1233, Aug. 2023.
- [33] T. Norrie et al., "The design process for Google's training chips: TPUv2 and TPUv3," *IEEE Micro*, vol. 41, no. 2, pp. 56–63, Mar. 2021.
- [34] P. Verma, "DSO.ai—A distributed system to optimize physical design flows," in *Proc. Int. Symp. Phys. Design*, Mar. 2024, pp. 115–116.
- [35] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Proces. Syst. (NeurIPS)*, vol. 27, 2014, pp. 1–9.
- [36] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [37] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, vol. 33, Vancouver, BC, Canada: Curran Associates, 2020, pp. 6840–6851.
- [38] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.
- [39] S. Yeo, Y. Jang, J.-Y. Sohn, D. Han, and J. Yoo, "Can we find strong lottery tickets in generative models?," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jun. 2023, vol. 37, no. 3, pp. 3267–3275.
- [40] J. Gu, S. Zhai, Y. Zhang, L. Liu, and J. Susskind, "BOOT: Data-free distillation of denoising diffusion models with bootstrapping," in *Proc. Mach. Learn. Res. (ICML)*, Jan. 2023, pp. 1–27.
- [41] J. Jiang, J. Du, D. Huang, Z. Chen, Y. Lu, and X. Liao, "Full-stack optimizing transformer inference on ARM many-core CPU," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 7, pp. 2221–2235, Jul. 2023.
- [42] Y. Wei et al., "APTMoE: Affinity-aware pipeline tuning for MoE models on bandwidth-constrained GPU nodes," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC24)*, Nov. 2024, pp. 1–14.
- [43] H. Wang et al., "Sofa: A compute-memory optimized sparsity accelerator via cross-stage coordinated tiling," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Nov. 2024, pp. 1247–1263.
- [44] W. Kong et al., "Cambricon-D: Full-network differential acceleration for diffusion models," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 903–914.
- [45] NVIDIA.(2020). *NVIDIA Ampere GA102 GPU Architecture Whitepaper*. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [46] Qualcomm.(2024). *Unlocking On-device Generative AI With an NPU and Heterogeneous Computing*. [Online]. Available: <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/Unlocking-on-device-generative-AI-with-an-NPU-and-heterogeneous-computing.pdf>
- [47] Z. Zhang and J. Li, "A review of artificial intelligence in embedded systems," *Micromachines*, vol. 14, no. 5, p. 897, Apr. 2023.
- [48] A. Alhussain and M. Lin, "Hardware-efficient deconvolution-based GAN for edge computing," in *Proc. 56th Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2022, pp. 172–176.
- [49] J. Gao et al., "Self-evolving GPT: A lifelong autonomous experiential learner," 2024, *arXiv:2407.08937*.
- [50] J. Lu et al., "AgentLens: Visual analysis for agent behaviors in LLM-based autonomous systems," *IEEE Trans. Vis. Comput. Graph.*, early access, May 3, 2024, doi: [10.1109/TVCG.2024.3394053](https://doi.org/10.1109/TVCG.2024.3394053).
- [51] M. Liu et al., "ChipNeMo: Domain-adapted LLMs for chip design," 2023, *arXiv:2311.00176*.
- [52] H. Pearce, B. Tan, and R. Karri, "DAVE: Deriving automatically Verilog from English," in *Proc. ACM/IEEE 2nd Workshop Mach. Learn. CAD (MLCAD)*, Nov. 2020, pp. 27–32.
- [53] S. Fernando, R. Kunzelmann, D. S. Lopera, J. A. Halabi, and W. Ecker, "Boosting productivity of hardware documentation using large language models," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, p. 1.
- [54] P.-Y. Lee and T.-C. Chen, "AI-driven innovations in IC designs: From planning to implementation," in *Proc. Int. VLSI Symp. Technol., Syst. Appl. (VLSI TSA)*, Apr. 2024, pp. 1–2.
- [55] L. Feng, W. Liu, C. Guo, K. Tang, C. Zhuo, and Z. Wang, "GANDSE: Generative adversarial network-based design space exploration for neural network accelerator design," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 3, pp. 1–20, May 2023.
- [56] R. Qiu, G. Li Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "CorrectBench: Automatic testbench generation with functional self-correction using LLMs for HDL design," 2024, *arXiv:2411.08510*.
- [57] Y. Mao, Y. You, X. Tan, Y. Huang, X. You, and C. Zhang, "FLAG: Formula-LLM-based auto-generator for baseband hardware," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.
- [58] S. Thakur et al., "Benchmarking large language models for automated verilog RTL code generation," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Apr. 2023, pp. 1–6.
- [59] K. Chang et al., "ChipGPT: How far are we from natural language hardware design," 2023, *arXiv:2305.14019*.
- [60] M. Krichen, "Generative adversarial networks," in *Proc. Int. Conf. Comput. Commun. Network. Techn. (ICCCNT)*, Sep. 2023, pp. 1–7.
- [61] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [62] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.
- [63] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [64] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [65] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.
- [66] Y. Wang, M. Pang, S. Chen, and H. Rao, "Consistency-GAN: Training GANs with consistency model," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 14, pp. 15743–15751.
- [67] L. Wang, L. Zeng, and L. Jian, "AEC-GAN: Adversarial error correction GANs for auto-regressive long time-series generation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jun. 2023, vol. 37, no. 8, pp. 10140–10148.
- [68] S. K. Pandey, "Neuroevolution of a multi-generator GAN (student abstract)," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 21, pp. 23608–23609.
- [69] H. Liu et al., "AdaptiveMix: Improving GAN training via feature space shrinkage," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 16219–16229.
- [70] C. Ma, "Uncertainty-aware GAN for single image super resolution," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 5, pp. 4071–4079.
- [71] Y. Zeng, J. Fu, H. Chao, and B. Guo, "Aggregated contextual transformations for high-resolution image inpainting," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 7, pp. 3266–3280, Jul. 2022.

- [72] X. Wang, K. Yen, Y. Hu, and H.-W. Shen, "SmartGD: A GAN-based graph drawing framework for diverse aesthetic goals," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, no. 8, pp. 5666–5678, Aug. 2024.
- [73] H. Zhou, R. Ma, L.-X. Zhang, L. Gao, A. Mahdavi-Amiri, and H. Zhang, "SAC-GAN: Structure-aware image composition," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, pp. 3151–3165, 2022.
- [74] Y. Li and G. Baciú, "SG-GAN: Adversarial self-attention GCN for point cloud topological parts generation," *IEEE Trans. Vis. Comput. Graph.*, vol. 28, no. 10, pp. 3499–3512, Oct. 2022.
- [75] W. Jiang, W. X. Zhao, J. Wang, and J. Jiang, "Continuous trajectory generation based on two-stage GAN," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jun. 2023, vol. 37, no. 4, pp. 4374–4382.
- [76] S. Wang, W. Zeng, X. Chen, Y. Ye, Y. Qiao, and C.-W. Fu, "ActFloor-GAN: Activity-guided adversarial networks for human-centric floorplan design," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 3, pp. 1610–1624, Mar. 2023.
- [77] M. Bazzaz and S. Cooper, "Controllable game level generation: Assessing the effect of negative examples in GAN models," in *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain. (AIIDE)*, Nov. 2024, vol. 20, no. 1, pp. 149–155.
- [78] K. Shubham, P. Sastry, and P. Ap, "Fusing conditional submodular GAN and programmatic weak supervision," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 13, pp. 15020–15028.
- [79] A. Yin, H. Li, K. Shen, S. Tang, and Y. Zhuang, "T2S-GPT: Dynamic vector quantization for autoregressive sign language production from text," 2024, *arXiv:2406.07119*.
- [80] J. Liu, J. Chen, Z. Liu, and H. Zhou, "Enabling feedback-free MIMO transmission for FD-RAN: A data-driven approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 2437–2454, Mar. 2025.
- [81] J. Song, Y. Yang, W. Peng, W. Zhou, F. Wang, and W. Yao, "MorphVAE: Advancing morphological design of voxel-based soft robots with variational autoencoders," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 9, pp. 10368–10376.
- [82] M. Lu, Z. Duan, F. Zhu, and Z. Ma, "Deep hierarchical video compression," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 8, pp. 8859–8867.
- [83] S. Moon, S. Cho, and D. Kim, "Feature unlearning for pre-trained GANs and VAEs," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 19, pp. 21420–21428.
- [84] L. Huang, Q. Qiu, and G. Sapiro, "PQ-VAE: Learning hierarchical discrete representations with progressive quantization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 7550–7558.
- [85] L. Chen et al., "OD-VAE: An omni-dimensional video compressor for improving latent video diffusion model," 2024, *arXiv:2409.01199*.
- [86] Z. Wang et al., "Revisiting VAE for unsupervised time series anomaly detection: A frequency perspective," in *Proc. ACM Web Conf.*, May 2024, pp. 3096–3105.
- [87] T. Yuan et al., "Accelerating the training of LLM using efficient activation rematerialization and optimal hybrid parallelism," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Jul. 2024, pp. 545–561.
- [88] W. Du et al., "Stacking your transformers: A closer look at model growth for efficient LLM pre-training," 2024, *arXiv:2405.15319*.
- [89] G. Fang et al., "MaskLLM: Learnable semi-structured sparsity for large language models," 2024, *arXiv:2409.17481*.
- [90] B. Butler, S. Yu, A. Mazaheri, and A. Jannesari, "PipeInfer: Accelerating LLM inference using asynchronous pipelined speculation," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC24)*, Nov. 2024, pp. 1–19.
- [91] M. Xu, D. Cai, Y. Wu, X. Li, and S. Wang, "FwdLLM: Efficient federated finetuning of large language models with perturbed inferences," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Jun. 2024, pp. 579–596.
- [92] Z. Tan, T. Chen, Z. Zhang, and H. Liu, "Sparsity-guided holistic explanation for LLMs with interpretable inference-time intervention," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jan. 2023, vol. 38, no. 19, pp. 21619–21627.
- [93] H. Oh et al., "ExeGPT: Constraint-aware resource scheduling for LLM inference," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2024, pp. 369–384.
- [94] Z. Bi et al., "OceanGPT: A large language model for ocean science tasks," 2023, *arXiv:2310.02031*.
- [95] C. M. Pham, A. Hoyle, S. Sun, P. Resnik, and M. Iyyer, "TopicGPT: A prompt-based topic modeling framework," 2023, *arXiv:2311.01449*.
- [96] Z. Yang et al., "An empirical study of GPT-3 for few-shot knowledge-based VQA," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 3, pp. 3081–3089.
- [97] X. Bi et al., "DeepSeek LLM: Scaling open-source language models with longtermism," 2024, *arXiv:2401.02954*.
- [98] A. Liu et al., "DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024, *arXiv:2405.04434*.
- [99] A. Liu et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.
- [100] H. Lu et al., "DeepSeek-VL: Towards real-world vision-language understanding," 2024, *arXiv:2403.05525*.
- [101] Z. Wu et al., "DeepSeek-VL2: Mixture-of-experts vision-language models for advanced multimodal understanding," 2024, *arXiv:2412.10302*.
- [102] D. Guo et al., "DeepSeek-coder: When the large language model meets programming {-} the rise of code intelligence," 2024, *arXiv:2401.14196*.
- [103] Q. Zhu et al., "DeepSeek-Coder-v2: Breaking the barrier of closed-source models in code intelligence," 2024, *arXiv:2406.11931*.
- [104] Z. Li et al., "Diffusion model for data-driven black-box optimization," 2024, *arXiv:2403.13219*.
- [105] Q. Guo, C. Ma, Y. Jiang, Z. Yuan, Y. Yu, and P. Luo, "EGC: Image generation and classification via a diffusion energy-based model," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 22895–22905.
- [106] H. Ma et al., "Plug-in diffusion model for sequential recommendation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jan. 2024, vol. 38, no. 8, pp. 8886–8894.
- [107] W. Weng et al., "ARTV: Auto-regressive text-to-video generation with diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 7395–7405.
- [108] B. Wang, G. Wu, T. Pang, Y. Zhang, and Y. Yin, "DiffAIL: Diffusion adversarial imitation learning," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 14, pp. 15447–15455.
- [109] J. Zhao, C. Xu, and B. Jiang, "IPED: An implicit perspective for relational triple extraction based on diffusion model," 2024, *arXiv:2403.00808*.
- [110] Y. Cao, X. Meng, P. Y. Mok, T.-Y. Lee, X. Liu, and P. Li, "AnimeDiffusion: Anime diffusion colorization," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, no. 10, pp. 6956–6969, Oct. 2024.
- [111] S. Zhang, L. Wu, C. Gong, and X. Liu, "LanguageFlow: Advancing diffusion language generation with probabilistic flows," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics Hum. Lang. Technol.*, 2024, pp. 3893–3905.
- [112] G. Xiao et al., "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 38087–38099.
- [113] H. Xi, Y. Chen, K. Zhao, K. Jun Teh, J. Chen, and J. Zhu, "Jetfire: Efficient and accurate transformer pretraining with INT8 data flow and per-block quantization," 2024, *arXiv:2403.12422*.
- [114] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Proces. Syst. (NeurIPS)*, vol. 36, Jan. 2023, pp. 10088–10115.
- [115] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa, "QuIP: 2-bit quantization of large language models with guarantees," in *Proc. Adv. Neural Inf. Proces. Syst. (NeurIPS)*, vol. 36, Jul. 2023, pp. 4396–4429.
- [116] X. Zheng et al., "BiDM: Pushing the limit of quantization for diffusion models," 2024, *arXiv:2412.05926*.
- [117] J. Lin et al., "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst. (MLSys)*, Jun. 2024, pp. 87–100.
- [118] S. Kim et al., "SqueezeLLM: Dense-and-sparse quantization," 2023, *arXiv:2306.07629*.
- [119] V. Egiastian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, "Extreme compression of large language models via additive quantization," 2024, *arXiv:2401.06118*.
- [120] W. Wang, W. Sun, and Y. Liu, "Improving transformer inference through optimized nonlinear operations with quantization-approximation-based strategy," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, Oct. 30, 2024, doi: [10.1109/tcad.2024.3488572](https://doi.org/10.1109/tcad.2024.3488572).
- [121] Y. Chen et al., "MixQ: Taming dynamic outliers in mixed-precision quantization by online prediction," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC24)*, Nov. 2024, pp. 1–15.
- [122] J. Guo et al., "Compressing large language models by joint sparsification and quantization," in *Proc. Mach. Learn. Res. (ICML)*, Apr. 2024, pp. 1–13.
- [123] Y. Ma et al., "Outlier-aware slicing for post-training quantization in vision transformer," in *Proc. Mach. Learn. Res. (ICML)*, May 2024, pp. 1–15.

- [124] H. Qin et al., "Accurate LoRA-finetuning quantization of LLMs via information retention," 2024, *arXiv:2402.05445*.
- [125] Z. Zhang et al., "Quantized side tuning: Fast and memory-efficient tuning of quantized large language models," 2024, *arXiv:2401.07159*.
- [126] Y. Xia, F. Fu, W. Zhang, J. Jiang, and C. Bin, "Efficient multi-task LLM quantization and serving for multiple LoRA adapters," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, Feb. 2024, pp. 1–29.
- [127] X. Ma, G. Fang, and X. Wang, "LLM-pruner: On the structural pruning of large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2023, pp. 21702–21720.
- [128] Y. An, X. Zhao, C. Yu, M. Tang, and J. Wang, "Fluctuation-based adaptive structured pruning for large language models," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Jan. 2023, vol. 38, no. 10, pp. 10865–10873.
- [129] Z. Yu et al., "EDGE-LLM: Enabling efficient large language model adaptation on edge devices via layerwise unified compression and adaptive layer tuning and voting," 2024, *arXiv:2406.15758*.
- [130] L. Yin et al., "Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity," 2023, *arXiv:2310.05175*.
- [131] S. Gao et al., "DISP-LLM: Dimension-independent structural pruning for large language models," 2024, *arXiv:2410.11988*.
- [132] P. Dong et al., "HeatViT: Hardware-efficient adaptive token pruning for vision transformers," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2023, pp. 442–455.
- [133] J. Chung, S. Hyun, S.-H. Shim, and J.-P. Heo, "Diversity-aware channel pruning for StyleGAN compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 7902–7911.
- [134] B. Deiseroth et al., "Divergent token metrics: Measuring degradation to prune away LLM components-and optimize quantization," 2023, *arXiv:2311.01544*.
- [135] A. Ganjanesh, S. Gao, H. Alipanah, and H. Huang, "Compressing image-to-image translation GANs using local density structures on their learned manifold," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 11, pp. 12118–12126.
- [136] D. Saxena, J. Cao, J. Xu, and T. Kulshrestha, "RG-GAN: Dynamic regenerative pruning for data-efficient generative adversarial networks," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 5, pp. 4704–4712.
- [137] W. Shin, B. H. Lee, J. S. Kim, H. J. Park, and S. W. Han, "MetricGAN-OKD: Multi-metric optimization of MetricGAN via online knowledge distillation for speech enhancement," in *Proc. Mach. Learn. Res. (ICML)*, Nov. 2023, pp. 31521–31538.
- [138] M. Gholami, M. Akbari, C. Hu, V. Masrani, Z. J. Wang, and Y. Zhang, "GOLD: Generalized knowledge distillation via out-of-distribution-guided language data generation," 2024, *arXiv:2403.19754*.
- [139] R. Aalishah, M. Navardi, and T. Mohsenin, "MambaLiteSR: Image super-resolution with low-rank mamba using knowledge distillation," 2025, *arXiv:2502.14090*.
- [140] A. Maekawa, S. Kosugi, K. Funakoshi, and M. Okumura, "DiLM: Distilling dataset into language model for text-level dataset distillation," 2024, *arXiv:2404.00264*.
- [141] Z. Yang et al., "Self-distillation bridges distribution gap in language model fine-tuning," 2024, *arXiv:2402.13669*.
- [142] T. Hu, M. Lin, L. You, F. Chao, and R. Ji, "Discriminator-cooperated feature map distillation for GAN compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 20351–20360.
- [143] M. Kang et al., "Information-theoretic GAN compression with variational energy-based model," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 18241–18255.
- [144] W. Wang et al., "CANDLE: Iterative conceptualization and instantiation distillation from large language models for commonsense reasoning," 2024, *arXiv:2401.07286*.
- [145] Z. Liao, H. Yu, J. Li, J. Wang, and W. Zhang, "D2LLM: Decomposed and distilled large language models for semantic search," 2024, *arXiv:2406.17262*.
- [146] J. Ko, S. Kim, T. Chen, and S.-Y. Yun, "DistiLLM: Towards streamlined distillation for large language models," 2024, *arXiv:2402.03898*.
- [147] W. Choi, W. Kyung Kim, M. Yoo, and H. Woo, "Embodied CoT distillation from LLM to off-the-shelf agents," 2024, *arXiv:2412.11499*.
- [148] Y. Zhang et al., "Enhancing multilingual capabilities of large language models through self-distillation from resource-rich languages," 2024, *arXiv:2402.12204*.
- [149] K. Feng, C. Li, X. Zhang, J. Zhou, Y. Yuan, and G. Wang, "Keypoint-based progressive chain-of-thought distillation for LLMs," 2024, *arXiv:2405.16064*.
- [150] J. Zhao et al., "Multistage collaborative knowledge distillation from a large language model for semi-supervised sequence generation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 14201–14214.
- [151] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Mach. Learn. Res. (ICML)*, Jan. 2022, pp. 19274–19286.
- [152] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," 2023, *arXiv:2302.01318*.
- [153] Z. Sun, A. T. Suresh, J. H. Ro, A. Beirami, H. Jain, and F. Yu, "SpecTr: Fast speculative decoding via optimal transport," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2023, pp. 30222–30242.
- [154] Z. Hu and H. Huang, "Accelerated speculative sampling based on tree Monte Carlo," in *Proc. Mach. Learn. Res. (ICML)*, 2024, pp. 1–35.
- [155] X. Miao et al., "SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2023, *arXiv:2305.09781*.
- [156] Z. Chen et al., "Sequoia: Scalable, robust, and hardware-aware speculative decoding," 2024, *arXiv:2402.12374*.
- [157] R. Svirschevski et al., "SpecExec: Massively parallel speculative decoding for interactive LLM inference on consumer devices," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 16342–16368.
- [158] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," 2024, *arXiv:2401.10774*.
- [159] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," 2024, *arXiv:2401.15077*.
- [160] Y. Zhou et al., "DistillSpec: Improving speculative decoding via knowledge distillation," 2023, *arXiv:2310.08461*.
- [161] S. Yang, G. Lee, J. Cho, D. Papailiopoulos, and K. Lee, "Predictive pipelined decoding: A compute-latency trade-off for exact LLM decoding," 2023, *arXiv:2307.05908*.
- [162] M. Navardi et al., "GenAI at the edge: Comprehensive survey on empowering edge devices," 2025, *arXiv:2502.15816*.
- [163] Y. Ge, Z. Ji, Y. Huang, Z. Zhang, X. You, and C. Zhang, "Automatic hybrid-precision quantization for MIMO detectors," *IEEE Trans. Signal Process.*, vol. 71, pp. 1039–1052, 2023.
- [164] Y. Zeng et al., "DL-Aqua: Deep-learning-based automatic quantization for MMSE MIMO detection," *IEEE Wireless Commun. Lett.*, vol. 13, no. 12, pp. 3355–3359, Dec. 2024.
- [165] A. Xu et al., "GausiQ: Generalized automatic hybrid-precision quantization for MIMO detection," *IEEE Wireless Commun. Lett.*, vol. 14, no. 3, pp. 596–600, Mar. 2025.
- [166] Y. Sun, H. Ji, Y. Zeng, Y. Huang, and C. Zhang, "BANQ: BayesOpt-based automatic non-uniform quantization for SCL polar decoding," *IEEE Wireless Commun. Lett.*, vol. 14, no. 3, pp. 576–580, Mar. 2025.
- [167] Y. Sun et al., "GenAlg-SCL: Genetic algorithm based SCL polar decoding towards list size optimization," *IEEE Trans. Green Commun. Netw.*, to be published.
- [168] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, "Speculative decoding: Exploiting speculative execution for accelerating Seq2seq generation," 2022, *arXiv:2203.16487*.
- [169] Y. Chen, X. Qi, J. Wang, and L. Zhang, "DisCo-CLIP: A distributed contrastive loss for memory efficient CLIP training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 22648–22657.
- [170] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2022, pp. 16344–16359.
- [171] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2023, pp. 1–14.
- [172] W. An et al., "Fire-flyer AI-HPC: A cost-effective software-hardware co-design for deep learning," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2024, pp. 1–23.
- [173] M. Isaev, N. McDonald, L. Dennison, and R. Vuduc, "Calculon: A methodology and tool for high-level co-design of systems and large language models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2023, pp. 1–14.
- [174] Z. Zhang, Y. Chen, B. He, and Z. Zhang, "NIOT: A novel inference optimization of transformers on modern CPUs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 6, pp. 1982–1995, Jun. 2023.
- [175] X. Wang et al., "LightSeq2: Accelerated training for transformer-based models on GPUs," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC22)*, Nov. 2022, pp. 1–14.

- [176] H. Mark Chen et al., "Hardware-aware parallel prompt decoding for memory-efficient acceleration of LLM inference," 2024, *arXiv:2405.18628*.
- [177] K. Lv, H. Guo, Q. Guo, and X. Qiu, "DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting," 2025, *arXiv:2503.00784*.
- [178] Y. Zhao, D. Wu, and J. Wang, "ALISA: Accelerating large language model inference via sparsity-aware KV caching," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 1005–1017.
- [179] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, "S³: Increasing GPU utilization during generative inference for higher throughput," in *Proc. Adv. Neural Inf. Proces. Syst. (NeurIPS)*, vol. 36, 2023, pp. 18015–18027.
- [180] C. Yu, T. Chen, Z. Gan, and J. Fan, "Boost vision transformer with GPU-friendly sparsity and quantization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 22658–22668.
- [181] W. Wang, S. Zhou, W. Sun, P. Sun, and Y. Liu, "SOLE: Hardware–software co-design of softmax and LayerNorm for efficient transformer inference," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [182] H. Qiu et al., "Power-aware deep learning model serving with μ -serve," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Aug. 2024, pp. 75–93.
- [183] M. R. Shaharear, A. N. Mazumder, and T. Mohsenin, "ViT-reg: Regression-focused hardware-aware fine-tuning for ViT on TinyML platforms," *IEEE Design Test*, early access, Dec. 23, 2024, doi: [10.1109/mdat.2024.3521320](https://doi.org/10.1109/mdat.2024.3521320).
- [184] R. Hwang et al., "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 1018–1031.
- [185] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 118–132.
- [186] H. Xia et al., "Quant-LLM: Accelerating the serving of large language models via FP6-centric algorithm-system co-design on modern GPUs," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Jun. 2024, pp. 699–713.
- [187] J. Cheng, C. Zhang, Z. Yu, C.-S. Bouganis, G. A. Constantinides, and Y. Zhao, "A dataflow compiler for efficient LLM inference using custom microscaling formats," 2023, *arXiv:2307.15517*.
- [188] J. Fang, Y. Yu, C. Zhao, and J. Zhou, "TurboTransformers: An efficient GPU serving system for transformer models," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, Feb. 2021, pp. 389–402.
- [189] Y. N. Lee, Y. Yu, and Y. Park, "CUrator: An efficient LLM execution engine with optimized integration of CUDA libraries," in *Proc. 23rd ACM/IEEE Int. Symp. Code Gener. Optim.*, Mar. 2025, pp. 209–224.
- [190] R. Sanovar, S. Bharadwaj, R. S. Amant, V. Rühle, and S. Rajmohan, "Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers," 2024, *arXiv:2405.10480*.
- [191] H. Shao, J. Lu, M. Wang, and Z. Wang, "An efficient training accelerator for transformers with hardware-algorithm co-optimization," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 11, pp. 1788–1801, Nov. 2023.
- [192] P. Dong et al., "EQ-ViT: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on versal ACAP architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 3949–3960, Nov. 2024.
- [193] G. Wang, S. Cai, W. Li, D. Lyu, and G. He, "BSViT: A bit-serial vision transformer accelerator exploiting dynamic patch and weight bit-group quantization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 9, pp. 4064–4077, Sep. 2024.
- [194] X. Wang et al., "Vision transformer acceleration via a versatile attention optimization framework," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, Dec. 9, 2025, doi: [10.1109/tcad.2024.3513265](https://doi.org/10.1109/tcad.2024.3513265).
- [195] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 1071–1085.
- [196] S. Q. Zhang, T. Tambe, N. Cuevas, G.-Y. Wei, and D. Brooks, "CAMEL: Co-designing AI models and eDRAMs for efficient on-device learning," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Mar. 2024, pp. 861–875.
- [197] M. Kang, H. Shin, and L.-S. Kim, "A framework for accelerating transformer-based language model on ReRAM-based architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 3026–3039, Sep. 2022.
- [198] H. Li, Z. Li, Z. Bai, and T. Mitra, "ASADI: Accelerating sparse attention using diagonal-based in-situ computing," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 774–787.
- [199] Z. Zhou, J. Liu, Z. Gu, and G. Sun, "Energon: Toward efficient acceleration of transformers using dynamic sparse attention," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 1, pp. 136–149, Jan. 2023.
- [200] J. Zhao, P. Zeng, G. Shen, Q. Chen, and M. Guo, "Hardware–software co-design enabling static and dynamic sparse attention mechanisms," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 9, pp. 2783–2796, Sep. 2024.
- [201] T. J. Ham et al., "ELSA: Hardware–software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2021, pp. 692–705.
- [202] T. Yang et al., "DTATrans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient transformer architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 509–520, Feb. 2023.
- [203] T. Yang et al., "DTQAtten: Leveraging dynamic token-based quantization for efficient attention architecture," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 700–705.
- [204] X. Shen et al., "HotaQ: Hardware oriented token adaptive quantization for large language models," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, Oct. 29, 2024, doi: [10.1109/tcad.2024.3487781](https://doi.org/10.1109/tcad.2024.3487781).
- [205] W. Li, A. Hu, N. Xu, and G. He, "Quantization and hardware architecture co-design for matrix-vector multiplications of large language models," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 6, pp. 2858–2871, Jun. 2024.
- [206] Y. Qin et al., "MECLA: Memory-compute-efficient LLM accelerator with scaling sub-matrix partition," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 1032–1047.
- [207] P. Plagwitz, F. Hannig, and J. Teich, "TRAC: Compilation-based design of transformer accelerators for FPGAs," in *Proc. 32nd Int. Conf. Field-Program. Log. Appl. (FPL)*, Aug. 2022, pp. 17–23.
- [208] K. Qian, H. Lu, Y. Liu, Z. Zhang, and K. Wang, "End-to-end compilation is all FPGAs need: A unified overlay-based FPGA compiler for deep learning," in *Proc. 30th Asia South Pacific Design Autom. Conf.*, Jan. 2025, pp. 258–264.
- [209] Y. Bai et al., "CSTrans-OPU: An FPGA-based overlay processor with full compilation for transformer networks via sparsity exploration," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [210] J. Ansel et al., "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2024, pp. 929–947.
- [211] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Design Implement.*, 2016, pp. 265–283.
- [212] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2021, pp. 2–14.
- [213] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 3505–3506.
- [214] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2018, pp. 578–594.
- [215] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop Mach. Learn. Program. Lang.*, Jun. 2019, pp. 10–19.
- [216] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *ACM SIGPLAN Notices*, vol. 48, no. 6, pp. 519–530, 2013.
- [217] V. Titopoulos, K. Alexandridis, C. Peltekis, C. Nicopoulos, and G. Dimitrakopoulos, "IndexMAC: A custom RISC-V vector instruction to accelerate structured-sparse matrix multiplications," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.

- [218] G. Jeong et al., "VEGETA: Vertically-integrated extensions for sparse/dense gEMM tile acceleration on CPUs," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2023, pp. 259–272.
- [219] H. Xia et al., "Flash-LLM: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity," *Proc. VLDB Endowment*, vol. 17, no. 2, pp. 211–224, Oct. 2023.
- [220] J. Liu et al., "TSTC: Two-level sparsity tensor core enabling both algorithm flexibility and hardware efficiency," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [221] P. S. Labini, M. Bernaschi, W. Nutt, F. Silvestri, and F. Vella, "Blocking sparse matrices to leverage dense-specific multiplication," in *Proc. IEEE/ACM Workshop Irregular Appl., Archit. Algorithms (IA3)*, Nov. 2022, pp. 19–24.
- [222] H. Wang, W. Yang, R. Hu, R. Ouyang, K. Li, and K. Li, "A novel parallel algorithm for sparse tensor matrix chain multiplication via TCU-acceleration," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 8, pp. 2419–2432, Aug. 2023.
- [223] C. Guo et al., "Accelerating sparse DNNs based on tiled GEMM," *IEEE Trans. Comput.*, vol. 73, no. 5, pp. 1275–1289, May 2024.
- [224] K. Ho, H. Zhao, A. Jog, and S. Mohanty, "Improving GPU throughput through parallel execution using tensor cores and CUDA cores," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2022, pp. 223–228.
- [225] Y. Sun et al., "Accelerating sparse deep neural network inference using GPU tensor cores," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2022, pp. 1–7.
- [226] J. Du, J. Jiang, J. Zheng, H. Zhang, D. Huang, and Y. Lu, "Improving computation and memory efficiency for real-world transformer inference on GPUs," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, pp. 1–22, Dec. 2023.
- [227] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. ACM SIGOPS 30th Symp. Operating Syst. Princ.*, Nov. 2024, pp. 590–606.
- [228] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.
- [229] C. Zhang et al., "JENGA: Effective memory management for serving LLM with heterogeneity," 2025, *arXiv:2503.18292*.
- [230] M. Dun, X. Zhang, H. Cao, Y. Zhang, J. Huang, and X. Ye, "Adaptive sparse deep neural network inference on resource-constrained cost-efficient GPUs," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2023, pp. 1–7.
- [231] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. Int. Conf. Mach. Learn.*, Aug. 2023, pp. 31094–31116.
- [232] P. Kluska, A. Castelló, F. Scheidegger, A. C. I. Malossi, and E. S. Quintana-Ortí, "QAttn: Efficient GPU kernels for mixed-precision vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 3648–3657.
- [233] Z. Du, Y. Li, C. Chen, and Z. Wang, "AttenTPU: Tensor processor for attention mechanism with fine-grained padding," in *Proc. IEEE Int. Conf. Integr. Circuits, Technol. Appl. (ICTA)*, Oct. 2023, pp. 101–102.
- [234] M. E. Elbity, P. S. Chandarana, B. Reidy, J. K. Eshraghian, and R. Zand, "APTPU: Approximate computing based tensor processing unit," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 12, pp. 5135–5146, Dec. 2022.
- [235] X. He et al., "Squaring the circle: Executing sparse matrix computations on FlexTPU—A TPU-like processor," in *Proc. Int. Conf. Parallel Architectures Compilation Techn.*, Oct. 2022, pp. 148–159.
- [236] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, "Dual-side sparse tensor core," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2021, pp. 1083–1095.
- [237] R. Y. Aminabadi et al., "DeepSpeed[-] inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC22)*, Nov. 2022, pp. 1–15.
- [238] X. He et al., "Sparse-TPU: Adapting systolic arrays for sparse matrices," in *Proc. 34th ACM Int. Conf. Supercomputing*, Jun. 2020, pp. 1–12.
- [239] N. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Architect.*, 2017, pp. 1–12.
- [240] S. Hong et al., "DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2022, pp. 616–630.
- [241] W. Ye, X. Zhou, J. Zhou, C. Chen, and K. Li, "Accelerating attention mechanism on FPGAs based on efficient reconfigurable systolic array," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 6, pp. 1–22, Nov. 2023.
- [242] Z. Luo, L. Lu, Y. Jin, L. Jia, and Y. Liang, "Calabash: Accelerating attention using a systolic array chain on FPGAs," in *Proc. Int. Conf. Field-Program. Log. Appl. (FPL)*, Sep. 2023, pp. 242–247.
- [243] Z. Li, Y. Lai, and H. Zhang, "Energy efficient FPGA-based accelerator for dynamic sparse transformer," in *Proc. 13th Int. Conf. Commun., Circuits Syst. (ICCCAS)*, May 2024, pp. 7–12.
- [244] B. Li et al., "FTRANS: Energy-efficient acceleration of transformers using FPGA," in *Proc. ACM/IEEE Int. Symp. Low Power Electron. Design*, Aug. 2020, pp. 175–180.
- [245] P. Qi, Y. Song, H. Peng, S. Huang, Q. Zhuge, and E. H.-M. Sha, "Accommodating transformer onto FPGA: Coupling the balanced model compression and FPGA-implementation optimization," in *Proc. Great Lakes Symp. VLSI*, Jun. 2021, pp. 163–168.
- [246] T. Zhao et al., "Towards a reconfigurable systolic array with multi-level packing for transformers," in *Proc. Archit. Syst. Support Transformer Models (ASSYST)*, Sep. 2023, pp. 1–5.
- [247] S. Zeng et al., "FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Apr. 2024, pp. 223–234.
- [248] Y. Qin, W. Lou, C. Wang, L. Gong, and X. Zhou, "Enhancing long sequence input processing in FPGA-based transformer accelerators through attention fusion," in *Proc. Great Lakes Symp. VLSI*, Jun. 2024, pp. 599–603.
- [249] Y. Han and Q. Liu, "HPTA: A high performance transformer accelerator based on FPGA," in *Proc. 33rd Int. Conf. Field-Programmable Log. Appl. (FPL)*, Sep. 2023, pp. 27–33.
- [250] H. Wang, Y. Bai, J. Yu, and K. Wang, "TransFRU: Efficient deployment of transformers on FPGA with full resource utilization," in *Proc. 29th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2024, pp. 521–526.
- [251] Y. Bai, H. Zhou, K. Zhao, J. Chen, J. Yu, and K. Wang, "Transformer-OPU: An FPGA-based overlay processor for transformer networks," in *Proc. IEEE 31st Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, May 2023, pp. 221–221.
- [252] H. Chen et al., "Understanding the potential of FPGA-based spatial acceleration for large language model inference," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, no. 1, pp. 1–29, Mar. 2025.
- [253] Y. Mao et al., "CFEACT: A CGRA-based framework enabling agile CNN and transformer accelerator design," in *Proc. 34th Int. Conf. Field-Program. Log. Appl. (FPL)*, Sep. 2024, pp. 213–219.
- [254] C. Tan, D. Patil, A. Tumeo, G. Weisz, S. Reinhardt, and J. Zhang, "VecPAC: A vectorizable and precision-aware CGRA," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [255] T. Kojima, A. Ohwada, and H. Amano, "Mapping-aware kernel partitioning method for CGRAs assisted by deep learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 5, pp. 1213–1230, May 2022.
- [256] B. Li, L. Ying-Guang, L. G.-B. Fu, P. Ye, and J. Hu, "A method of accelerating CNN computation on DSP," *J. Phys. Conf. Ser.*, vol. 1684, no. 1, Nov. 2020, Art. no. 012063.
- [257] W. Niu, J. Guan, X. Shen, Y. Wang, G. Agrawal, and B. Ren, "GCD2: A globally optimizing compiler for mapping DNNs to mobile DSPs," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2022, pp. 512–529.
- [258] A. Xie, Y. Hu, A. Cheng, Z. Tang, P. Liu, and X. Zhang, "Advancing matrix decomposition efficiency: A study on FT-matrix DSP based SVD optimization," in *Proc. IEEE 10th Int. Conf. Cyber Secur. Cloud Comput. (CSCloud)/IEEE 9th Int. Conf. Edge Comput. Scalable Cloud (EdgeCom)*, Jul. 2023, pp. 464–469.
- [259] Y. Wu, Z. Wang, and W. D. Lu, "PIM GPT a hybrid process in memory accelerator for autoregressive transformers," *Npj Unconventional Comput.*, vol. 1, no. 1, pp. 1–13, Jul. 2024.
- [260] R. Guo et al., "CIMFormer: A systolic CIM-array-based transformer accelerator with token-pruning-aware attention reformulating and principal possibility gathering," *IEEE J. Solid-State Circuits*, vol. 59, no. 10, pp. 3317–3329, Oct. 2024.
- [261] S. Liu et al., "HARDSEA: Hybrid analog-ReRAM clustering and digital-SRAM in-memory computing accelerator for dynamic sparse self-attention in transformer," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 2, pp. 269–282, Feb. 2024.

- [262] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, "SwiftTron: An efficient hardware accelerator for quantized transformers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jun. 2023, pp. 1–9.
- [263] Y. Qin et al., "Ayaka: A versatile transformer accelerator with low-rank estimation and heterogeneous dataflow," *IEEE J. Solid-State Circuits*, vol. 59, no. 10, pp. 3342–3356, Oct. 2024.
- [264] M.-G. Lin, J.-P. Wang, Y.-J. Luo, and A.-Y. A. Wu, "A 28nm 64.5 TOPS/W sparse transformer accelerator with partial product-based speculation and sparsity-adaptive computation," in *Proc. IEEE Asia Pac. Conf. Circuits Syst. (APCCAS)*, Jun. 2024, pp. 664–668.
- [265] N. Hajinazar et al., "SIMDRAM: A framework for bit-serial SIMD processing using DRAM," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2021, pp. 329–345.
- [266] S. Roy, M. Ali, and A. Raghunathan, "PIM-DRAM: Accelerating machine learning workloads using processing in commodity DRAM," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 701–710, Dec. 2021.
- [267] Y. An et al., "StreamPIM: Streaming matrix computation in racetrack memory," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 297–311.
- [268] M. He et al., "Newton: A DRAM-maker's accelerator-in-memory (AiM) architecture for machine learning," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2020, pp. 372–385.
- [269] Y.-C. Kwon et al., "A 20nm 6GB function-in-memory dram, based on HBM2 with a 1.2TFLOPS programmable computing unit using bank-level parallelism, for machine learning applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 64, Oct. 2021, pp. 350–352.
- [270] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future PIM architectures by demystifying a commercial PIM technology," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 263–279.
- [271] P. B. Upama et al., "Evolution of quantum computing: A systematic survey on the use of quantum computing tools," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jun. 2022, pp. 520–529.
- [272] M. Broughton et al., "TensorFlow quantum: A software framework for quantum machine learning," 2020, *arXiv:2005.32164*.
- [273] H. Zhu et al., "Lightening-transformer: A dynamically-operated optically-interconnected photonic transformer accelerator," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 686–703.
- [274] G. Desoli et al., "A 40–310TOPS/W SRAM-based all-digital up to 4b in-memory computing multi-tiled NN accelerator in FD-SOI 18 nm for deep-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Sep. 2023, pp. 260–262.
- [275] S. Kim, S. Kim, S. Um, S. Kim, K. Kim, and H.-J. Yoo, "Neuro-CIM: A 310.4 TOPS/W neuromorphic computing-in-memory processor with low WL/BL activity and digital-analog mixed-mode neuron firing," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 38–39.
- [276] W. Cao, Y. Zhao, A. Boloor, Y. Han, X. Zhang, and L. Jiang, "Neural-PIM: Efficient processing-in-memory with neural approximation of peripherals," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2142–2155, Sep. 2022.
- [277] R. Khaddam-Aljameh et al., "HERMES-core—A 1.59-TOPS/mm² PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs," *IEEE J. Solid-State Circuits*, vol. 57, no. 4, pp. 1027–1038, Apr. 2022.
- [278] K. Prabhu et al., "CHIMERA: A 0.92-TOPS, 2.2-TOPS/W edge AI accelerator with 2-MByte on-chip foundry resistive RAM for efficient training and inference," *IEEE J. Solid-State Circuits*, vol. 57, no. 4, pp. 1013–1026, Apr. 2022.
- [279] W. Tang et al., "Arvon: A heterogeneous system-in-package integrating FPGA and DSP chiplets for versatile workload acceleration," *IEEE J. Solid-State Circuits*, vol. 59, no. 4, pp. 1235–1245, Apr. 2024.
- [280] Y. Li, A. Louri, and A. Karanth, "SPRINT: A high-performance, energy-efficient, and scalable chiplet-based accelerator with photonic interconnects for CNN inference," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2332–2345, Oct. 2022.
- [281] B. Zimmer et al., "A 0.32–128 TOPS, scalable multi-chip-module-based deep neural network inference accelerator with ground-referenced signaling in 16 nm," *IEEE J. Solid-State Circuits*, vol. 55, no. 4, pp. 920–932, Apr. 2020.
- [282] Z. Tan, Y. Wu, Y. Zhang, H. Shi, W. Zhang, and K. Ma, "A scalable multi-chiplet deep learning accelerator with hub-side 2.5D heterogeneous integration," in *Proc. IEEE Hot Chips 35 Symp. (HCS)*, Aug. 2023, pp. 1–17.
- [283] A. Smith et al., "11.1 AMD Instinct™ MI300 series modular chiplet package—HPC and AI accelerator for exa-class systems," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2024, pp. 490–492.
- [284] E. Talpes et al., "The microarchitecture of DOJO, Tesla's exa-scale computer," *IEEE Micro*, vol. 43, no. 3, pp. 31–39, May 2023.
- [285] M. Davies et al., "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proc. IEEE*, vol. 109, no. 5, pp. 911–934, May 2021.
- [286] H. A. Gonzalez et al., "SpiNNaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning," 2024, *arXiv:2401.04491*.
- [287] G. Wang, S. Ma, Y. Wu, J. Pei, R. Zhao, and L. Shi, "End-to-end implementation of various hybrid neural networks on a cross-paradigm neuromorphic chip," *Frontiers Neurosci.*, vol. 15, Feb. 2021, Art. no. 615279.
- [288] Z. Wang et al., "A hybrid heterogeneous neural network accelerator based on systolic array," in *Proc. IEEE 6th Int. Conf. AI Circuits Syst. (AICAS)*, Apr. 2024, pp. 154–158.
- [289] J. Seekings, P. Chandarana, M. Ardakani, M. Mohammadi, and R. Zand, "Towards efficient deployment of hybrid SNNs on neuromorphic and edge AI hardware," in *Proc. Int. Conf. Neuromorph. Syst. (ICONS)*, Jul. 2024, pp. 71–77.
- [290] S. Lu and A. Sengupta, "Hybrid neuromorphic systems: An algorithm-application-hardware-neuroscience co-design perspective: Invited special session paper," in *Proc. IEEE 4th Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Jun. 2022, pp. 210–213.
- [291] M. Verhelst, M. Shi, and L. Mei, "ML processors are going multi-core: A performance dream or a scheduling nightmare?," *IEEE Solid State Circuits Mag.*, vol. 14, no. 4, pp. 18–27, Fall. 2022.
- [292] X. Di, H.-G. Yang, Y. Jia, Z. Huang, and N. Mao, "Exploring efficient acceleration architecture for winograd-transformed transposed convolution of GANs on FPGAs," *Electronics*, vol. 9, no. 2, p. 286, Feb. 2020.
- [293] S. Kim, S. Kang, D. Han, S. Kim, S. Kim, and H.-J. Yoo, "An energy-efficient GAN accelerator with on-chip training for domain specific optimization," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2020, pp. 1–4.
- [294] T. Yin, W. Mao, J. Lu, and Z. Wang, "A reconfigurable accelerator for generative adversarial network training based on FPGA," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2021, pp. 144–149.
- [295] P. Yang, W. Mao, J. Lin, and Z. Wang, "A computation-efficient solution for acceleration of generative adversarial network," in *Proc. 18th IEEE Int. New Circuits Syst. Conf. (NEWCAS)*, Jun. 2020, pp. 210–213.
- [296] S. Sarkar et al., "CFD surrogates for data center sustainability using 3D U-Net convolutional neural network," in *Proc. 23rd IEEE Intersociety Conf. Thermal Thermomech. Phenomena Electron. Syst. (ITHERM)*, May 2024, pp. 1–9.
- [297] N. Flynn and X. Qian, "Heat sink design optimization via GAN-CNN combined deep-learning," in *Proc. ASME Des. Eng. Tech. Conf. (DETC)*, vol. 87301, 2023, pp. 1–9.
- [298] Z. Du, K. Chen, S. Chen, J. He, X. Zhu, and X. Jin, "Deep learning GAN-based data generation and fault diagnosis in the data center HVAC system," *Energy Buildings*, vol. 289, Jun. 2023, Art. no. 113072.
- [299] M. Wood, "High current density power modules mitigate the environmental impact of power-intensive GenAI," *IEEE Power Electron. Mag.*, vol. 11, no. 2, pp. 20–25, Jun. 2024.
- [300] S. Ilager, K. Ramamohanarao, and R. Buyya, "Thermal prediction for efficient energy management of clouds using machine learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1044–1056, May 2021.
- [301] K. Qian et al., "Alibaba HPN: A data center network for large language model training," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 691–706.
- [302] C. Ji, Z. Zhang, X. You, and C. Zhang, "Pipelined implementations for belief propagation polar decoder: From formula to hardware," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, Oct. 2019, pp. 248–253.
- [303] C. Ji, Y. Shen, Z. Zhang, X. You, and C. Zhang, "Autogeneration of pipelined belief propagation polar decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1703–1716, Jul. 2020.
- [304] Z. Zhong, X. You, and C. Zhang, "Auto-generation of pipelined hardware designs for polar encoder," in *Proc. China Semiconductor Technol. Int. Conf. (CSTIC)*, Mar. 2018, pp. 1–4.

- [305] Z. Zhong, W. J. Gross, Z. Zhang, X. You, and C. Zhang, "Polar compiler: Auto-generator of hardware architectures for polar encoders," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 6, pp. 2091–2102, Jun. 2020.
- [306] W. Zhao, C. Li, Z. Ji, Y. You, X. You, and C. Zhang, "Automatic timing-driven top-level hardware design for digital signal processing," in *Proc. IEEE 15th Int. Conf. ASIC (ASICON)*, Oct. 2023, pp. 1–4.
- [307] W. Zhao et al., "Flexible high-level synthesis library for linear transformations," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 7, pp. 3348–3352, Jul. 2024.
- [308] C. Ji, Y. You, W. Qian, Y. Huang, and C. Zhang, "Fast construction and exploration of performance-cost design space for belief propagation polar decoders," *Sci. China Inf. Sci.*, vol. 68, no. 3, pp. 1–2, Mar. 2025.
- [309] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "RTLCode: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [310] B. Nadimi and H. Zheng, "A multi-expert large language model architecture for verilog code generation," 2024, *arXiv:2404.08029*.
- [311] M. DeLorenzo et al., "Make every move count: LLM-based high-quality RTL code generation using MCTS," 2024, *arXiv:2402.03289*.
- [312] S. Thakur et al., "VeriGen: A large language model for verilog code generation," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–31, May 2024.
- [313] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLML: An open-source benchmark for design RTL generation with large language model," in *Proc. 29th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2024, pp. 722–727.
- [314] A. Allam and M. Shalan, "RTL-repo: A benchmark for evaluating LLMs on large-scale RTL design projects," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [315] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Invited paper: VerilogEval: Evaluating large language models for verilog code generation," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–8.
- [316] P. Vijayaraghavan et al., "VHDL-eval: A framework for evaluating large language models in VHDL code generation," 2024, *arXiv:2406.04379*.
- [317] C. Batten, N. Pinckney, M. Liu, H. Ren, and B. Khailany, "PyHDL-eval: An LLM evaluation framework for hardware design using Python-embedded DSLs," in *Proc. ACM/IEEE Int. Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–17.
- [318] F. R. Kashanaki, M. Zakharov, and J. Renau, "HDL-eval benchmarking LLMs for multiple HDLs," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [319] M. DeLorenzo, V. Gohil, and J. Rajendran, "CreativEval: Evaluating creativity of LLM-based hardware code generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [320] P. Vijayaraghavan et al., "Chain-of-descriptions: Improving code LLMs for VHDL code generation and summarization," in *Proc. ACM/IEEE Int. Symp. Mach. Learn. CAD*, Sep. 2024, pp. 1–10.
- [321] K. Chang et al., "Data is all you need: Finetuning LLMs for chip design via an automated design-data augmentation framework," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [322] Y.-A. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. Lin, "MG-verilog: Multi-grained dataset towards enhanced LLM-assisted verilog generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [323] W. Fang et al., "Transferable presynthesis PPA estimation for RTL designs with data augmentation techniques," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 1, pp. 200–213, Jan. 2025.
- [324] D. Vungarala et al., "SA-DS: A dataset for large language model-driven AI accelerator design generation," 2024, *arXiv:2404.10875*.
- [325] Y. Liao, T. Adegbiya, R. Lysecky, and R. Tandon, "Skip the benchmark: Generating system-level high-level synthesis data using generative machine learning," 2024, *arXiv:2404.14754*.
- [326] A. Nakkab, S. Q. Zhang, R. Karri, and S. Garg, "Rome was not built in a single step: Hierarchical prompting for LLM-based chip design," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–11.
- [327] Y. Du, H. Deng, S. C. Liew, K. Chen, Y. Shao, and H. Chen, "The power of large language models for wireless communication system development: A case study on FPGA platforms," 2023, *arXiv:2307.07319*.
- [328] S. A. Sheikholeslam and A. Ivanov, "SynthAI: A multi agent generative AI framework for automated modular HLS design generation," 2024, *arXiv:2405.16072*.
- [329] K. Chang et al., "Improving large language model hardware generating quality through post-LLM search," in *Proc. Mach. Learn. Syst.*, Jul. 2023, pp. 1–13.
- [330] Y.-D. Tsai, M. Liu, and H. Ren, "RTLFixer: Automatically fixing RTL syntax errors with large language models," 2023, *arXiv:2311.16543*.
- [331] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2023, pp. 1–6.
- [332] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "AutoChip: Automating HDL generation using LLM feedback," 2023, *arXiv:2311.04887*.
- [333] K. Thorat et al., "Advanced large language model (LLM)-driven verilog development: Enhancing power, performance, and area optimization in code synthesis," 2023, *arXiv:2312.01022*.
- [334] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "BetterV: Controlled verilog generation with discriminative guidance," 2024, *arXiv:2402.03375*.
- [335] Y. Fu et al., "GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [336] T. Liu et al., "ChatChisel: Enabling agile hardware design with large language models," in *Proc. 2nd Int. Symp. Electron. Design Autom. (ISED)*, May 2024, pp. 710–716.
- [337] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel, "NI2spec: Interactively translating unstructured natural language to temporal logics with large language models," in *Proc. Lect. Notes Comput. Sci. (CAV)*, Jan. 2023, pp. 383–396.
- [338] K. Chang et al., "Natural language is not enough: Benchmarking multi-modal generative AI for verilog generation," 2024, *arXiv:2407.08473*.
- [339] S.-Z. Wong, G.-W. Wan, D. Liu, and X. Wang, "VGV: Verilog generation using visual capabilities of multi-modal large language models," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [340] L. Collini, S. Garg, and R. Karri, "C2HLS: Leveraging large language models to bridge the software-to-hardware design gap," 2024, *arXiv:2412.00214*.
- [341] K. Xu, G. L. Zhang, X. Yin, C. Zhuo, U. Schlichtmann, and B. Li, "Automated C/C++ program repair for high-level synthesis via large language models," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–9.
- [342] C. Xiong, C. Liu, H. Li, and X. Li, "HLS-Pilot: LLM-based high-level synthesis," 2024, *arXiv:2408.06810*.
- [343] Q. Zeng, Y. Wang, Z. Wang, and W. Mao, "An automated hardware design framework for various DNNs based on ChatGPT," in *Proc. IEEE 37th Int. Syst. Chip Conf. (SOCC)*, Sep. 2024, pp. 1–6.
- [344] X. Wang et al., "ChatCPU: An agile CPU design and verification platform with LLM," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [345] R. Moravej et al., "The graph's apprentice: Teaching an LLM low level knowledge for circuit quality estimation," 2024, *arXiv:2411.00843*.
- [346] P. Chang, T. Adegbiya, Y. Liao, C. Talarico, A. Li, and J. Roveda, "Deep inverse design for high-level synthesis," 2024, *arXiv:2407.08797*.
- [347] Q. Huang, C. Hong, J. Wawrzyniec, M. Subedar, and Y. S. Shao, "Learning a continuous and reconstructible latent space for hardware accelerator design," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, May 2022, pp. 277–287.
- [348] M. Mirka, M. F. France Pillois, G. Sassatelli, and A. Gamatié, "GANNoC: A framework for automatic generation of NoC topologies using generative adversarial networks," in *Proc. Drone Syst. Eng. Rapid Simul. Perform. Eval., Methods Tools Proc.*, Jan. 2021, pp. 51–58.
- [349] M. Mirka, M. France-Pillois, G. Sassatelli, and A. Gamatié, "A generative AI for heterogeneous network-on-chip design space pruning," in *Proc. Design Autom. Test Europe Conf. Exhib. (DATE)*, 2022, pp. 1135–1138.
- [350] R. Ranjan, S. Ullah, S. S. Sahoo, and A. Kumar, "SyFAXO-GeN: Synthesizing FPGA-based approximate operators with generative networks," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 402–409.
- [351] X. Zhou et al., "SeaDAG: Semi-autoregressive diffusion for conditional directed acyclic graph generation," 2024, *arXiv:2410.16119*.
- [352] X. Li, X. Li, L. Chen, X. Zhang, M. Yuan, and J. Wang, "Circuit transformer: A transformer that preserves logical equivalence," 2024, *arXiv:2403.13838*.
- [353] X. Li, X. Li, L. Chen, X. Zhang, M. Yuan, and J. Wang, "Logic synthesis with generative deep neural networks," 2024, *arXiv:2406.04699*.

- [354] S. Fernando, R. Kunzelmann, D. S. Lopera, J. A. Halabi, and W. Ecker, "Leveraging large language models for the automated documentation of hardware designs," in *Proc. 13th Medit. Conf. Embedded Comput. (MECO)*, Jun. 2024, pp. 1–6.
- [355] W. Fu et al., "Hardware Phi-1.5B: A large language model encodes hardware domain specific knowledge," in *Proc. 29th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2024, pp. 349–354.
- [356] H. Wu et al., "ChatEDA: A large language model powered autonomous agent for EDA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 10, pp. 3184–3197, Oct. 2024.
- [357] A. Sharma, T.-D. Ene, K. Kunal, M. Liu, Z. Hasan, and H. Ren, "Assessing economic viability: A comparative analysis of total cost of ownership for domain-adapted large language models versus state-of-the-art counterparts in chip design coding assistance," 2024, *arXiv:2404.08850*.
- [358] M. Tomlinson, J. Li, and A. S. Andreou, "Designing silicon brains using LLM: Leveraging ChatGPT for automated description of a spiking neuron array," in *Proc. Argentine Conf. Electron. (CAE)*, Mar. 2024, pp. 154–159.
- [359] P. Vitolo, G. Psaltakis, M. Tomlinson, G. Domenico Licciardo, and A. G. Andreou, "Natural language to verilog: Design of a recurrent spiking neural network using large language models and ChatGPT," 2024, *arXiv:2405.01419*.
- [360] W. Salcedo, S. Achour, and C. McBeth, "Leveraging generative AI for rapid design and verification of a vector processor SoC," *IEEE Design Test*, vol. 41, no. 6, pp. 8–18, Dec. 2024.
- [361] R. Xu, S. Ma, Y. Wang, Y. Guo, D. Li, and Y. Qiao, "Heterogeneous systolic array architecture for compact CNNs hardware accelerators," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2860–2871, Nov. 2022.
- [362] C. Lv, Z. Wei, W. Qian, J. Ye, F. Chang, and Z. He, "GPT-LS: Generative pre-trained transformer with offline reinforcement learning for logic synthesis," in *Proc. IEEE Int. Conf. Comput. Des. VLSI Comput. Process. (ICCD)*, Nov. 2023, pp. 320–326.
- [363] A. Ghose et al., "Use cases and deployment of ML in IC physical design," in *Proc. 30th Asia South Pacific Design Autom. Conf.*, Jan. 2025, pp. 669–675.
- [364] T. H. Teo, M. Xiang, E. Goh, and H.-K. Hsu, "Open-AI driven open-source open-access sustainable ICs design flow," in *Proc. IEEE 17th Int. Symp. Embedded Multicore/Many-Core Syst.-Chip (MCSoc)*, Dec. 2024, pp. 361–365.
- [365] Y.-C. Lu, H. Ren, H.-H. Hsiao, and S. K. Lim, "DREAM-GAN: Advancing DREAMPlace towards commercial-quality using generative adversarial learning," in *Proc. Int. Symp. Phys. Des. (ISPD)*, Jul. 2023, pp. 141–148.
- [366] Y.-C. Lu, H. Ren, H.-H. Hsiao, and S. K. Lim, "GAN-place: Advancing open source placers to commercial-quality using generative adversarial networks and transfer learning," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 2, pp. 1–17, Mar. 2024.
- [367] Y.-C. Lu, J. Lee, A. Agnesina, K. Samadi, and S. K. Lim, "GAN-CTS: A generative adversarial framework for clock tree prediction and optimization," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2019, pp. 1–8.
- [368] Y.-C. Lu, J. Lee, A. Agnesina, K. Samadi, and S. K. Lim, "A clock tree prediction and optimization framework using generative adversarial learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 3104–3117, Sep. 2022.
- [369] R. Liang, A. Agnesina, G. Pradipta, V. A. Chhabria, and H. Ren, "Invited paper: CircuitOps: An ML infrastructure enabling generative AI for VLSI circuit optimization," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–6.
- [370] D. Utyamishv and I. Partin-Vaisband, "Multiterminal pathfinding in practical VLSI systems with deep neural networks," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 4, pp. 1–19, Jul. 2023.
- [371] S. Park, D. Kim, S. Kwon, and S. Kang, "Routability prediction and optimization using explainable AI," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–8.
- [372] R. Liang, S. Nath, A. Rajaram, J. Hu, and H. Ren, "BufFormer: A generative ML framework for scalable buffering," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 264–270.
- [373] P. Mangalagiri et al., "CDLS: Constraint driven generative AI framework for analog layout synthesis," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [374] E. I. Baugarten-Leon, S. Ortega-Cisneros, M. Abdelmoneum, R. Y. V. Morales, and G. Pinedo-Diaz, "The genesis of AI by AI integrated circuit: Where AI creates AI," *Electronics*, vol. 13, no. 9, p. 1704, Apr. 2024.
- [375] H.-H. Hsiao, Y.-C. Lu, P. Vanna-Iampikul, and S. K. Lim, "FastTuner: Transferable physical design parameter optimization using fast reinforcement learning," in *Proc. Int. Symp. Phys. Design*, Mar. 2024, pp. 93–101.
- [376] J. Zou, X. Wang, J. Guo, W. Liu, Q. Zhang, and C. Huang, "Circuit as set of points," in *Proc. Adv. Neural Inf. Proces. Syst. (NeurIPS)*, vol. 36, Jan. 2023, pp. 32468–32480.
- [377] Y.-C. Lu, S. Nath, V. Khandelwal, and S. K. Lim, "Doomed run prediction in physical design by exploiting sequential flow and graph learning," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–9.
- [378] D. Kasprowicz and G. Kasprowicz, "Generative modeling of semiconductor devices for statistical circuit simulation," *Electronics*, vol. 13, no. 11, p. 2003, May 2024.
- [379] C. Zhao et al., "Generative AI for secure physical layer communications: A survey," *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 1, pp. 3–26, Feb. 2025.
- [380] J. Wang et al., "Generative AI for integrated sensing and communication: Insights from the physical layer perspective," *IEEE Wireless Commun.*, vol. 31, no. 5, pp. 246–255, Oct. 2024.
- [381] A. B. Kahng, "Solvers, engines, tools and flows: The next wave for AI/ML in physical design," in *Proc. Int. Symp. Phys. Design*, Mar. 2024, pp. 117–124.
- [382] R. Qiu, G. Li Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design," 2024, *arXiv:2407.03891*.
- [383] R. Ma et al., "VerilogReader: LLM-aided hardware test generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [384] K. Qayyum, M. Hassan, S. Ahmadi-Pour, C. K. Jha, and R. Drechsler, "From bugs to fixes: HDL bug identification and patching using LLMs and RAG," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
- [385] J. Bhandari, J. Knechtel, R. Narayanaswamy, S. Garg, and R. Karri, "LLM-aided testbench generation and bug detection for finite-state machines," 2024, *arXiv:2406.17132*.
- [386] M. Liu, M. Kang, G. B. Hamad, S. Suhaib, and H. Ren, "Domain-adapted LLMs for VLSI design and verification: A case study on formal verification," in *Proc. IEEE 42nd VLSI Test Symp. (VTS)*, Apr. 2024, pp. 1–4.
- [387] M. Hassan, S. Ahmadi-Pour, K. Qayyum, C. K. Jha, and R. Drechsler, "LLM-guided formal verification coupled with mutation testing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–2.
- [388] D. Schwachhofer et al., "Training large language models for system-level test program generation targeting non-functional properties," in *Proc. IEEE Eur. Test Symp. (ETS)*, May 2024, pp. 1–4.
- [389] C. Xiao et al., "LLM-based processor verification: A case study for neuromorphic processor," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.
- [390] L. Francisco and S. Arikati, "LLM based physical verification run-set generator," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–7.
- [391] C.-C. Chang, C.-T. Ho, Y. Li, Y. Chen, and H. Ren, "DRC-coder: Automated DRC checker code generation using LLM autonomous agent," 2024, *arXiv:2412.05311*.
- [392] S. Wang, Z. Zhong, Y. Zhao, and L. Zuo, "A variational autoencoder enhanced deep learning model for wafer defect imbalanced classification," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 11, no. 12, pp. 2055–2060, Dec. 2021.
- [393] Y.-Y. Bao, E.-C. Li, H.-Q. Yang, and B.-B. Jia, "Wafer map defect classification using autoencoder-based data augmentation and convolutional neural network," 2024, *arXiv:2411.11029*.
- [394] D. Kim and P. Kang, "Dynamic clustering for wafer map patterns using self-supervised learning on convolutional autoencoders," *IEEE Trans. Semicond. Manuf.*, vol. 34, no. 4, pp. 444–454, Nov. 2021.
- [395] P. Tulala, H. Mahyar, E. Ghalebi, and R. Grosu, "Unsupervised wafermap patterns clustering via variational autoencoders," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–8.
- [396] J. Hwang and H. Kim, "Variational deep clustering of wafer map patterns," *IEEE Trans. Semicond. Manuf.*, vol. 33, no. 3, pp. 466–475, Aug. 2020.
- [397] J. Yu, "Enhanced stacked denoising autoencoder-based feature learning for recognition of wafer map defects," *IEEE Trans. Semicond. Manuf.*, vol. 32, no. 4, pp. 613–624, Nov. 2019.
- [398] J. Yu and J. Liu, "Two-dimensional principal component analysis-based convolutional autoencoder for wafer map defect detection," *IEEE Trans. Ind. Electron.*, vol. 68, no. 9, pp. 8789–8797, Sep. 2021.

- [399] H. Hu, N. Nguyen, C. He, and P. Li, "Advanced outlier detection using unsupervised learning for screening potential customer returns," in *Proc. IEEE Int. Test Conf. (ITC)*, Nov. 2020, pp. 1–10.
- [400] Y. Liao, Z. P. Najafi-Haghi, H.-J. Wunderlich, and B. Yang, "Efficient and robust resistive open defect detection based on unsupervised deep learning," in *Proc. IEEE Int. Test Conf. (ITC)*, Sep. 2022, pp. 185–193.
- [401] J. Wang, Z. Yang, J. Zhang, Q. Zhang, and W. K. Chien, "AdaBalGAN: An improved generative adversarial network with imbalanced learning for wafer defective pattern recognition," *IEEE Trans. Semicond. Manuf.*, vol. 32, no. 3, pp. 310–319, Aug. 2019.
- [402] H. Mahyar, P. Tulala, E. Ghalebi, and R. Grosu, "DeepWafer: A generative wafermap model with deep adversarial networks," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2022, pp. 126–131.
- [403] S. Park and C. You, "Deep convolutional generative adversarial networks-based data augmentation method for classifying class-imbalanced defect patterns in wafer bin map," *Appl. Sci.*, vol. 13, no. 9, p. 5507, Apr. 2023.
- [404] S.-N. Chen and S.-H. Chen, "Application of generative adversarial networks for virtual silicon data generation and design-technology co-optimization: A study on WAT and CP," *IEEE Access*, vol. 12, pp. 6532–6545, 2024.
- [405] J. Yu and J. Liu, "Multiple granularities generative adversarial network for recognition of wafer map defects," *IEEE Trans. Ind. Informat.*, vol. 18, no. 3, pp. 1674–1683, Mar. 2022.
- [406] C.-C. Lu et al., "Transformer and its variants for identifying good dice in bad neighborhoods," in *Proc. IEEE 42nd VLSI Test Symp. (VTS)*, Apr. 2024, pp. 1–7.
- [407] C. W. Lin et al., "Utilizing transformer deep learning based outlier detection to screen out reliability weak ICs," in *Proc. IEEE Int. Rel. Phys. Symp. (IRPS)*, Apr. 2024, pp. 1–5.
- [408] Y. Wang, Y. Wei, and H. Wang, "A class imbalanced wafer defect classification framework based on variational autoencoder generative adversarial network," *Meas. Sci. Technol.*, vol. 34, no. 2, Feb. 2023, Art. no. 024008.
- [409] L.-C. Wang, "LLM-assisted analytics in semiconductor test (invited)," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–7.
- [410] P. Girard, S. Blanton, and L.-C. Wang, *Machine Learning Support for Fault Diagnosis of System-on-Chip*. Cham, Switzerland: Springer, 2023.
- [411] K. Taha, "Observational and experimental insights into machine learning-based defect classification in wafers," 2023, *arXiv:2310.10705*.
- [412] M.-J. Wu, J.-S. Jang, and J.-L. Chen, "Wafer map failure pattern recognition and similarity ranking for large-scale data sets," *IEEE Trans. Semicond. Manuf.*, vol. 28, no. 1, pp. 1–12, Feb. 2015.
- [413] M. J. Yang, Y. J. Zeng, and L.-C. Wang, "WM-graph: Graph-based approach for wafermap analytics," in *Proc. IEEE Int. Test Conf. (ITC)*, Nov. 2024, pp. 111–120.
- [414] Y. J. Zeng, M. J. Yang, and L.-C. Wang, "Wafer map pattern analytics driven by natural language queries," in *Proc. IEEE Int. Test Conf. Asia (ITC-Asia)*, Aug. 2022, pp. 31–36.
- [415] K. Zhu et al., "Generative-adversarial-network-guided well-aware placement for analog circuits," in *Proc. 27th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2022, pp. 519–525.
- [416] B. Xu et al., "WellGAN: Generative-adversarial-network-guided well generation for analog/mixed-signal circuit layout," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [417] C. Liu, Y. Liu, Y. Du, and L. Du, "LADAC: Large language model-driven auto-designer for analog circuits," 2024, *techrxiv:170473941.10097233*.
- [418] Y. Lai et al., "AnalogCoder: Analog circuit design via training-free code generation," 2024, *arXiv:2405.14918*.
- [419] K. Zhu et al., "GeniusRoute: A new analog routing paradigm using generative neural network guidance," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2019, pp. 1–8.
- [420] K. Zhu, H. Chen, M. Liu, and D. Z. Pan, "Tutorial and perspectives on MAGICAL: A silicon-proven open-source analog IC layout system," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 2, pp. 715–720, Feb. 2023.
- [421] K. Kunal et al., "Align: Open-source analog layout automation from the ground up," in *Proc. 56th Annu. Design Autom. Conf. 2019*, 2019, pp. 1–4.
- [422] T. Dhar et al., "ALIGN: A system for automating analog layout," *IEEE Design Test*, vol. 38, no. 2, pp. 8–18, Apr. 2021.
- [423] G. E. Aliffi et al., "AMBEATion: Analog mixed-signal back-end design automation with machine learning and artificial intelligence techniques," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.
- [424] A. M. Rashed, Z. A. Abd-Elghafar, and I. Nashaat, "A complete automated analog flow from design generation to post-layout verification," in *Proc. 11th Int. Jpn.-Afr. Conf. Electron., Commun., Comput. (JAC-ECC)*, Dec. 2023, pp. 180–183.
- [425] C. Park, H. C. Nam, J. Park, and J. Jeon, "FlowSim: An invertible generative network for efficient statistical analysis under process variations," in *Proc. Int. Conf. Simul. Semiconductor Processes Devices (SISPAD)*, vol. 10242, Sep. 2023, pp. 157–160.
- [426] Y.-T. Chen, C.-F. Nien, C. Hsia, and C.-Y. Li, "Interactive analog IC layout tool with real-time parasitic-aware automatic routing assistance," in *Proc. IEEE Asia Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2024, pp. 149–153.
- [427] A. Hammoud et al., "Human language to analog layout using GLayout layout automation framework," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–7.
- [428] R. S. Gopalakrishnan et al., "A generalized methodology for well island generation and well-tap insertion in analog/mixed-signal layouts," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 5, pp. 1–25, Sep. 2023.
- [429] Y. Li et al., "A customized graph neural network model for guiding analog IC placement," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2020, pp. 1–9.
- [430] Z. Zhao and L. Zhang, "An automated topology synthesis framework for analog integrated circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4325–4337, Dec. 2020.



Chuan Zhang (Senior Member, IEEE) received the B.E. degree in microelectronics and the M.E. degree in very-large scale integration (VLSI) design from Nanjing University, Nanjing, China, in 2006 and 2009, respectively, and the Ph.D. degree from the Department of Electrical and Computer Engineering, University of Minnesota (UMN), Twin Cities, USA, in 2012.

He is currently a Young Chair Professor with Southeast University. He is also with LEADS, National Mobile Communications Research Laboratory, Frontiers Science Center for Mobile Information Communications and Security of MoE, Quantum Information Center, Southeast University; and Purple Mountain Laboratories, Nanjing. His current research interests include algorithms and implementations for signal processing and communication systems.

Dr. Zhang is a member of the Applied Signal Processing Systems TC of the IEEE Signal Processing Society, the Circuits and Systems for Communications TC, the VLSI Systems and Applications TC, and the Digital Signal Processing TC of the IEEE Circuits and Systems Society. He received the Best Contribution Award of the IEEE Asia Pacific Conference on Circuits and Systems (APCCAS) in 2018; the Best Paper Award in 2016; the Best (Student) Paper Award of the IEEE International Conference on DSP in 2016; three Best (Student) Paper Awards of the IEEE International Conference on ASIC in 2015, 2017, 2019, and 2023; the Best Paper Award Nomination of the IEEE Workshop on Signal Processing Systems in 2015; three Excellent Paper Awards and two Excellent Poster Presentation Awards of the International Collaboration Symposium on Information Production and Systems in 2016 and 2018; the Outstanding Achievement Award of the Intel Collaborative Research Institute in 2018; and the Merit (Student) Paper Award of the IEEE APCCAS in 2008. He also received the Three-Year University-Wide Graduate School Fellowship of UMN and the Doctoral Dissertation Fellowship of UMN. He serves as a Senior Editorial Board (SEB) Member for IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS, and a Steering Committee Member for IEEE TRANSACTIONS ON MOBILE COMPUTING. He served as an Associate Editor for IEEE TRANSACTIONS ON SIGNAL PROCESSING, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, and IEEE OPEN JOURNAL OF CIRCUITS AND SYSTEMS and a Corresponding Guest Editor for IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS (three times). He was a Distinguished Lecturer and is the Chair of the Circuits and Systems for Communications TC of the IEEE Circuits and Systems Society.



You You received the B.S. degree in information science and engineering from Southeast University in 2012, the double M.S.E. degrees in communications and signal processing from the University of Leeds and Nanjing University of Science and Technology, in 2015 and 2016, respectively, and the Ph.D. degree from the University of Leeds in 2021. He is currently an Associate Professor with Purple Mountain Laboratories, Nanjing, China. His research interests include algorithms and implementations for signal processing and communication systems.



Jongsun Park (Senior Member, IEEE) received the B.S. degree in electronics engineering from Korea University, Seoul, South Korea, in 1998, and the M.S. and Ph.D. degrees in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2000 and 2005, respectively.

He joined the Electrical Engineering Faculty, Korea University, in 2008. He was with the Digital Radio Processor System Design Group, Texas Instruments, Dallas, TX, USA. From 2005 to 2008, he was with the Signal Processing Technology Group, Marvell Semiconductor, Santa Clara, CA, USA, where he developed efficient hardware architectures of sophisticated digital signal processing and digital communication algorithms. His research interests focus on variation-tolerant, low-power, high-performance VLSI architectures and circuit designs for deep learning, and digital signal/image processing. He is a member of the Circuits and Systems for Communication Technical Committee of the IEEE Circuits and Systems Society. He has served in the technical program committees of various IEEE/ACM conferences, including ISCAS, DAC, ICCAD, ISLPED, ASP-DAC, HOST, and VLSI-SoC. He served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, IEEE OPEN JOURNAL OF CIRCUITS AND SYSTEMS, and *IEEE Circuits and Systems Magazine*; and a Guest Editor for IEEE TRANSACTIONS ON MULTI-SCALE COMPUTING SYSTEMS. He is an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR ARTIFICIAL INTELLIGENCE and IEEE SOLID STATE CIRCUIT LETTERS.



Naigang Wang received the B.S. degree in materials science and engineering from Jilin University in 1999 and the Ph.D. degree in materials science and engineering from the University of Florida in 2010. He is currently a Senior Research Scientist and the Manager with the IBM T. J. Watson Research Center. He has published over 80 journals and conference papers and filed more than 90 patents. His current research interests include advancing hardware-friendly machine learning algorithms, model architectures, and software-hardware

co-design for AI accelerators. He received the five IBM Outstanding Technical Achievement Awards. He has served with the research community in various capacities, including as the Area Chair for ICML, NeurIPS, and AAAI; and an Action Editor for *Transactions on Machine Learning Research*.



Li Zhang (Senior Member, IEEE) is an Associate Professor in communications with the School of Electronic and Electrical Engineering, University of Leeds. She leads the Wireless Communication Group, Institute of Communication and Power Networks (ICaPNet). Her group is also actively investigating wireless communication challenges in high-speed railway systems. She has been a member of U.K. EPSRC Peer Review College since 2006 and has served on grant review panels for both EPSRC and Royal Society. She also regularly reviews grant

proposals for international funding bodies, including Qatar National Research Fund, and agencies in Denmark and France; and book proposals for various academic publishers. She has acted as a Ph.D. examiner for several universities within and outside the U.K. She has served as an external examiner for M.Sc. programs at other U.K. institutions. Her research focuses on wireless communications, including massive MIMO, mmWave and THz communications, non-terrestrial networks (NTNs), mobile edge computing, and 5G/6G systems. She received the Nuffield Award for newly appointed lecturers in 2005 and became a Fellow of the Higher Education Academy in 2006. She has been served on the technical programme committees of most major IEEE communications conferences since 2006. She is currently an associate editor of several IEEE journals.